Apache Cassandra Database

1

Introduction

Let's start with some definitions.

NoSQL Databases

In general, they share the following features:

- Schema-free databases
- Easy replication support
- Simple API
- Distributed
- Open Source
- BASE (instead of ACID)
- Huge amount of data
- Horizontally scalable

Apache Cassandra

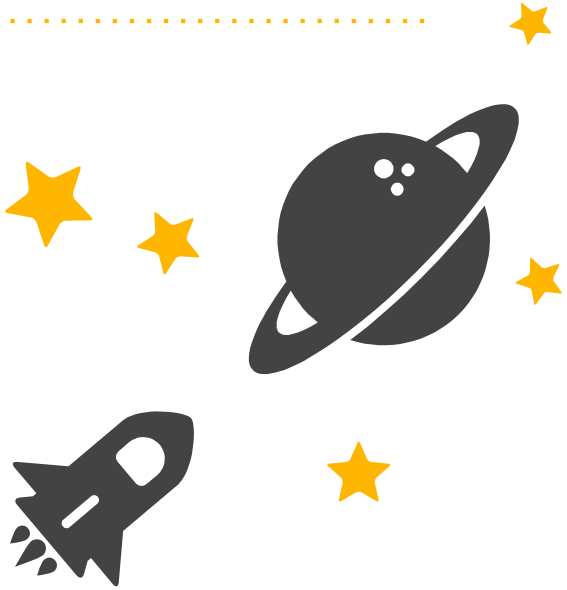
A distributed NoSQL database system for managing large amounts of structured data across many commodity servers, while providing highly available service and no single point of failure.

Caracteristics

- Data security
- Data sharing
- Physical independence
- Speed of access
- Verification of integrity
- Manipulability
- Limitation of the roundness

Cassandra support most of the General DBMS characteristics

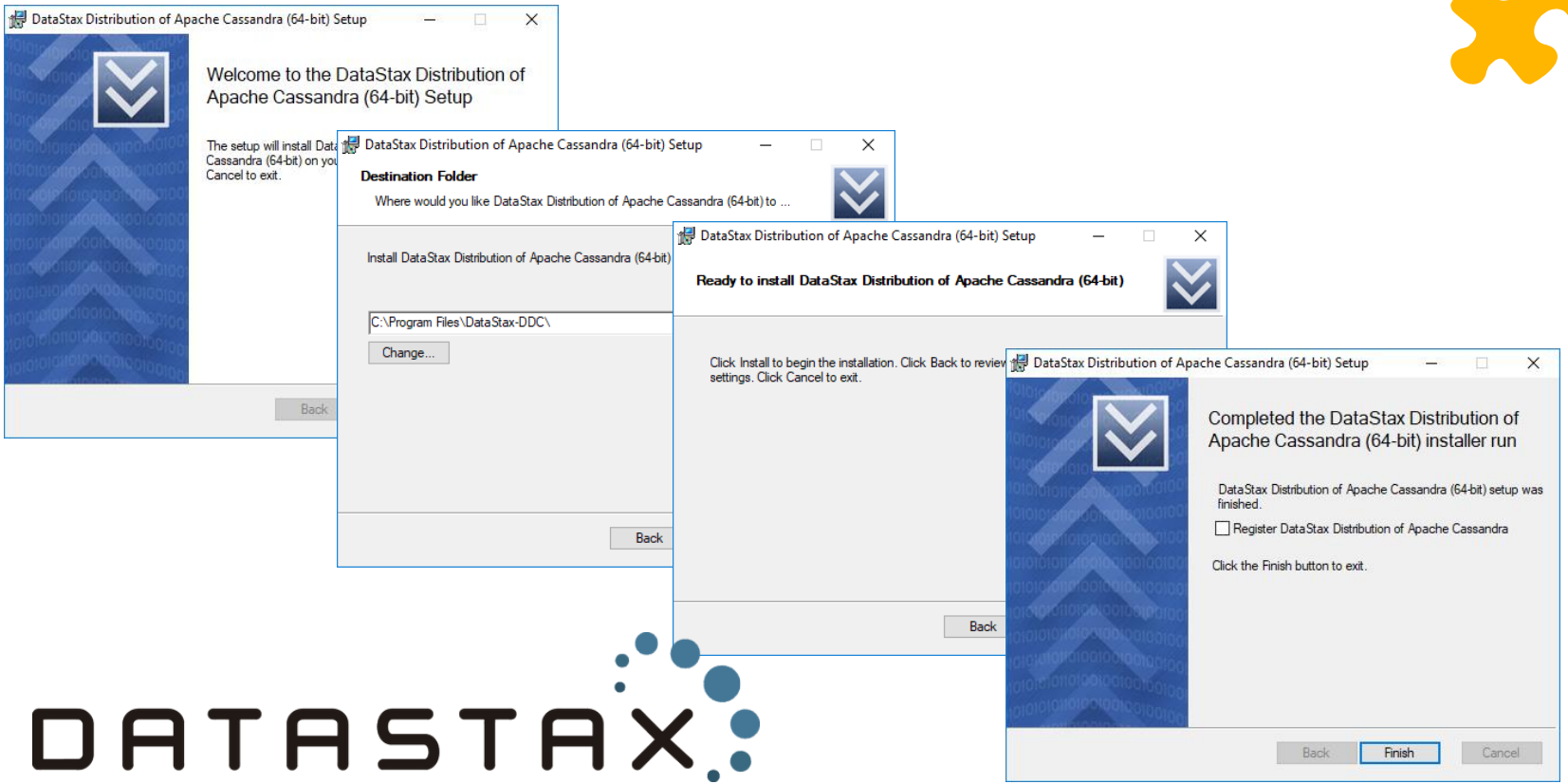
The Instalation



To strat using cassandra we need to set a workplace for it first.

Requirements:

- The latest version of Java 8
- The latest version of Python 2.7 or 3.6
- Download the Software (DataStax Community Edition for Apache Cassandra™)


DataStax Distribution of Apache Cassandra (64-bit) Setup
Welcome to the DataStax Distribution of Apache Cassandra (64-bit) Setup
The setup will install Data
Cassandra (64-bit) on you
Cancel to exit.
Back
DataStax Distribution of Apache Cassandra (64-bit) Setup
Destination Folder
Where would you like DataStax Distribution of Apache Cassandra (64-bit) to ...
Install DataStax Distribution of Apache Cassandra (64-bit)
C:\Program Files\DataStax-DDC\
Change...
Back
DataStax Distribution of Apache Cassandra (64-bit) Setup
Ready to install DataStax Distribution of Apache Cassandra (64-bit)
Click Install to begin the installation. Click Back to review
settings. Click Cancel to exit.
Back
DataStax Distribution of Apache Cassandra (64-bit) Setup
Completed the DataStax Distribution of Apache Cassandra (64-bit) installer run
DataStax Distribution of Apache Cassandra (64-bit) setup was finished.
Register DataStax Distribution of Apache Cassandra
Click the Finish button to exit.
Back
Finish
Cancel

DATASTAX

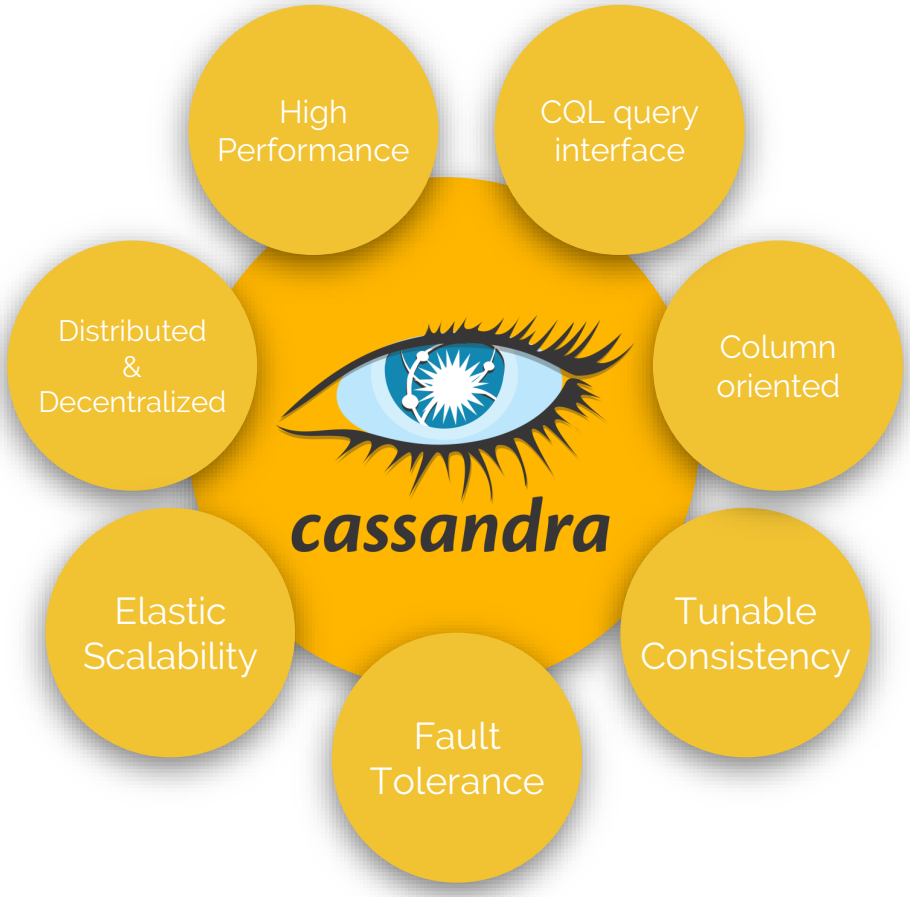
2

Key Principles

The “Must” Be understood of the cassandra

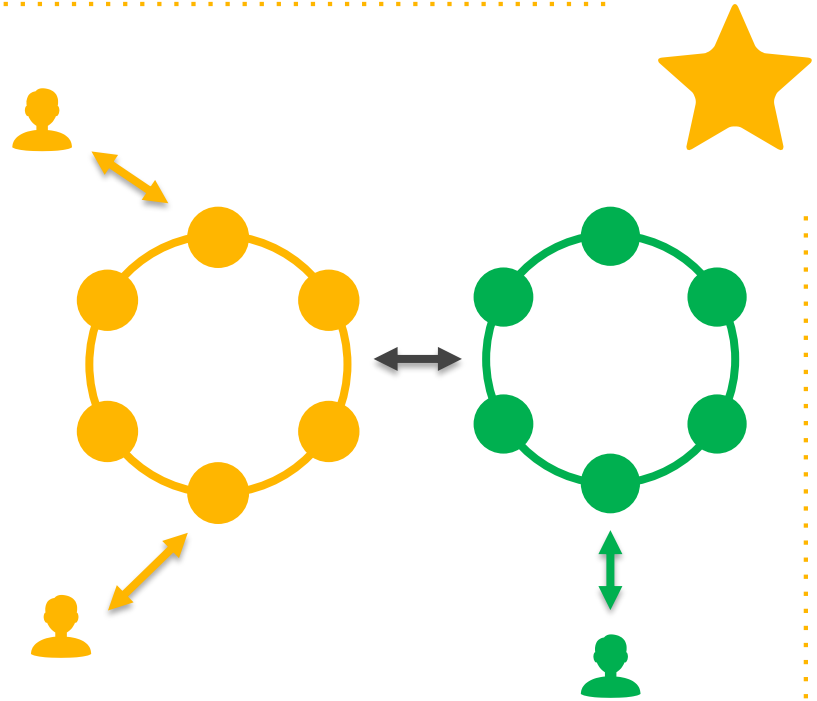
Key Features

This features makes the Cassandra Empire !



Distributed & Decentralized

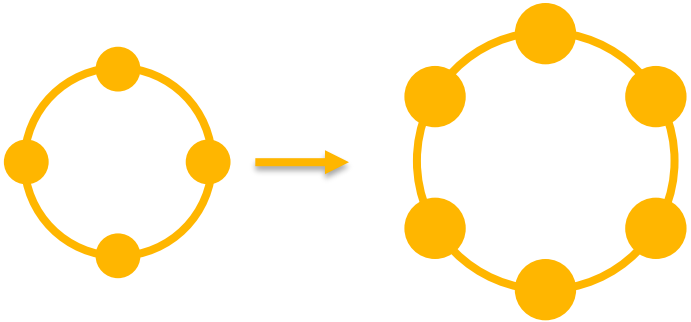
- Distributed: Capable of running on multiple machines
- Decentralized: No single point of failure
- No master-slave issues due to peer-to-peer architecture (protocol "gossip")



Read- and write-requests to any node

Elastic Scalability

- Cassandra scales horizontally, adding more machines that have all or some of the data on
- Adding of nodes increase performance throughput linearly
- Decreasing and increasing the node count happen seamlessly

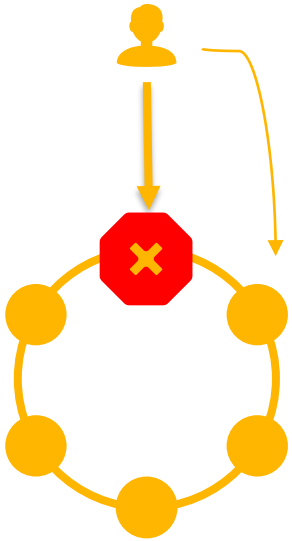


Linearly scales to terabytes and petabytes of data

High Availability & Fault Tolerance

High Availability?

- Multiple networked computers operating in a cluster
- Facility for recognizing node failures
- Forward failing over requests to another part of the system



No single point of failure due to the peer-to-peer architecture

Column oriented
Key-Value Store

- Data is stored in sparse multidimensional hash tables
- A row can have multiple columns not necessarily the same amount of columns for each row
- Each row has a unique key, which also determines partitioning

R1	**C1 Key**	**C2 Key**	**C3Key**	
	C1 Value	C3 Value	C3 Value	.....
R2	C4 Key	C5 Key		
	C4 Value	C5 Value		
.....				

No relations!

Cassandra Query Language

- *"CQL 3 is the default and primary interface into the Cassandra DBMS"*
- Familiar SQL-like syntax that maps to Cassandras storage engine and simplifies data modelling

"SQL-like" but NOT relational SQL

Cassandra Query Language

```
CRETE TABLE songs (
        Id uuid PRIMARY KEY, title text,
        Album text, Artist text,
        data blob );
```

```
SELECT * FROM songs
WHERE id = 'a3e64f8f...';
```

```
SELECT *  FROM songs ;
```

```
INSERT INTO songs (id, title, album, artist)
VALUES( 'a3e64f8f...', ‘Hazim ra3d', ‘Spacetoon', ‘Tarkan‘ );
```

Cassandra Query Language

```
INSERT INTO songs (id, title)
VALUES( 'a3e64f8f...', 'Al Kanas');
```

This is Possible With Cassandra

Cassandra Query Language

The resulting table in RDMBS is this:

id	title	artist	album	data
a3e64f8f…	Hazim Ra3d	Tarkan	Spacetoon	*null*
g617Dd23…	Al Kanas	*null*	*null*	*null*

Cassandra Query Language

The resulting table in Cassandra is this:

id	title	artist	album	data
a3e64f8f…	Hazim Ra3d	Tarkan	Spacetoon	
g617Dd23…	Al Kanas			

MySQL Comparision:

Statistics based on 50 GB Data

	Cassandra	MySQL
Average Write	**0.12 ms**	**~300 ms**
Average Read	**15 ms**	**~350 ms**

Stats provided by Authors using Facebook data.

And Much More…

The Data Model

How the Database is Organized ?

Data Model

Cluster:

Cassandra database is distributed over several machines that operate together. The outermost container is known as the Cluster. For failure handling, every node contains a replica, and in case of a failure, the replica takes charge. Cassandra arranges the nodes in a cluster, in a ring format, and assigns data to them.

Data Model

Keyspace

Outermost container for data (one or more column families), like database in RDBMS.

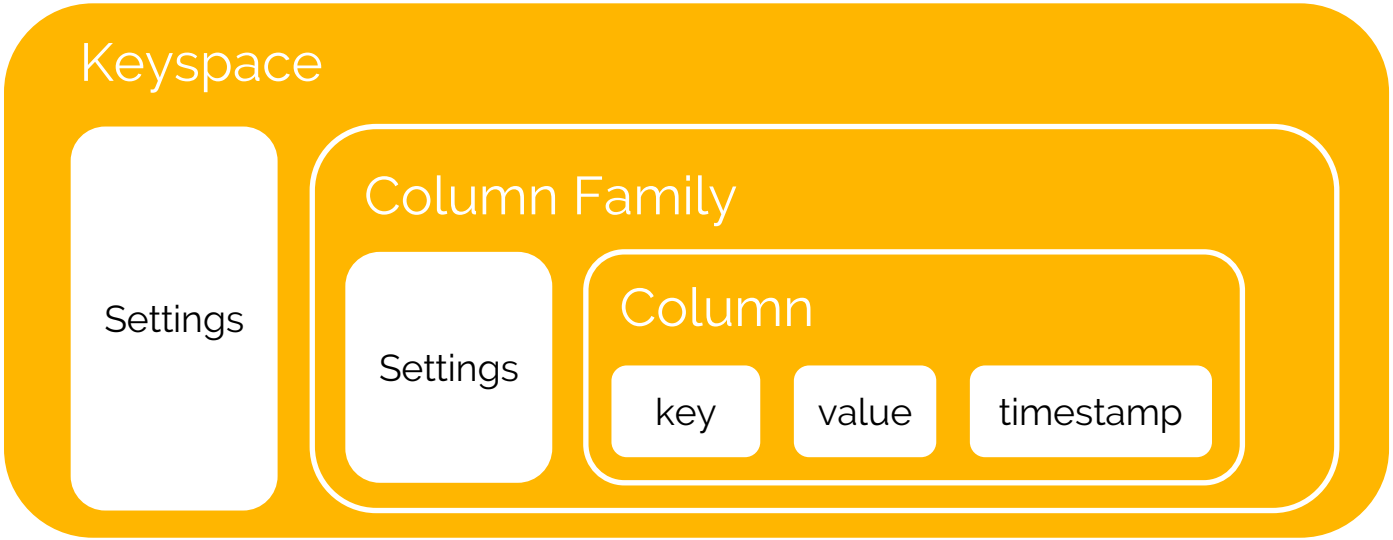
Column family

Contains Super columns or Columns (but not both).

Column

Basic data structures with: key, value, timestamp

Data Model


Keyspace
Settings
Column Family
Settings
Column
key
value
timestamp

Introduction to Cassandra

Russ Katz
Solutions Engineer(Central) @ DataStax

How do we handle massive transactional data and never go down?

DATASTAX

Google

Bigtable, 2006

amazon.com

Dynamo, 2007

Schema
Memtables
Compaction
SStables
Commit Log

facebook

Cluster Architecture
Partitioning
Replication
Gossip
Anti-Entropy
Hints

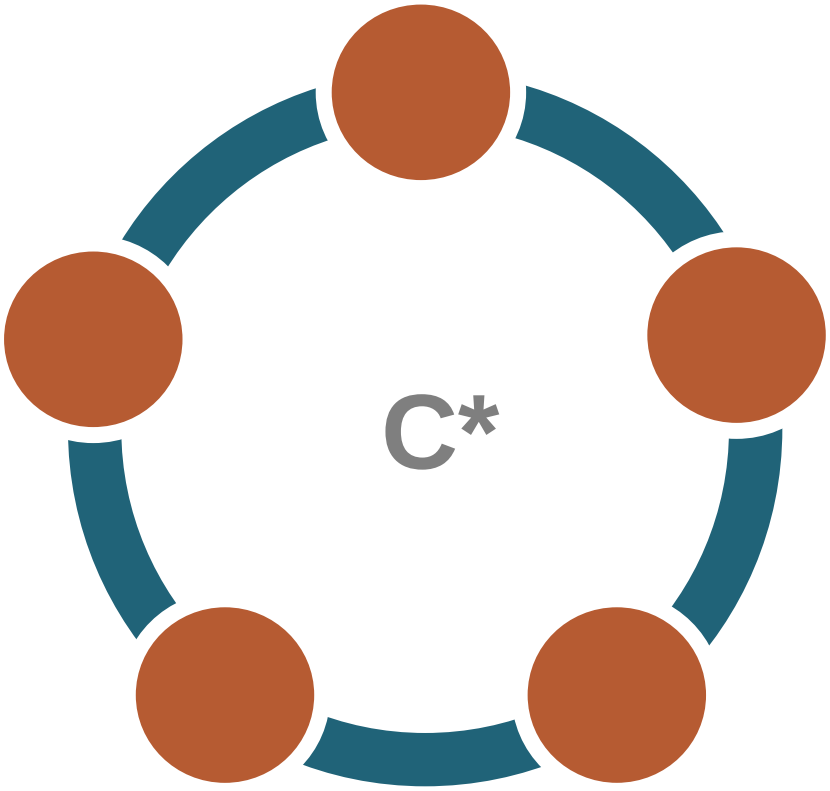
Cassandra

The Apache Software Foundation

What is Cassandra?

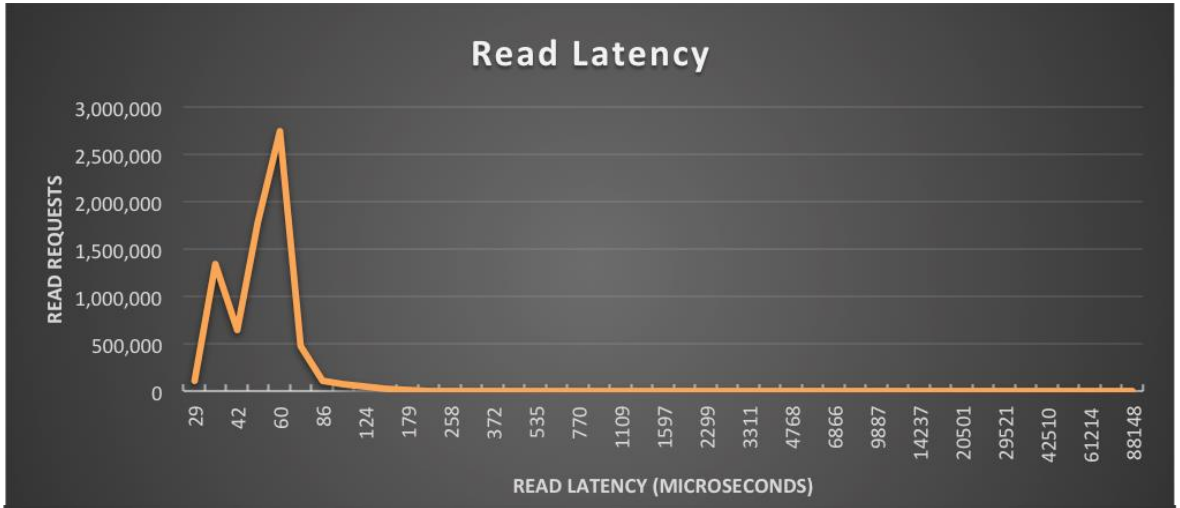
Distributed Database

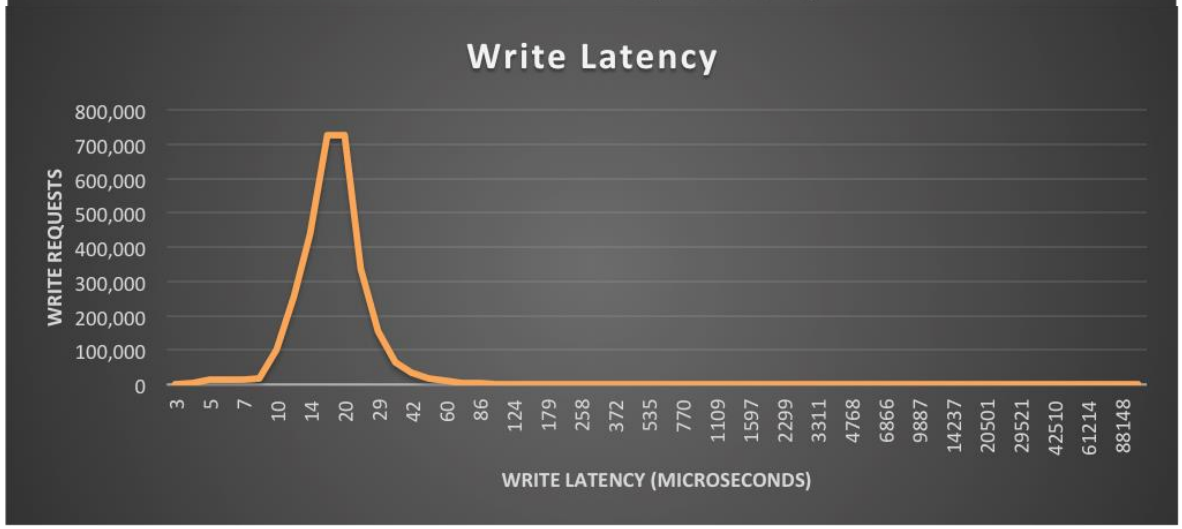
- ✓ Individual DBs (nodes)
- ✓ Working in a cluster
- ✓ Nothing is shared



Why Cassandra?

It’s Fast (Low Latency)





Why Cassandra?

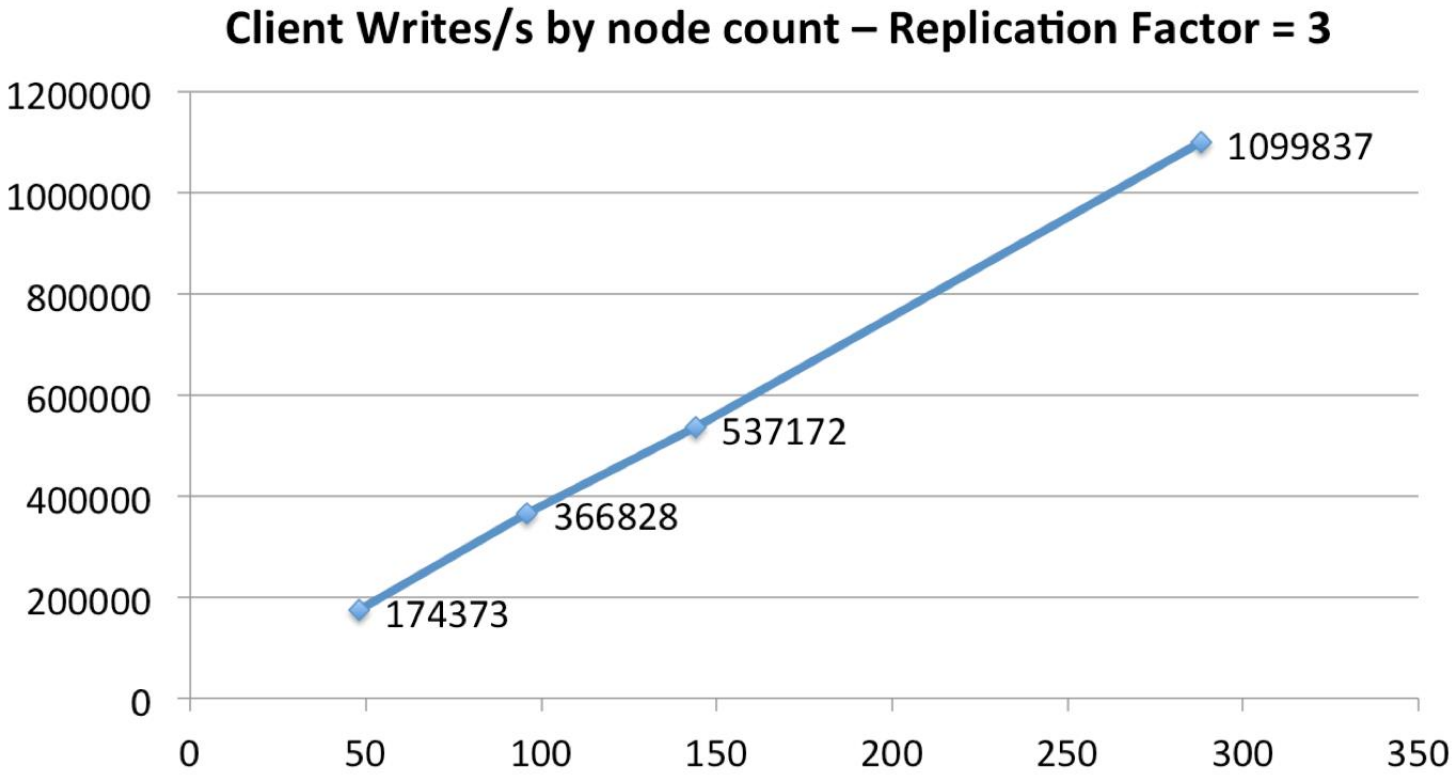
DATASTAX

It's Always On

NETFLIX

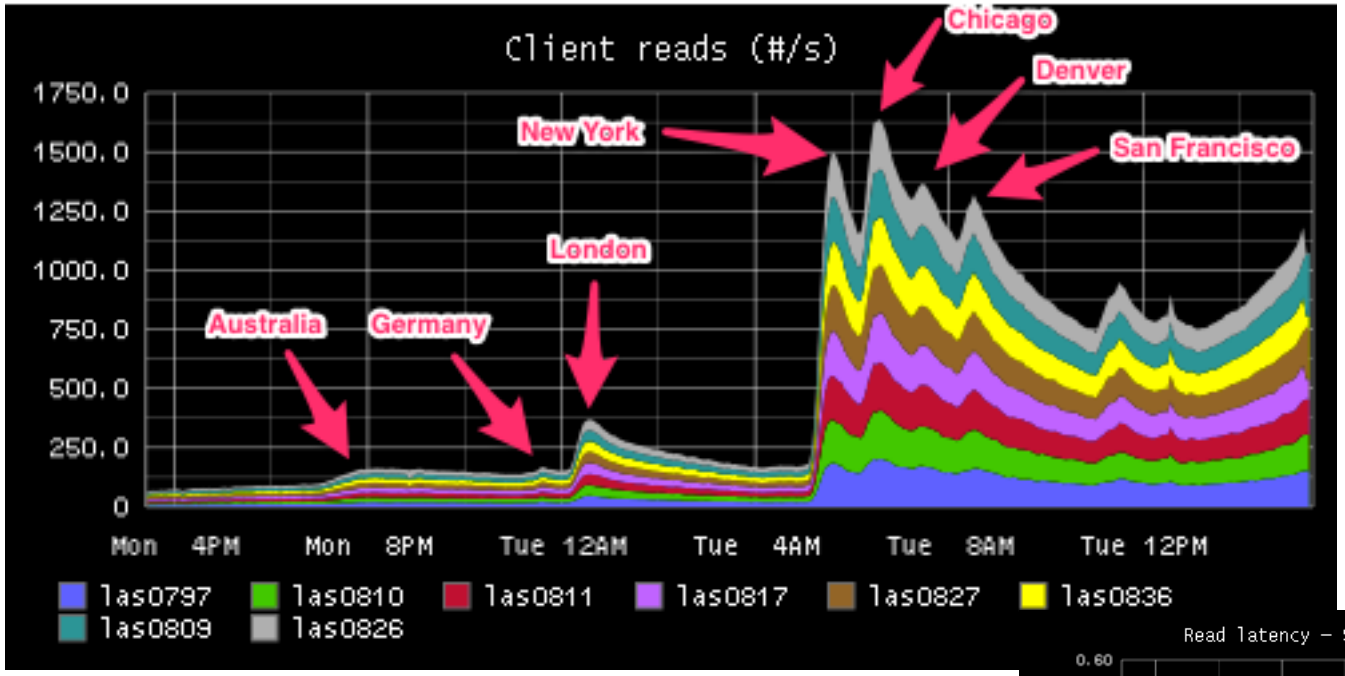
Why Cassandra?

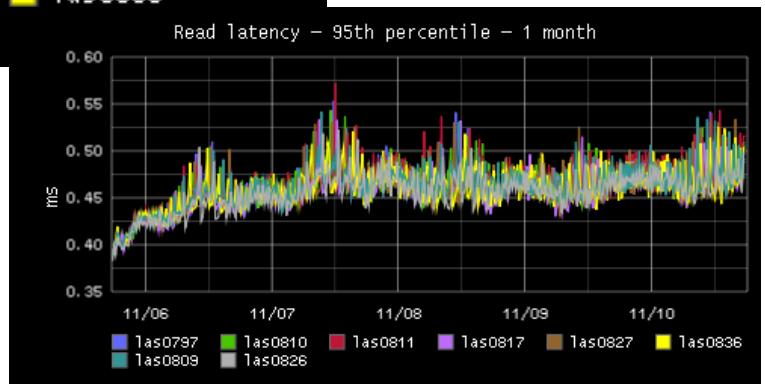
It's Hugely Scalable (High Throughput)



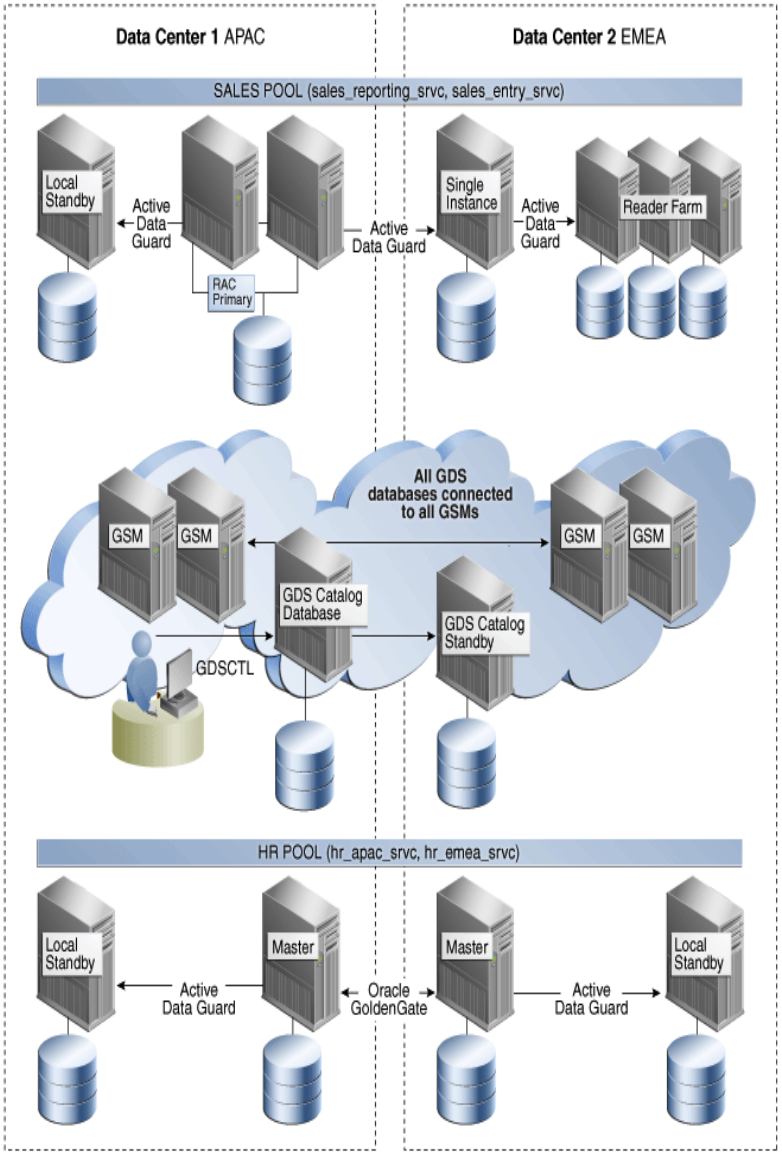
Why Cassandra?

It is natively multi-data center with distributed data

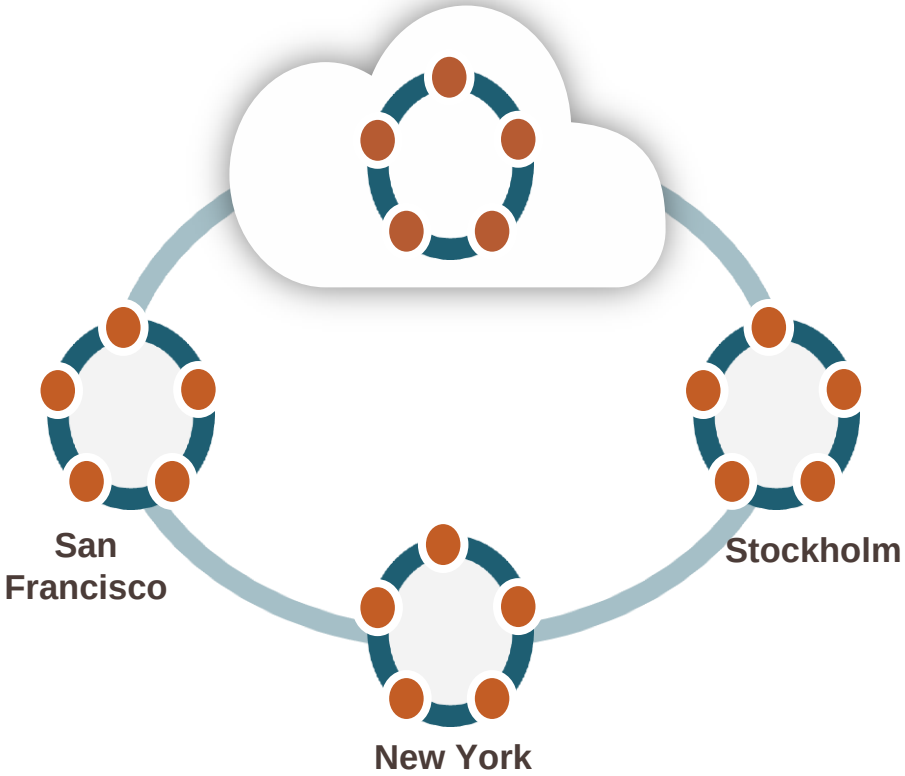




Operational Simplicity



OR



©2014 DataStax Confidential. Do not distribute without consent.

Why Cassandra?

DATASTAX

It has a flexible data model

Tables, wide rows, partitioned and distributed

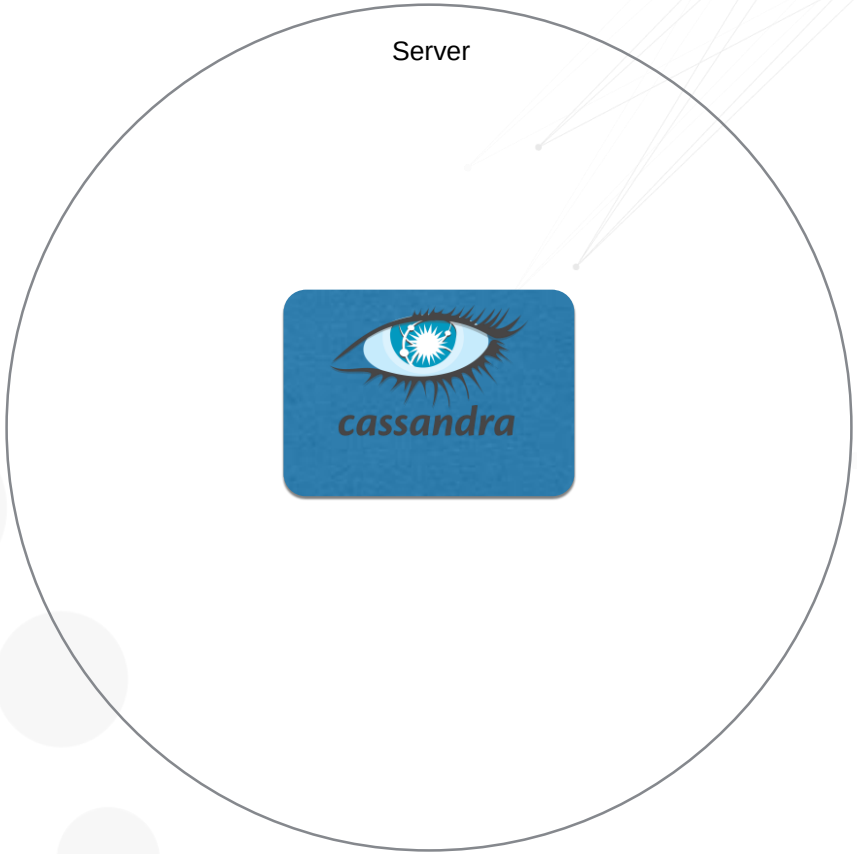
- ✓ Data
- ✓ Blobs (documents, files, images)
- ✓ Collections (Sets, Lists, Maps)
- ✓ UDTs

Row Key1	Column Key1	Column Key2	Column Key3	...
	Column Value1	Column Value2	Column Value3	...
⋮				

Access it with CQL ← familiar syntax to SQL

Basic Architecture

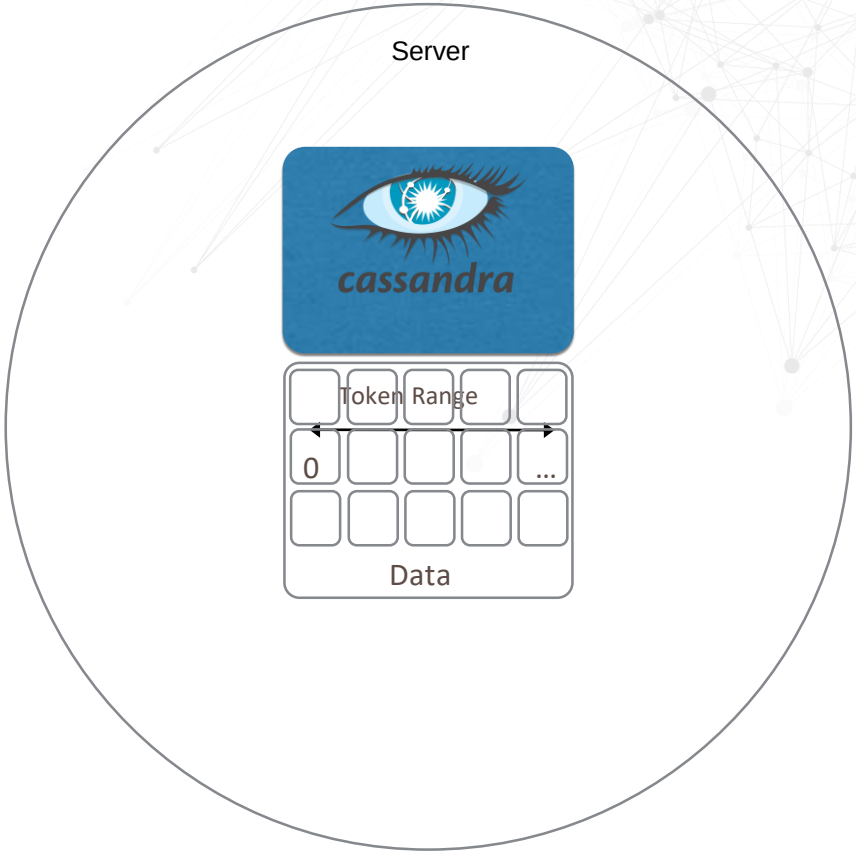
Node



Active Everywhere—Every Cloud | DATASTAX®

Token

- Each partition is a 64 bit value
- Consistent hash between -2^{63} to $+2^{63}-1$
- Each node owns a range of those values
- The token is the beginning of that range to the next node's token value
- Virtual Nodes break these down further



Active Everywhere—Every Cloud | DATASTAX

The cluster

Token	Range
0	0-100

Active Everywhere—Every Cloud | DATASTAX

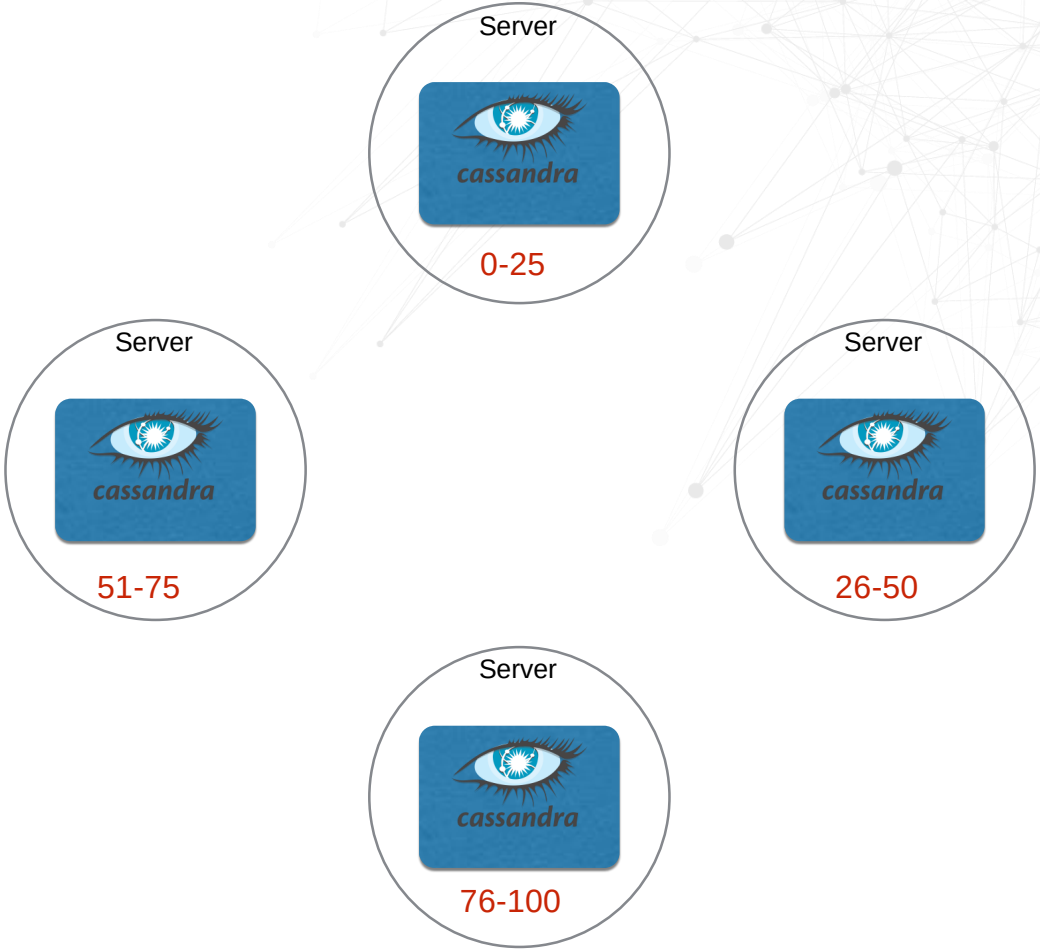
The cluster

Token	Range
0	0-50
51	51-100

Active Everywhere—Every Cloud | DATASTAX

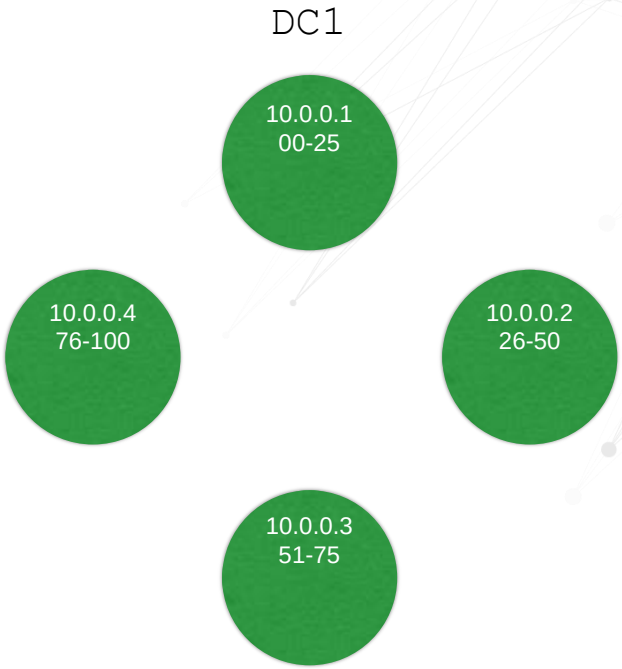
The cluster

Token	Range
0	0-25
26	26-50
51	51-75
76	76-100



Active Everywhere—Every Cloud | DATASTAX

Replication

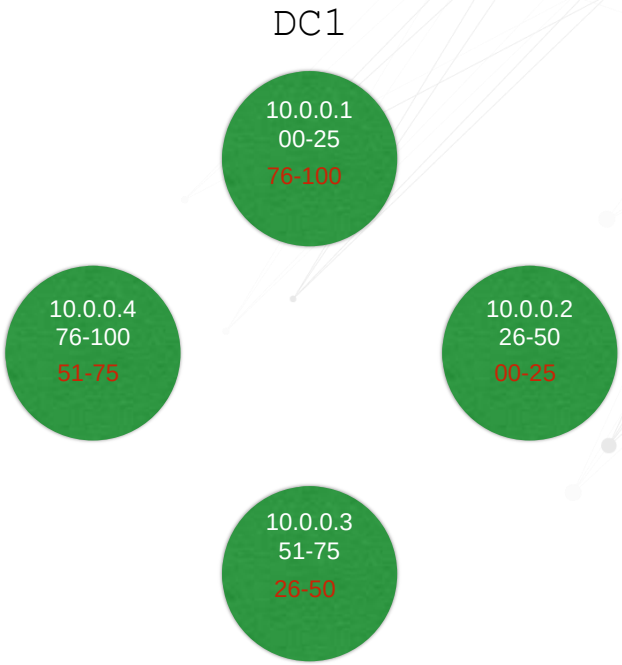


DC1: RF=1

Node	Primary
10.0.0.1	00-25
10.0.0.2	26-50
10.0.0.3	51-75
10.0.0.4	76-100

Active Everywhere—Every Cloud | DATASTAX®

Replication

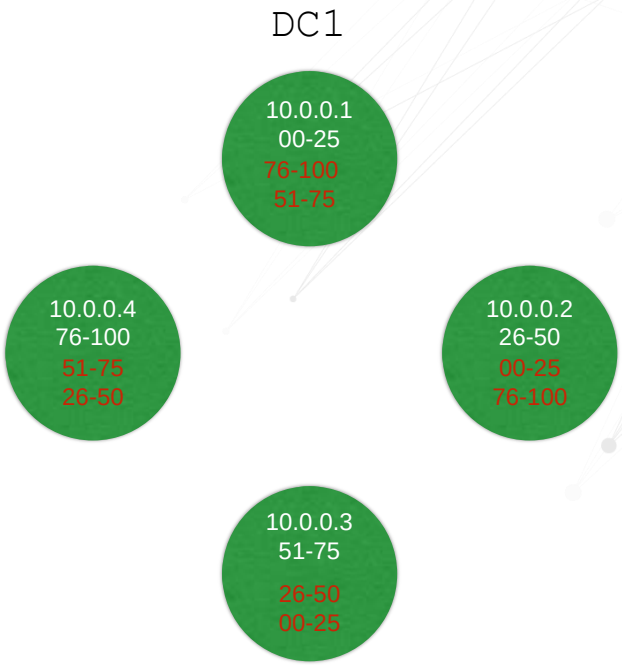


DC1: RF=2

Node	Primary	Replica
10.0.0.1	00-25	76-100
10.0.0.2	26-50	00-25
10.0.0.3	51-75	26-50
10.0.0.4	76-100	51-75

Active Everywhere—Every Cloud | DATASTAX®

Replication

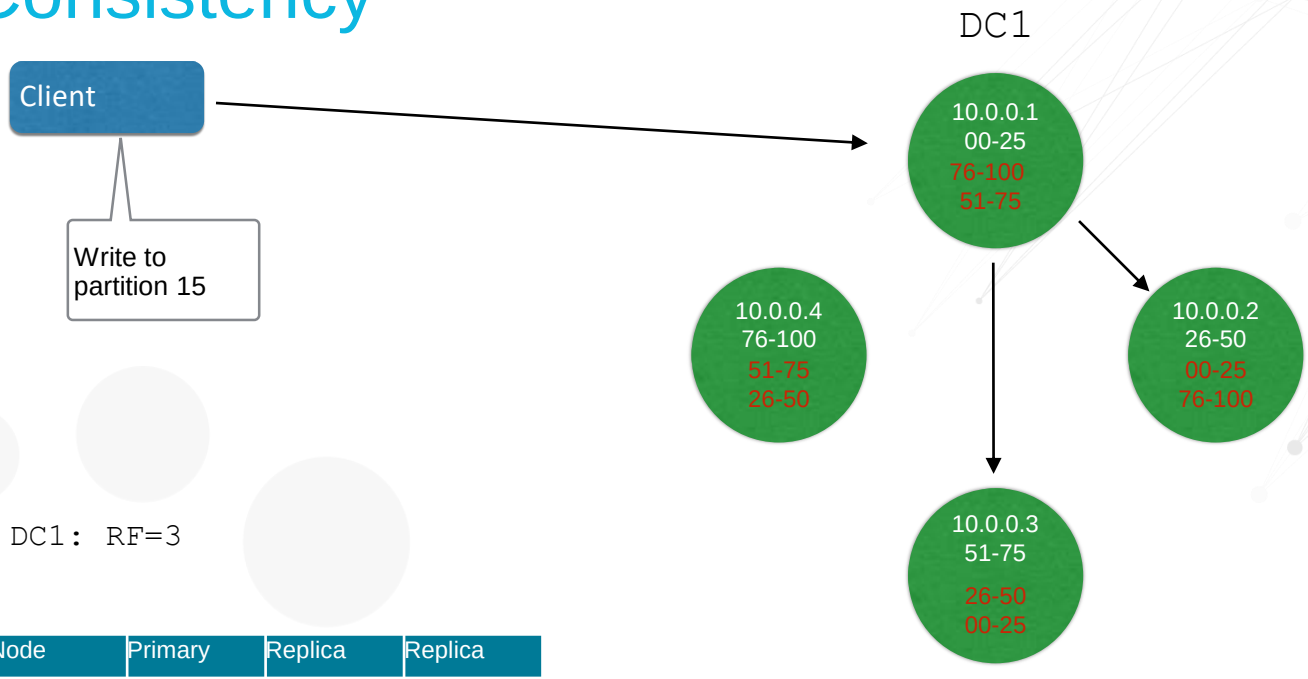


DC1: RF=3

Node	Primary	Replica	Replica
10.0.0.1	00-25	76-100	51-75
10.0.0.2	26-50	00-25	76-100
10.0.0.3	51-75	26-50	00-25
10.0.0.4	76-100	51-75	26-50

Active Everywhere—Every Cloud | DATASTAX®

Consistency



DC1: RF=3

Node	Primary	Replica	Replica
10.0.0.1	00-25	76-100	51-75
10.0.0.2	26-50	00-25	76-100
10.0.0.3	51-75	26-50	00-25
10.0.0.4	76-100	51-75	26-50

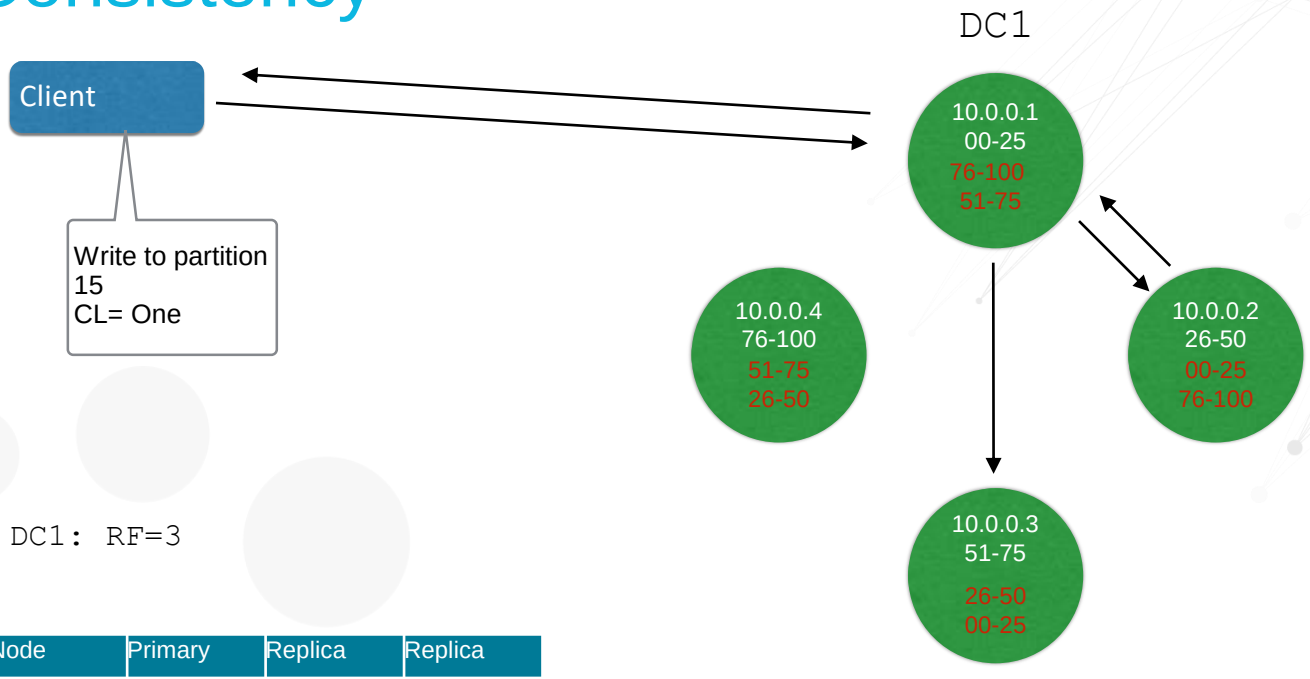
Active Everywhere—Every Cloud | DATASTAX

Consistency level

Consistency Level	Number of Nodes Acknowledged
One	One replica acknowledges read One replica commits write
Quorum	51% nodes agree on read or commit write
Local Quorum	51% in local DC

Active Everywhere—Every Cloud | DATASTAX

Consistency

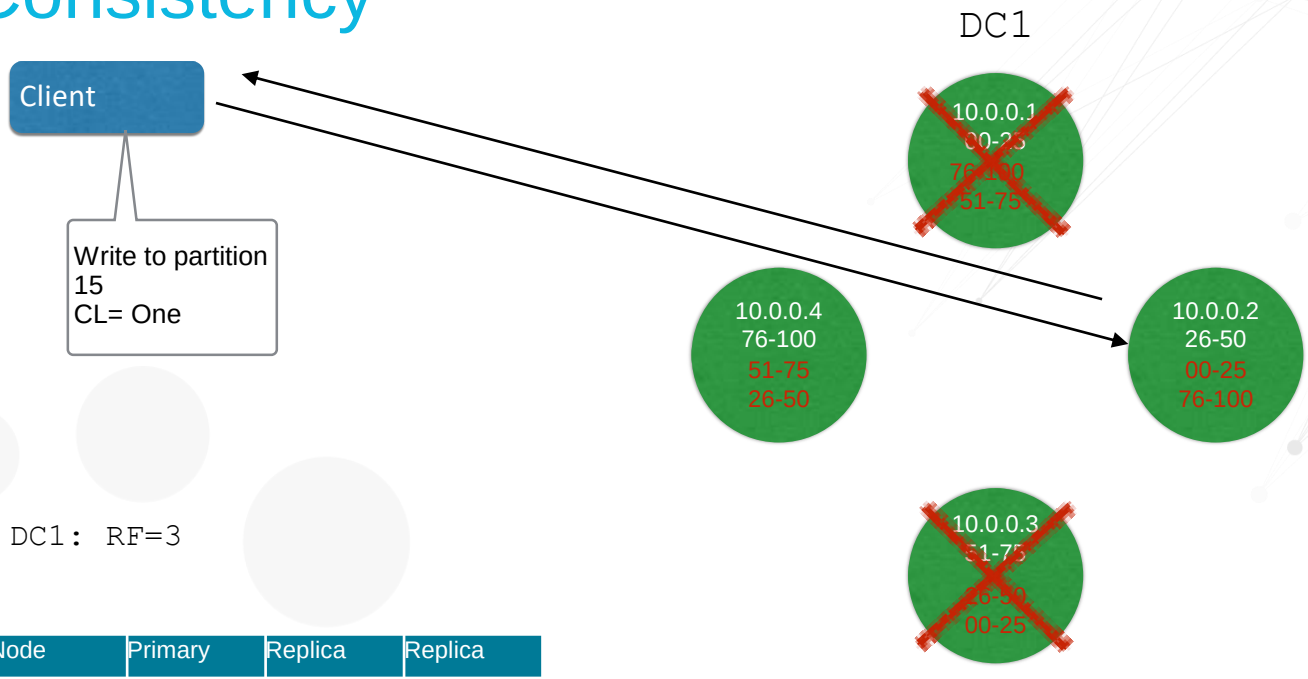


DC1: RF=3

Node	Primary	Replica	Replica
10.0.0.1	00-25	76-100	51-75
10.0.0.2	26-50	00-25	76-100
10.0.0.3	51-75	26-50	00-25
10.0.0.4	76-100	51-75	26-50

Active Everywhere—Every Cloud | DATASTAX

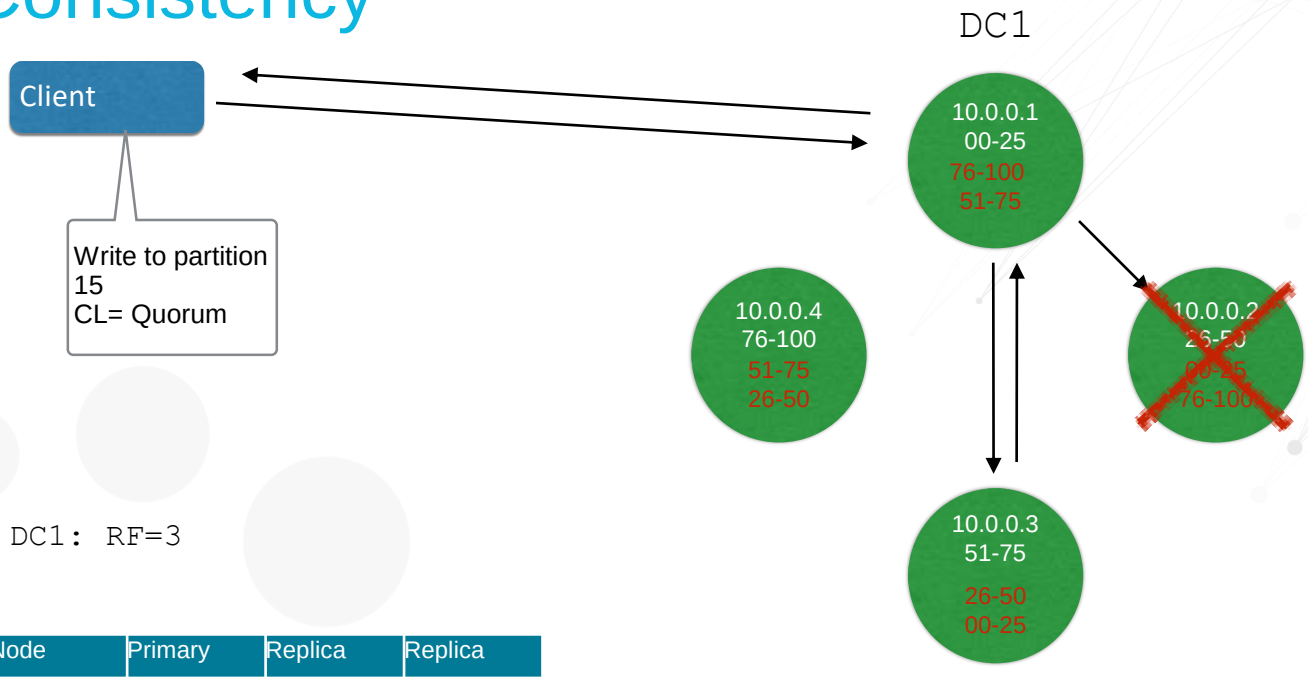
Consistency



DC1: RF=3

Node	Primary	Replica	Replica
10.0.0.1	00-25	76-100	51-75
10.0.0.2	26-50	00-25	76-100
10.0.0.3	51-75	26-50	00-25
10.0.0.4	76-100	51-75	26-50

Consistency

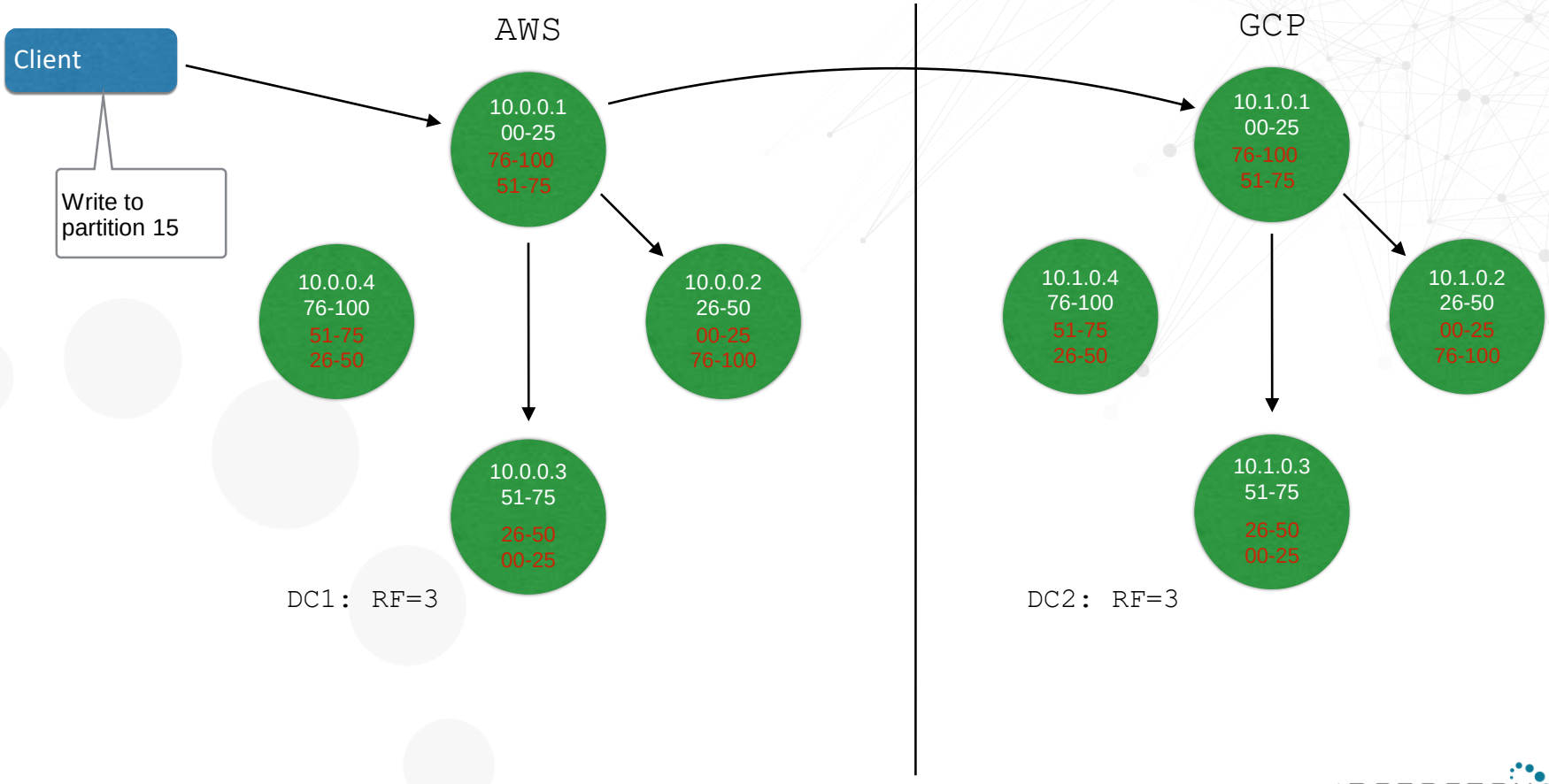


DC1: RF=3

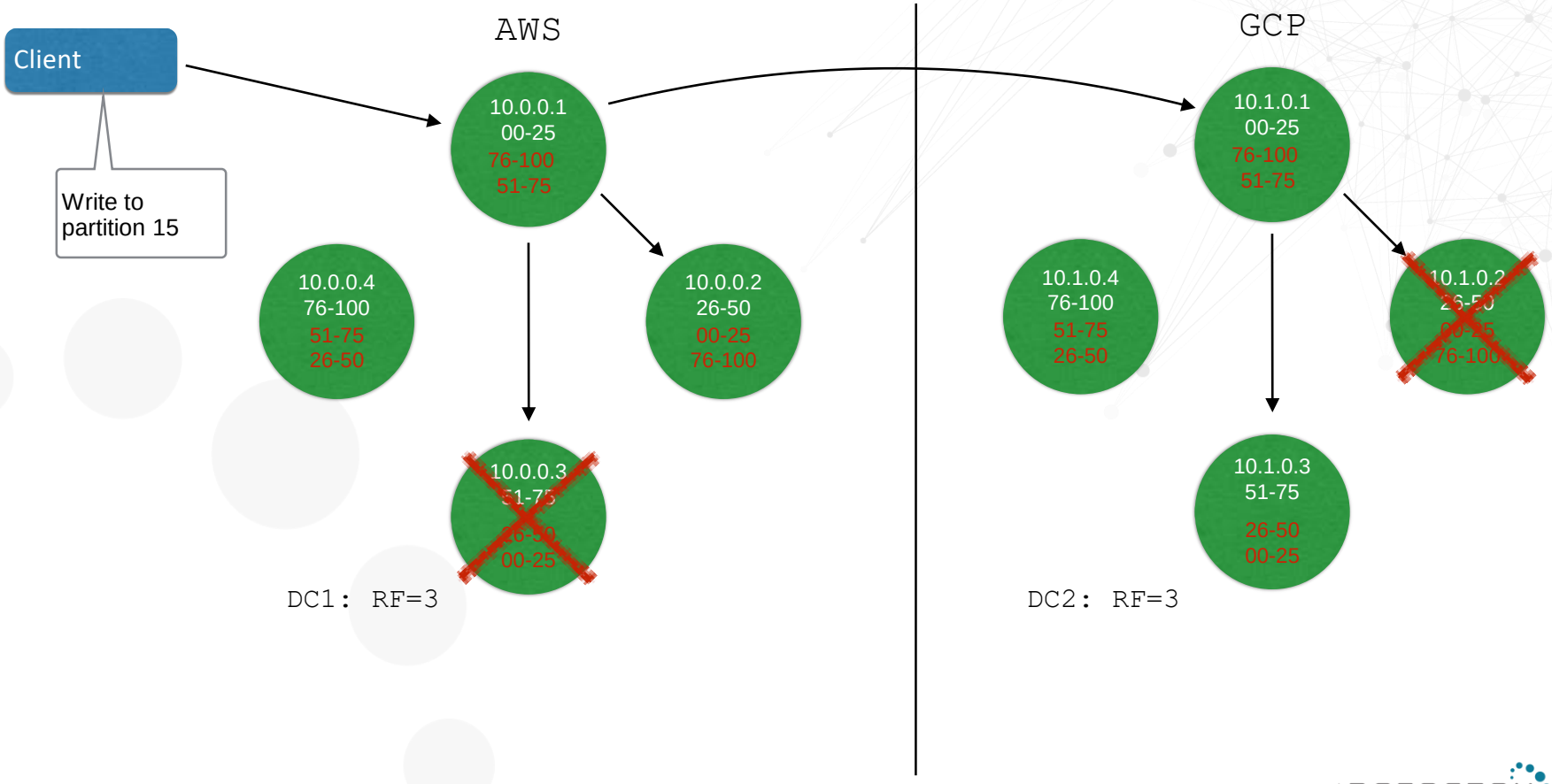
Node	Primary	Replica	Replica
10.0.0.1	00-25	76-100	51-75
10.0.0.2	26-50	00-25	76-100
10.0.0.3	51-75	26-50	00-25
10.0.0.4	76-100	51-75	26-50

Active Everywhere—Every Cloud | DATASTAX

Multi-datacenter

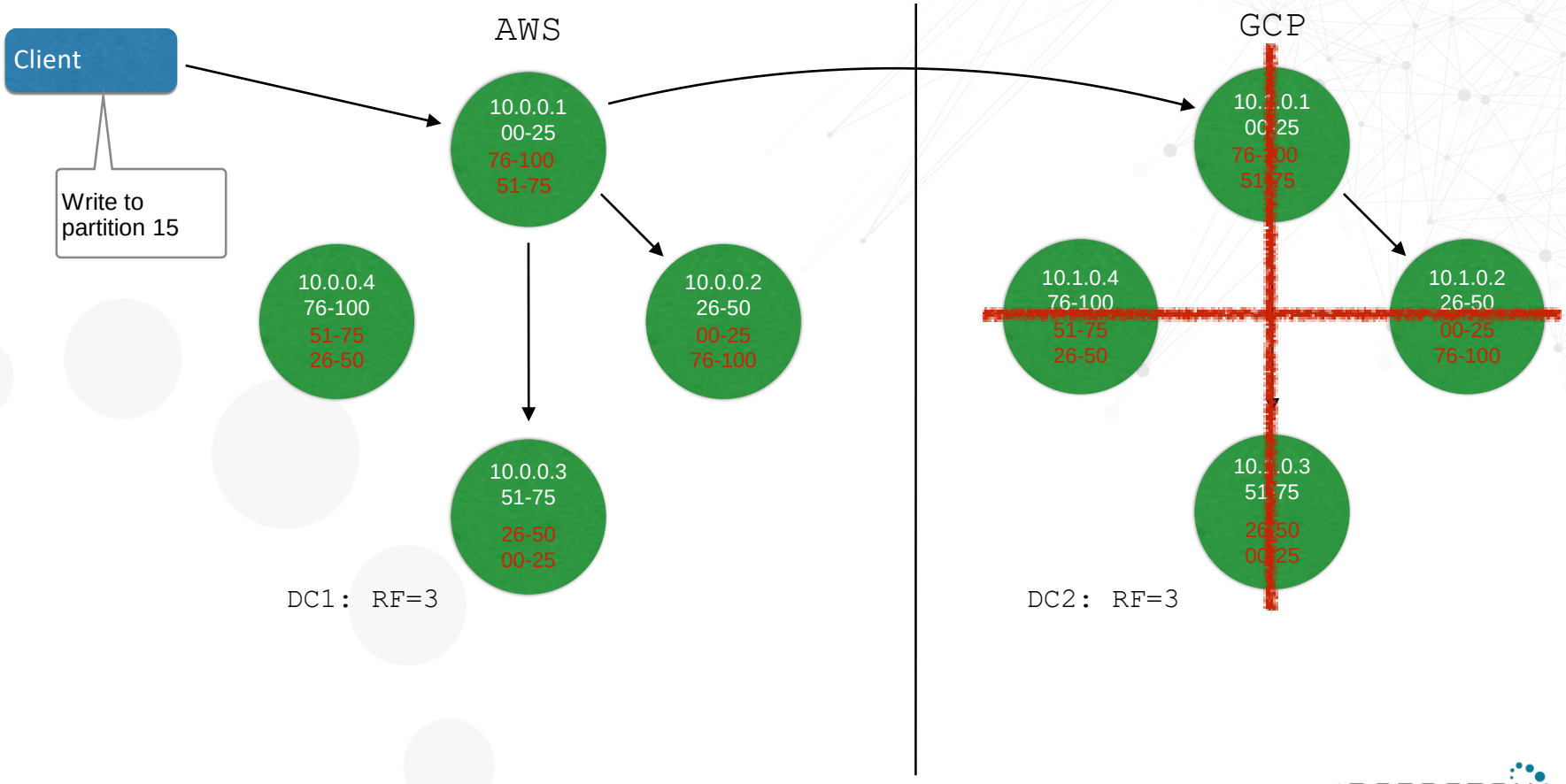


Multi-datacenter



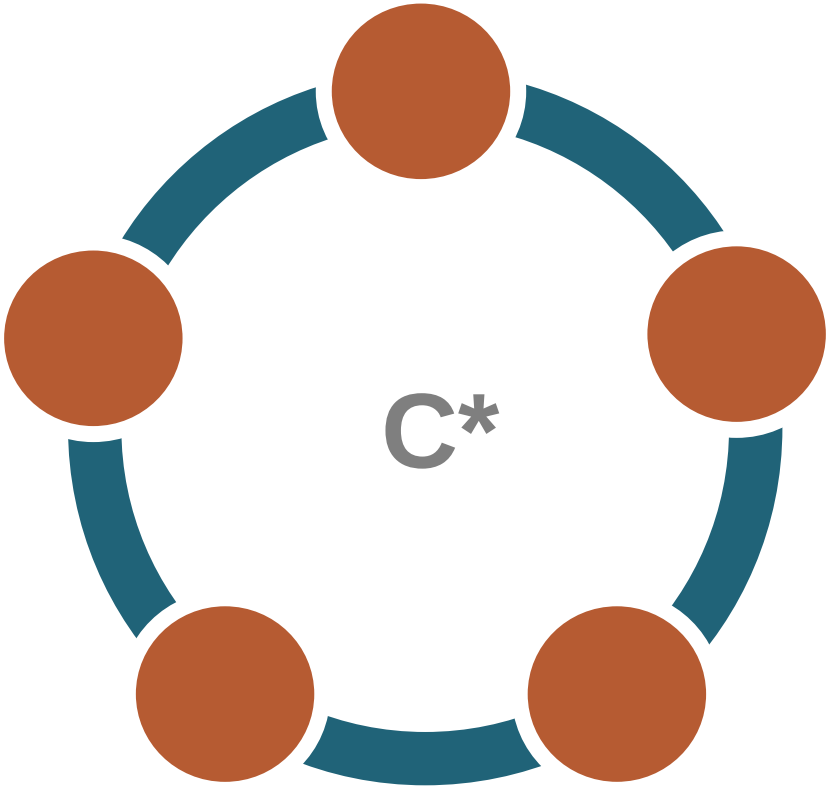
Active Everywhere—Every Cloud | DATASTAX

Multi-datacenter



How Does The Database Work? The Basics

It’s a Cluster of Nodes



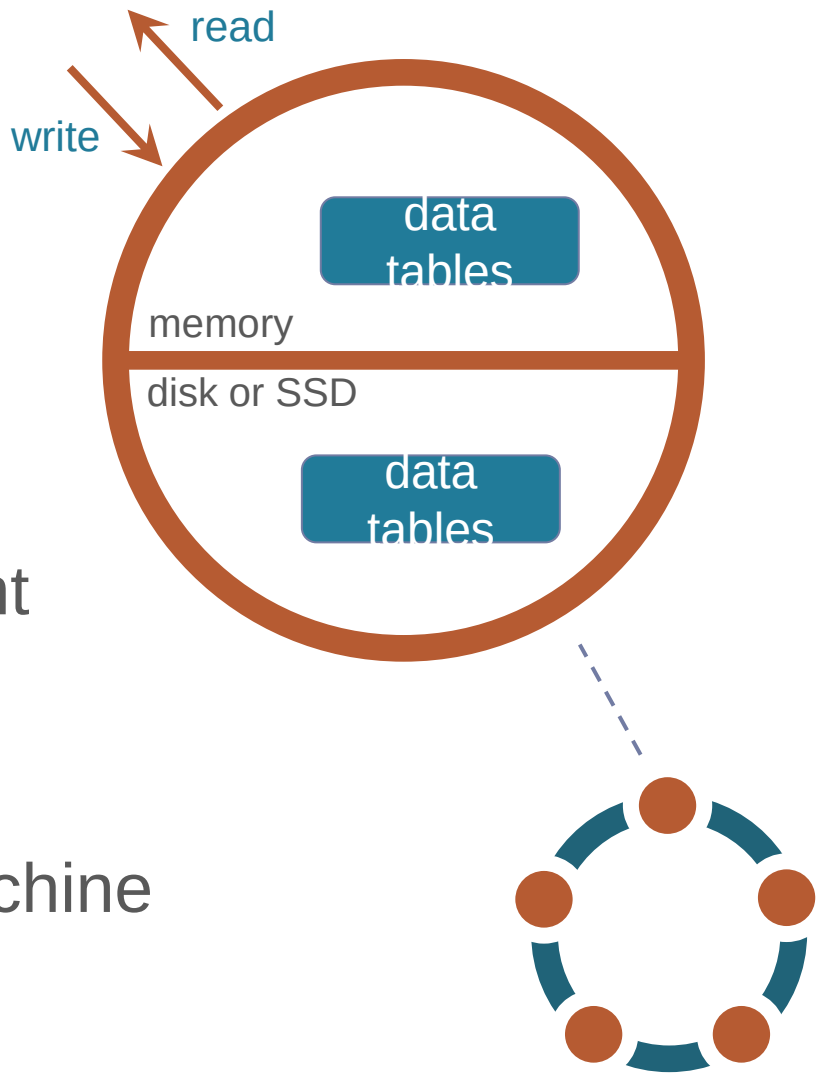
Each Cassandra node is…

Fully functional database

Very fast (low latency)

In-memory and/or persistent storage

One machine or Virtual Machine



Why is Cassandra so fast?

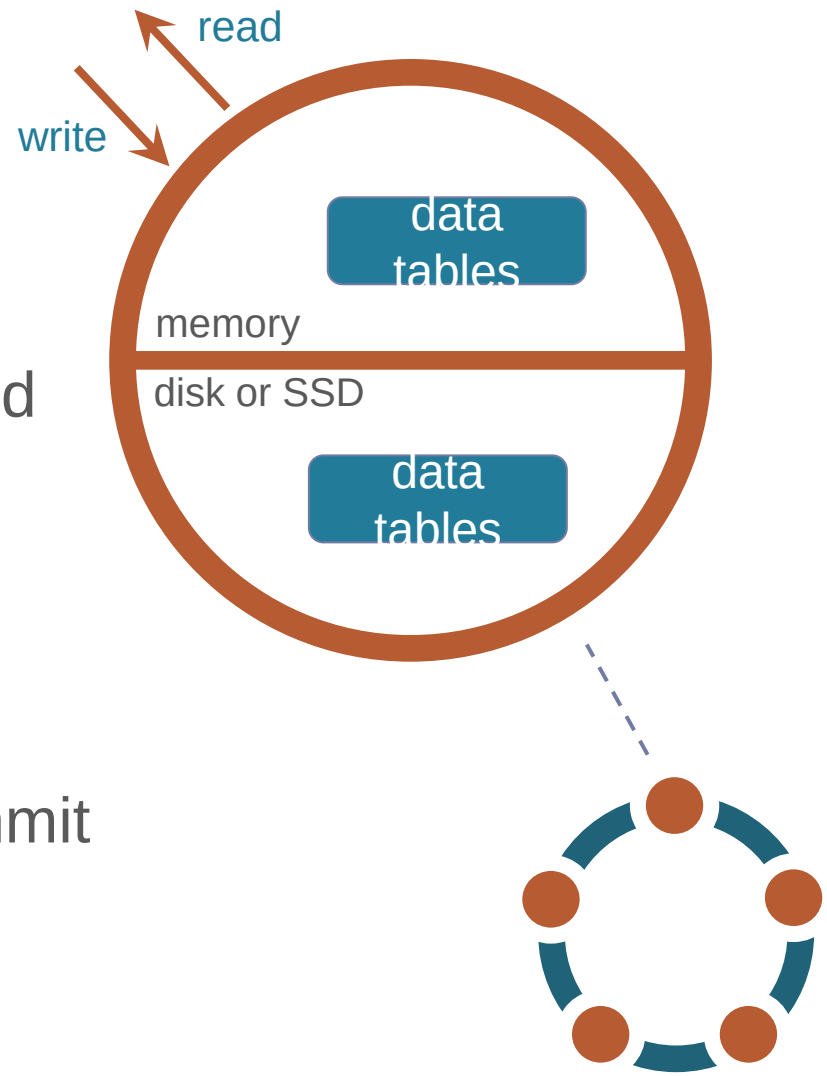
Low Latency Nodes
Distributed Workload

Writes

- Durable write to log file (fast)
- No db file read or lock needed

Reads

- Get data from memory first
- Optimize storage layer IO
- No locking or two phase commit



A Cassandra Cluster

read or write

Nodes in a peer-to-peer cluster

No single point of failure

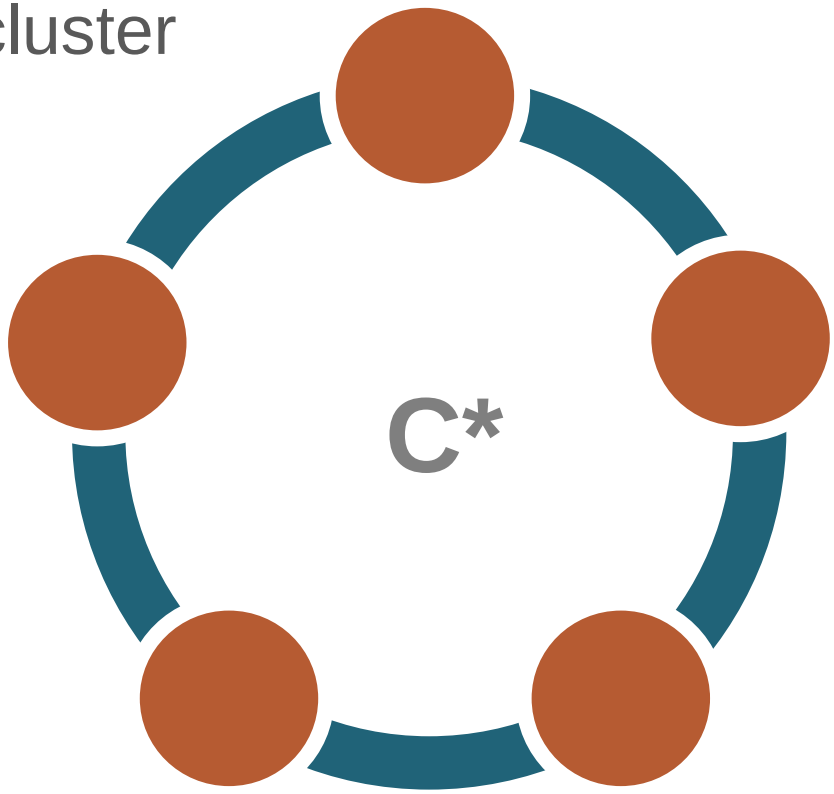
Built in data replication

Data is always available

100% Uptime

Across data centers

Failure avoidance



A Cassandra Cluster

Nodes in a peer-to-peer cluster

No single point of failure

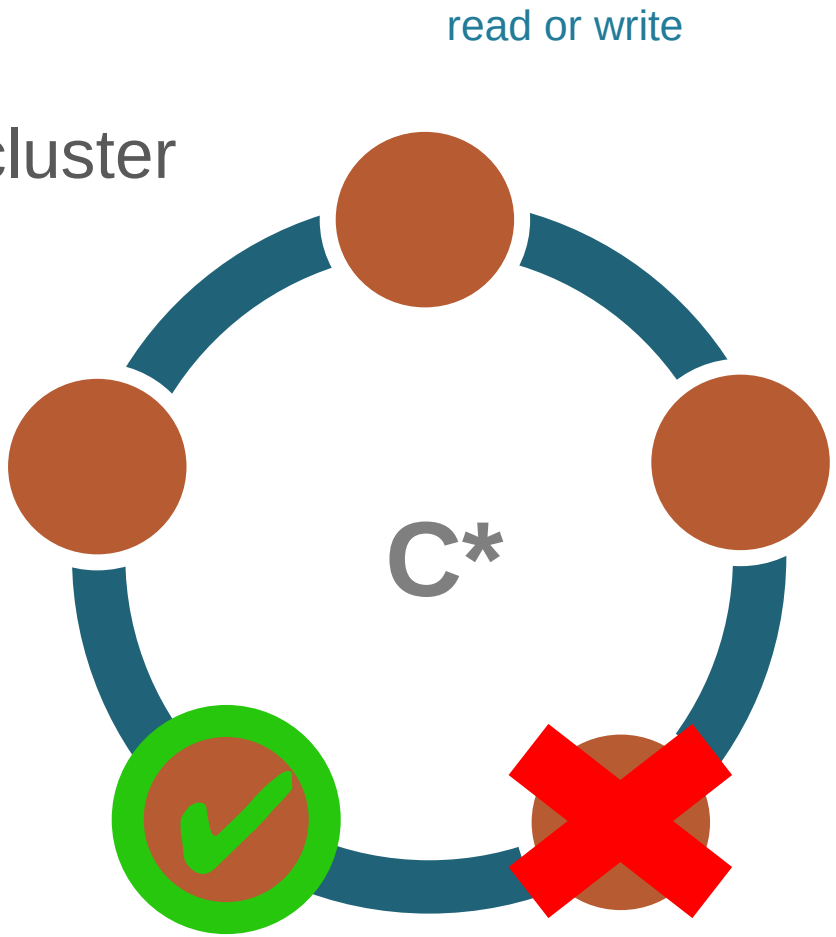
Built in data replication

Data is always available

100% Uptime

Across data centers

Failure avoidance



A Cassandra Cluster

Nodes in a peer-to-peer cluster

No single point of failure

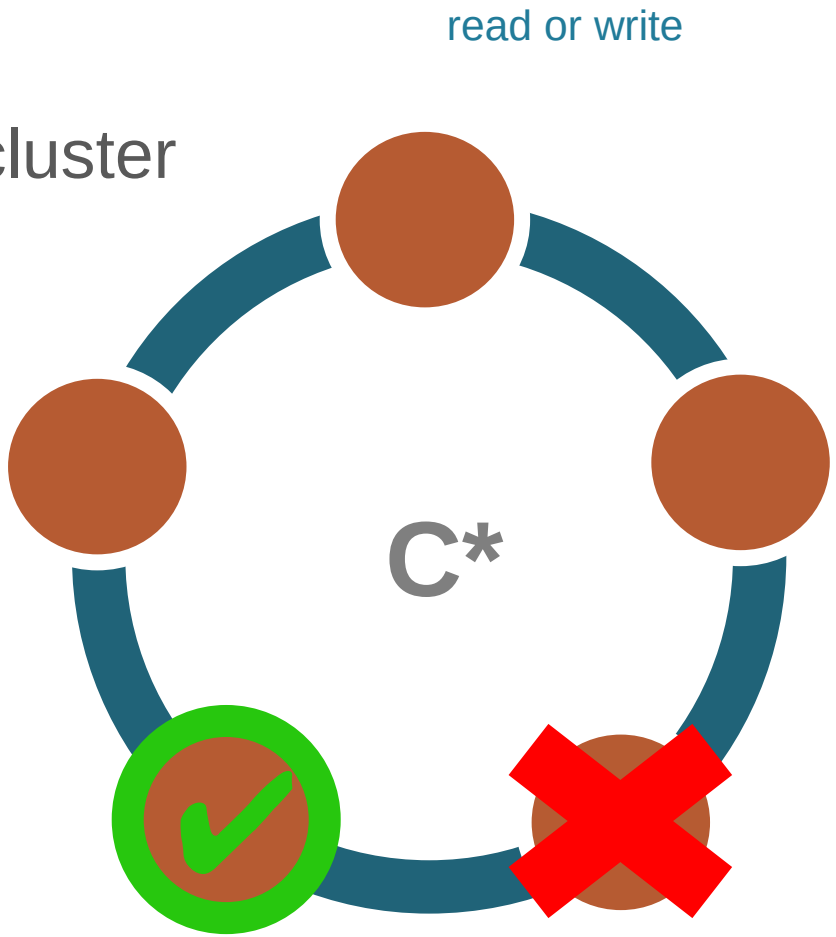
Built in data replication

Data is always available

100% Uptime

Across data centers

Failure avoidance



A Cassandra Cluster

Nodes in a peer-to-peer cluster

No single point of failure

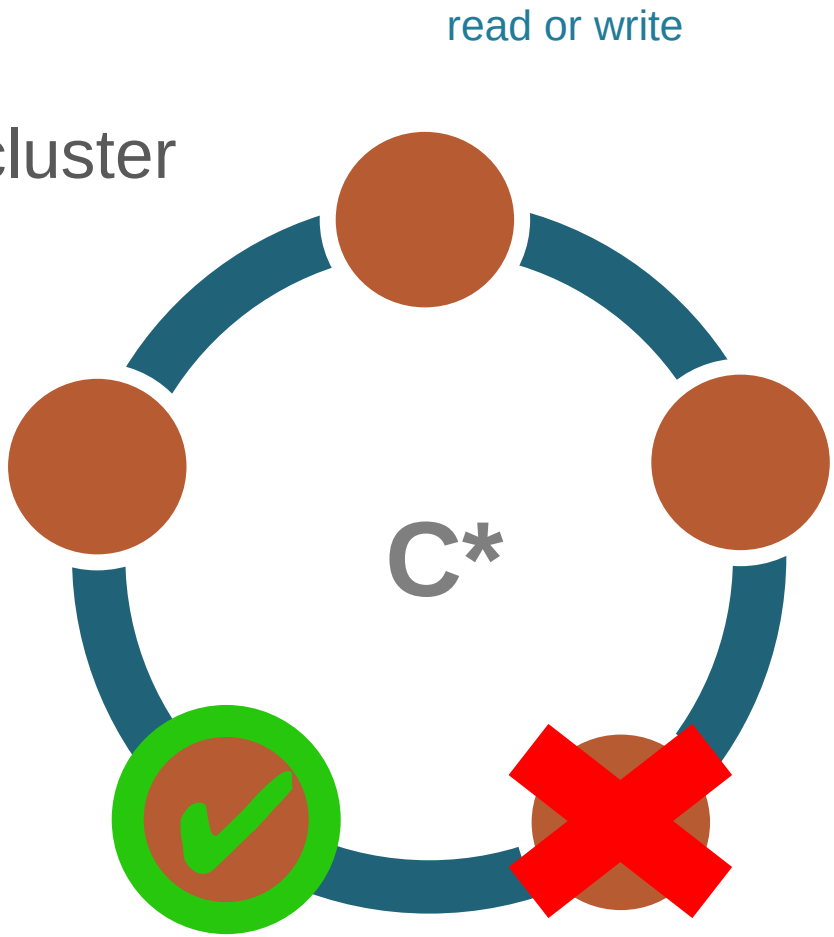
Built in data replication

Data is always available

100% Uptime

Across data centers

Failure avoidance



A Cassandra Cluster

Nodes in a peer-to-peer cluster

No single point of failure

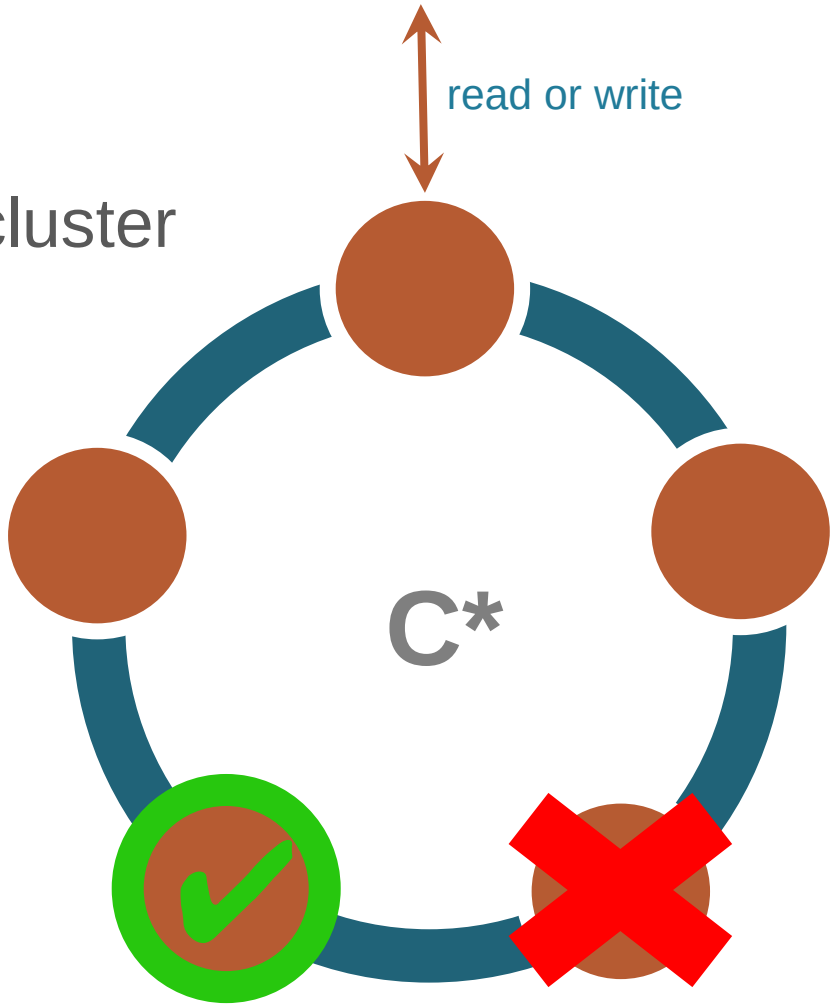
Built in data replication

Data is always available

100% Uptime

Across data centers

Failure avoidance



A Cassandra Cluster

Nodes in a peer-to-peer cluster

No single point of failure

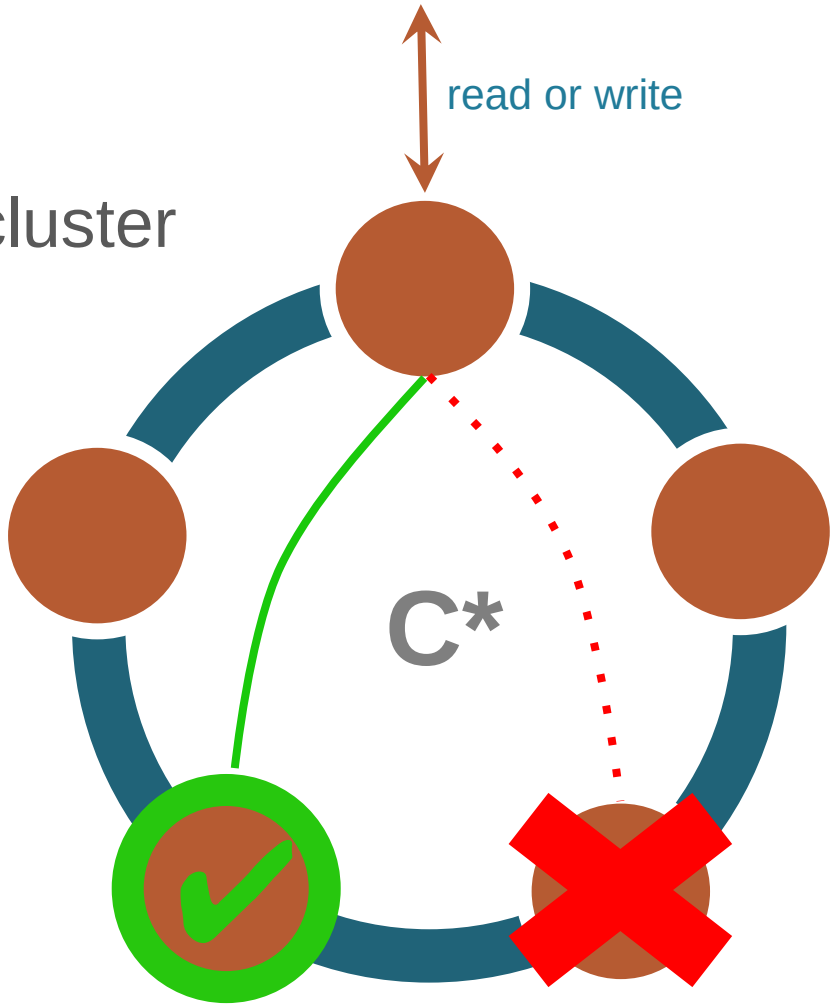
Built in data replication

Data is always available

100% Uptime

Across data centers

Failure avoidance



A Cassandra Cluster

Nodes in a peer-to-peer cluster

No single point of failure

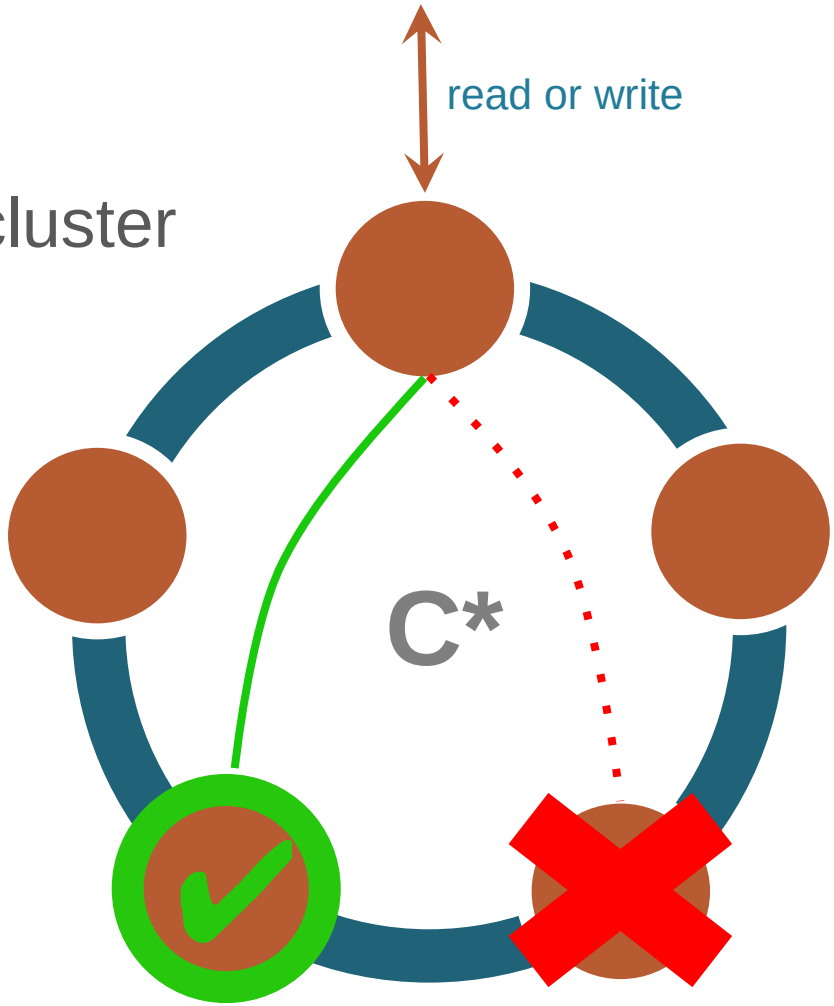
Built in data replication

Data is always available

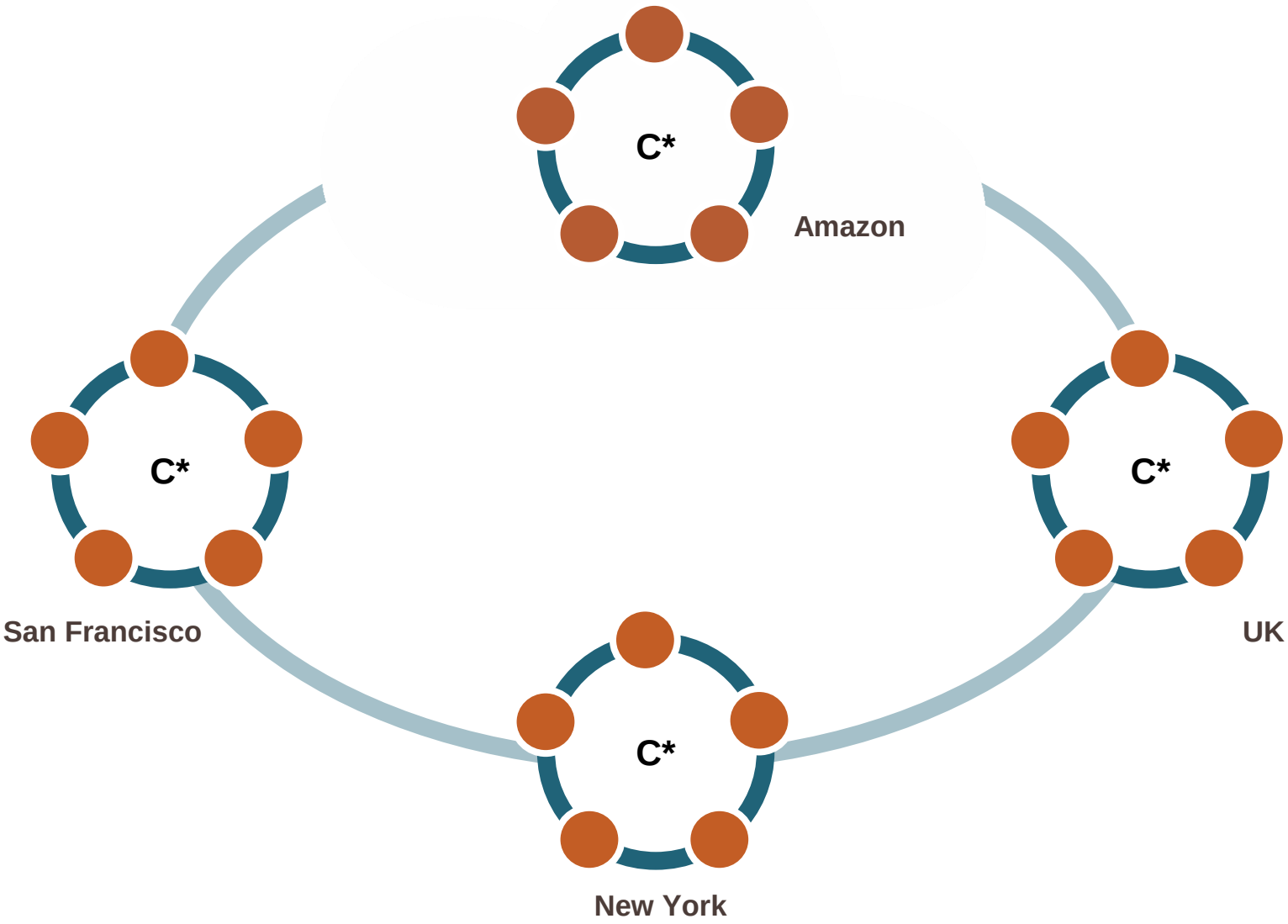
100% Uptime

Across data centers

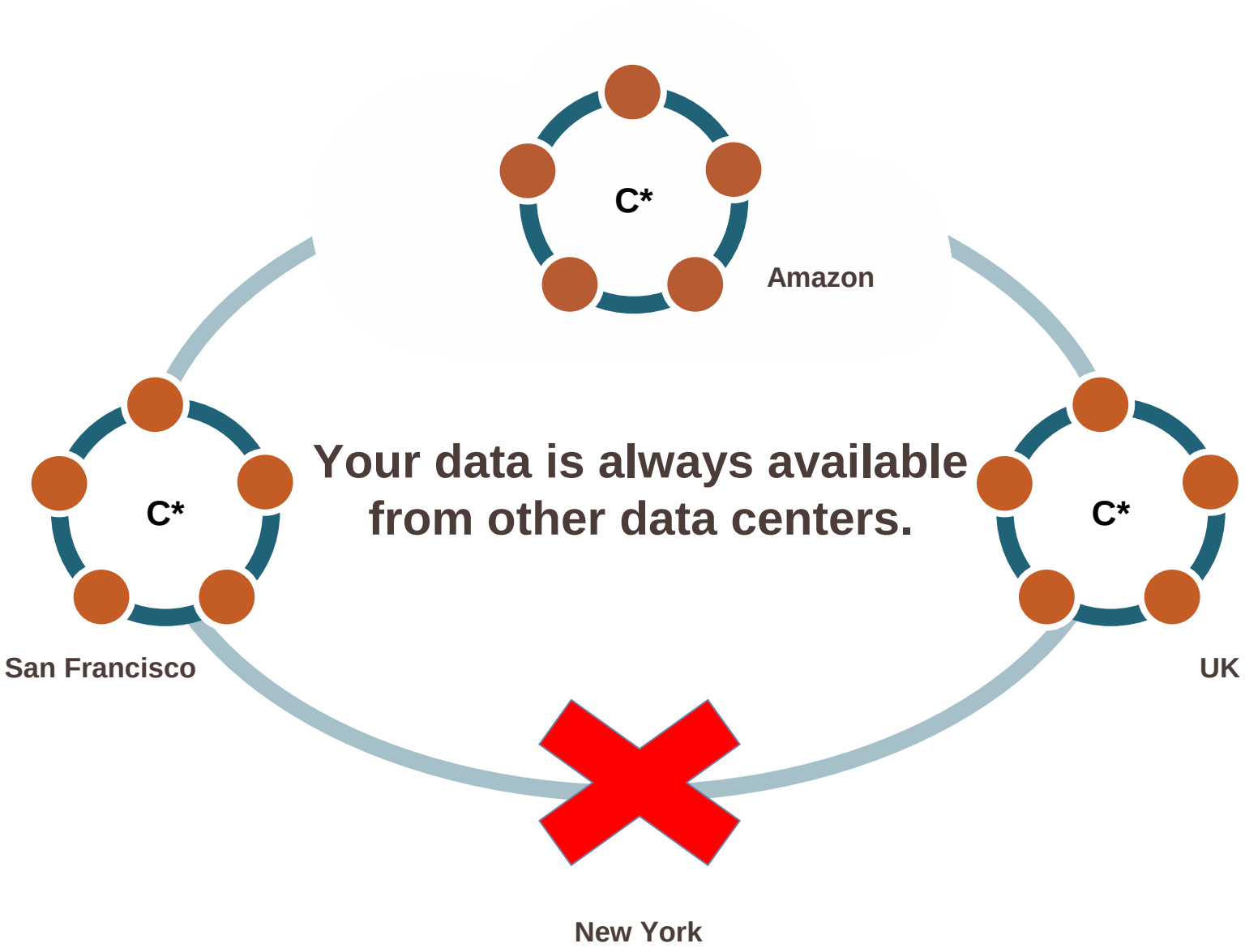
Failure avoidance



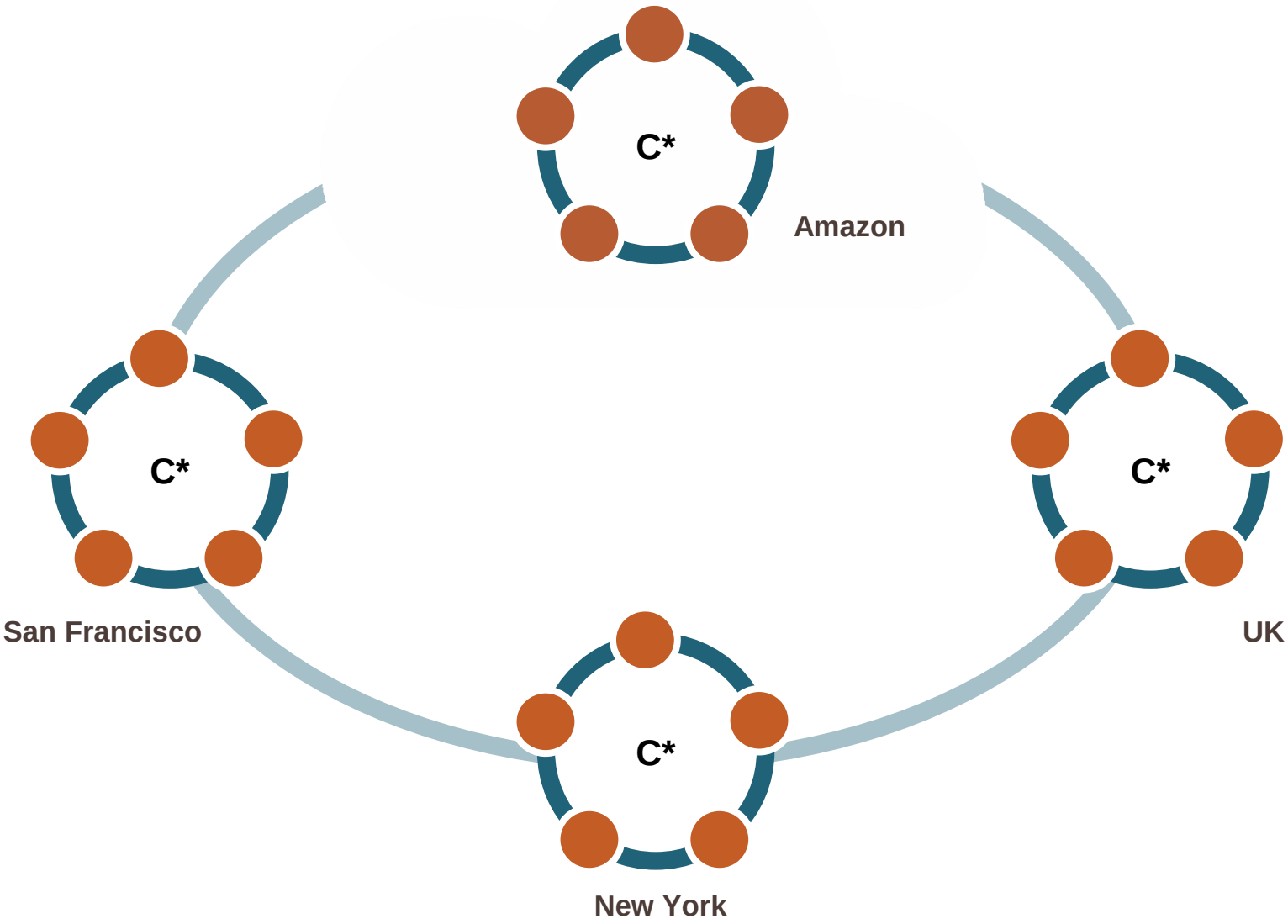
Multi-data center deployment



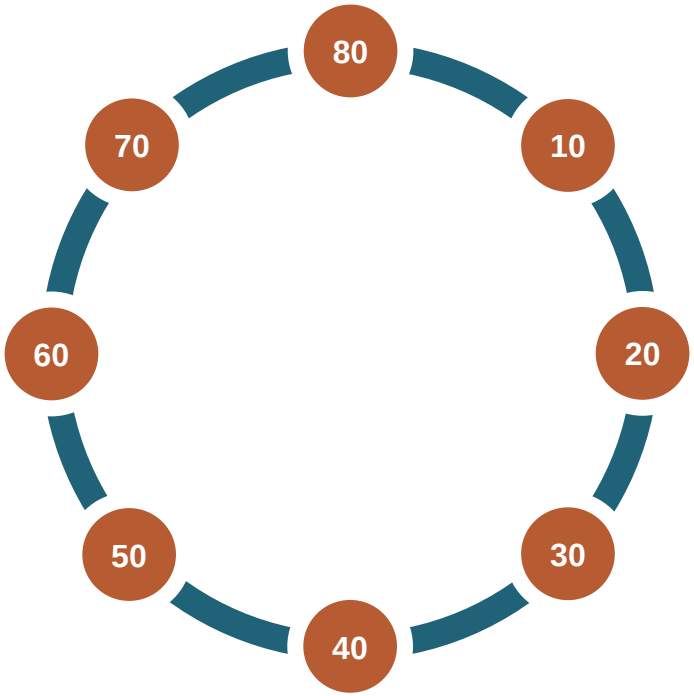
If a data center goes offline…



…and recovers automatically



Cassandra Cluster Architecture



Each Node ~ box or VM

(technically it's a JVM)

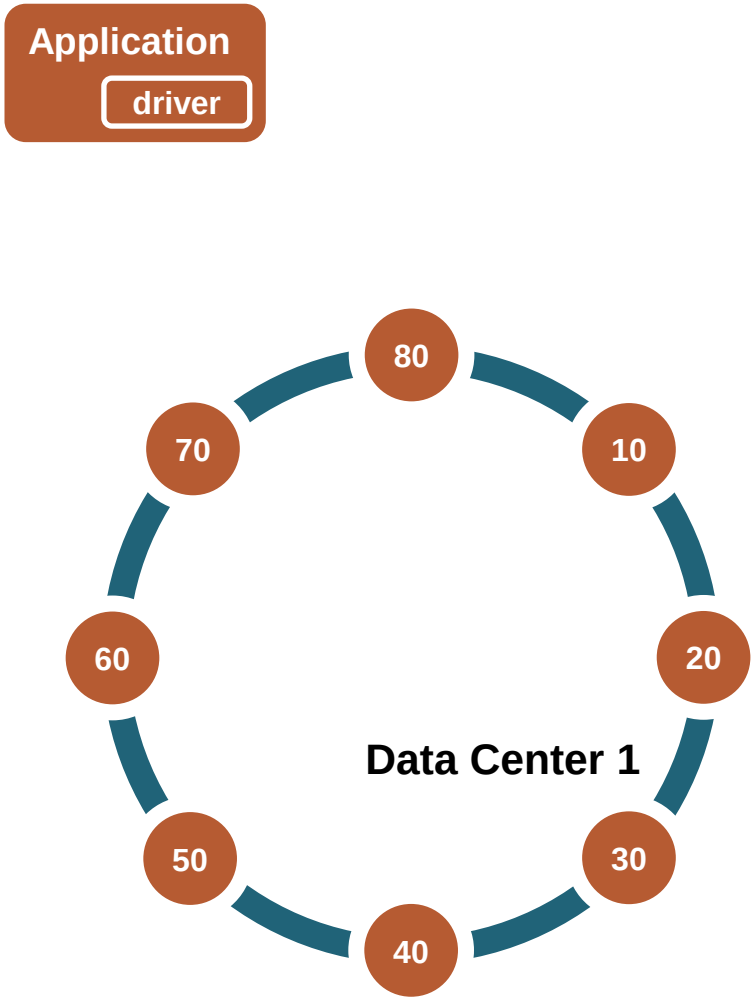
Each node has same Cassandra database functionality

System/hardware failures happen

Snitch – topology (DC and rack)

Gossip – state of each node

Transaction Load Balancing



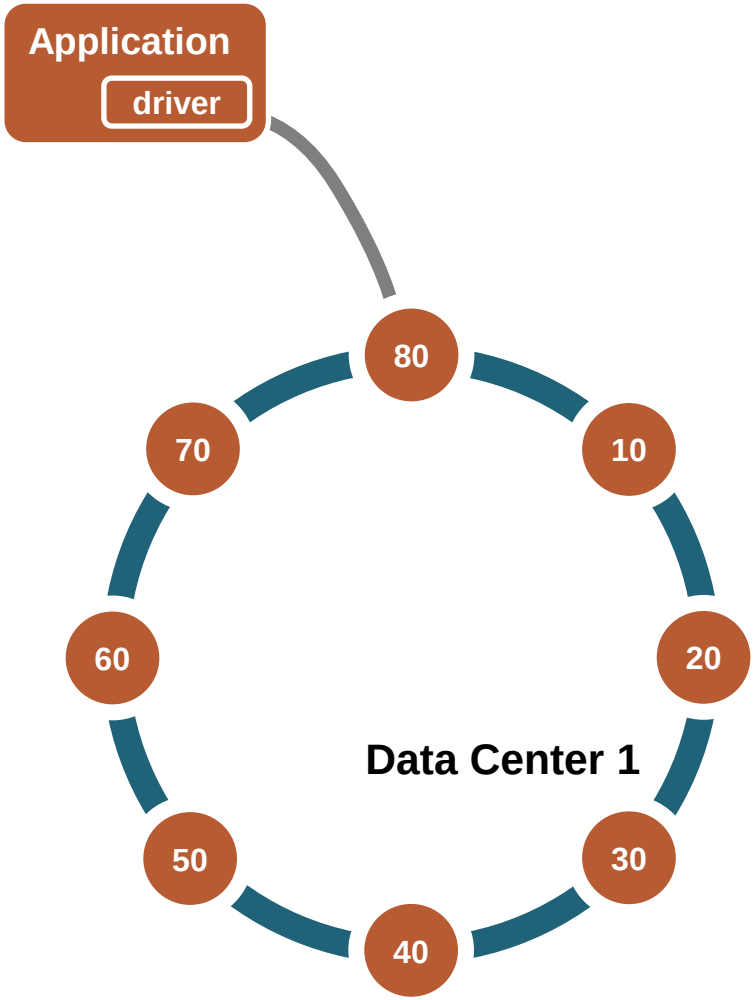
Driver manages connection pool

Driver has load balancing policies that can be applied

Each transaction has a different connection point in the cluster

Asynch operations are faster

Transaction Load Balancing



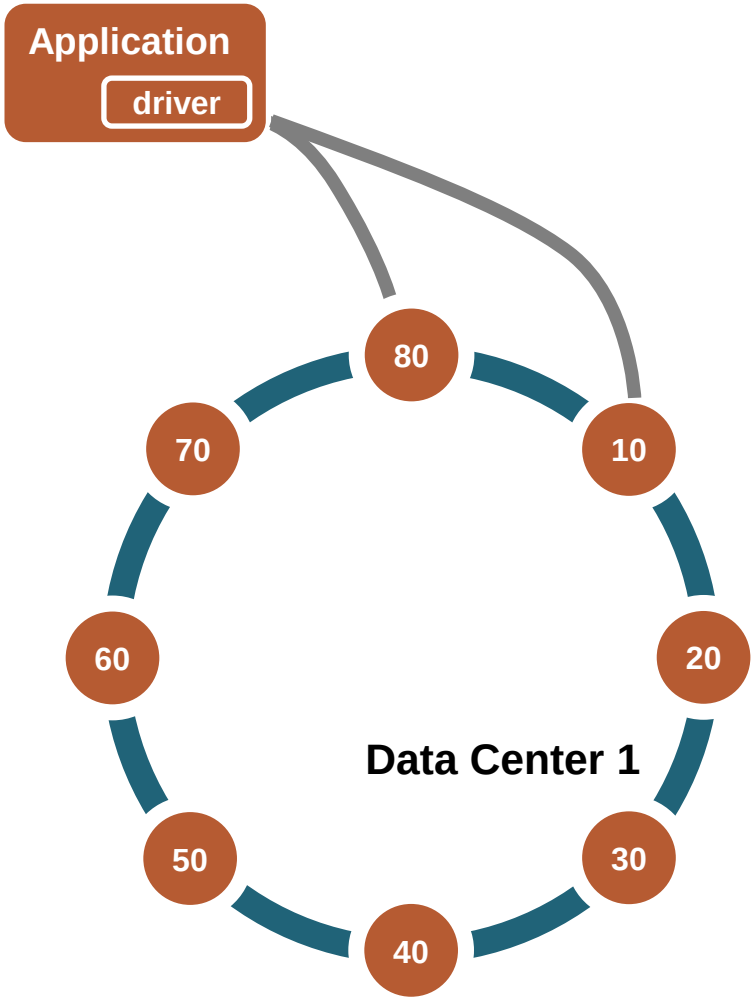
Driver manages connection pool

Driver has load balancing policies that can be applied

Each transaction has a different connection point in the cluster

Asynch operations are faster

Transaction Load Balancing



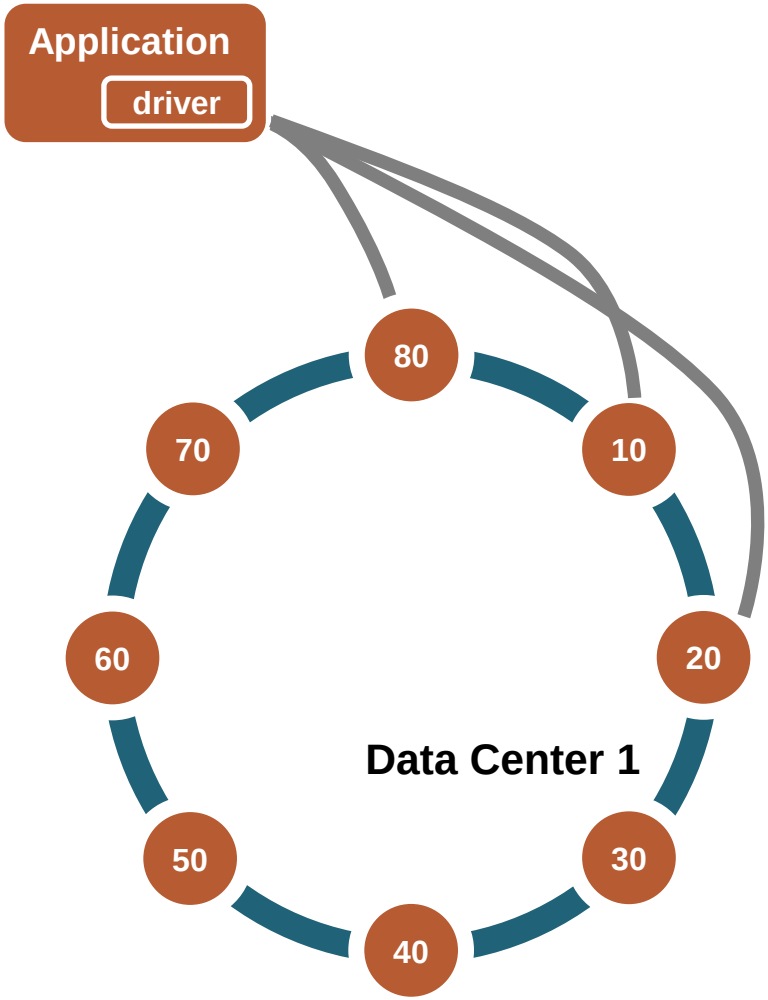
Driver manages connection pool

Driver has load balancing policies that can be applied

Each transaction has a different connection point in the cluster

Asynch operations are faster

Transaction Load Balancing



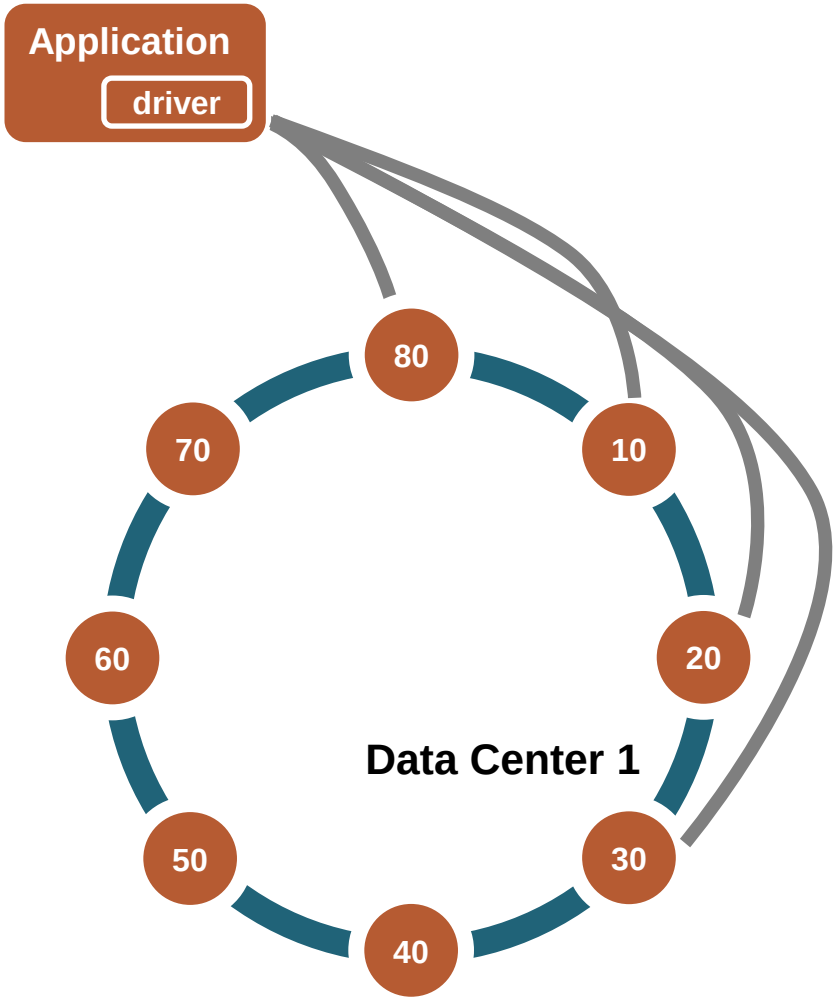
Driver manages connection pool

Driver has load balancing policies that can be applied

Each transaction has a different connection point in the cluster

Asynch operations are faster

Transaction Load Balancing



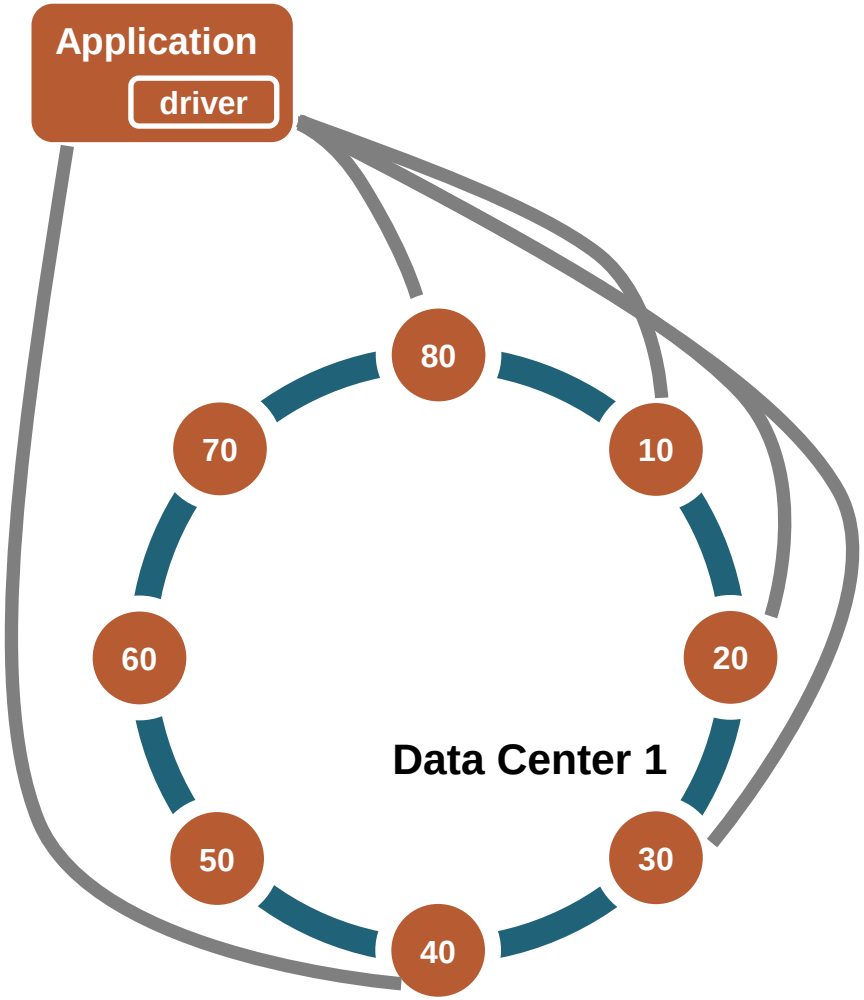
Driver manages connection pool

Driver has load balancing policies that can be applied

Each transaction has a different connection point in the cluster

Asynch operations are faster

Transaction Load Balancing



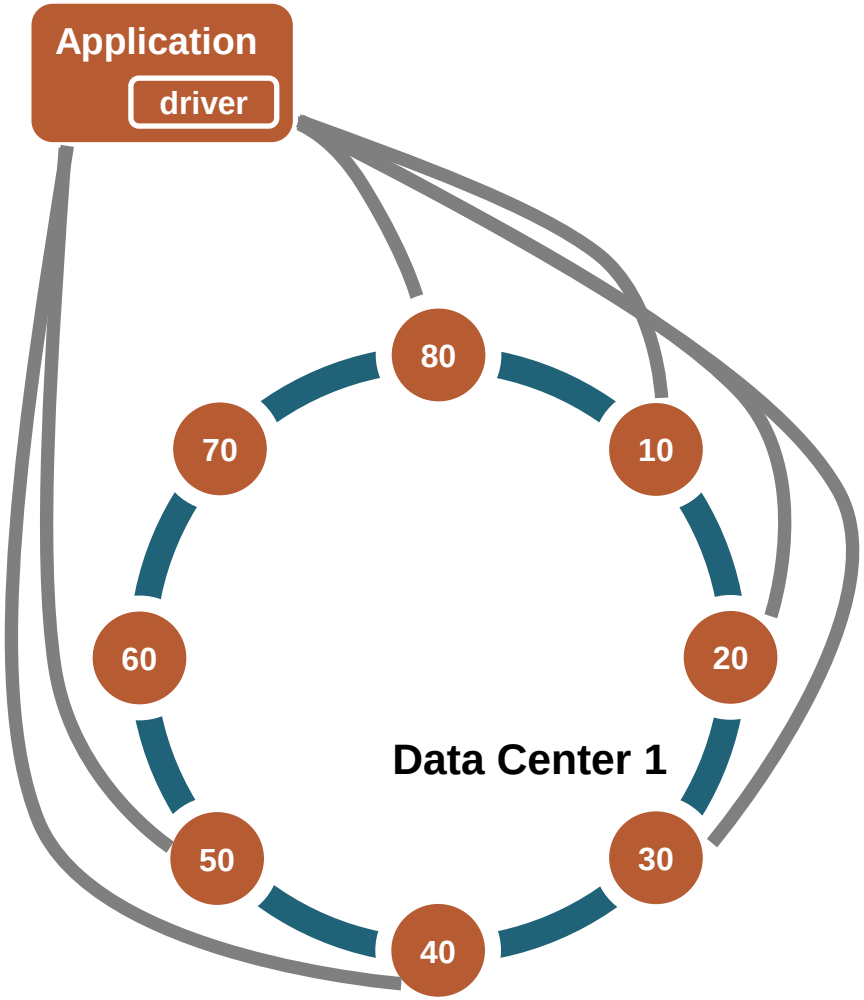
Driver manages connection pool

Driver has load balancing policies that can be applied

Each transaction has a different connection point in the cluster

Asynch operations are faster

Transaction Load Balancing



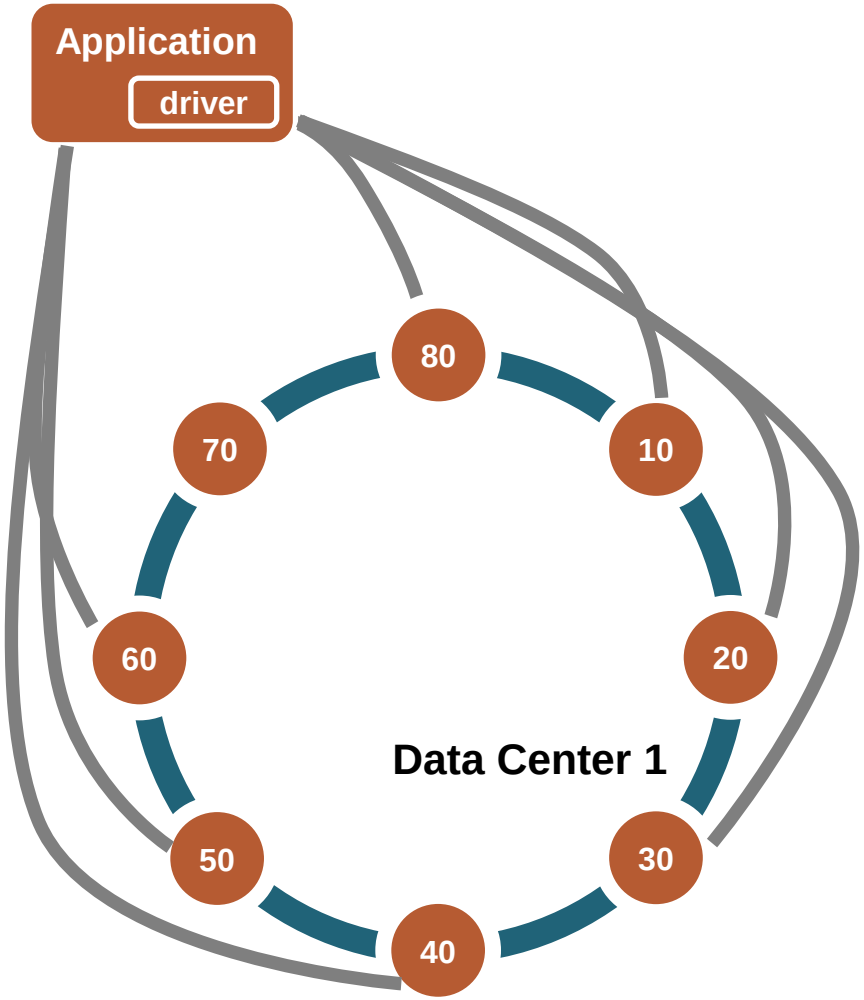
Driver manages connection pool

Driver has load balancing policies that can be applied

Each transaction has a different connection point in the cluster

Asynch operations are faster

Transaction Load Balancing



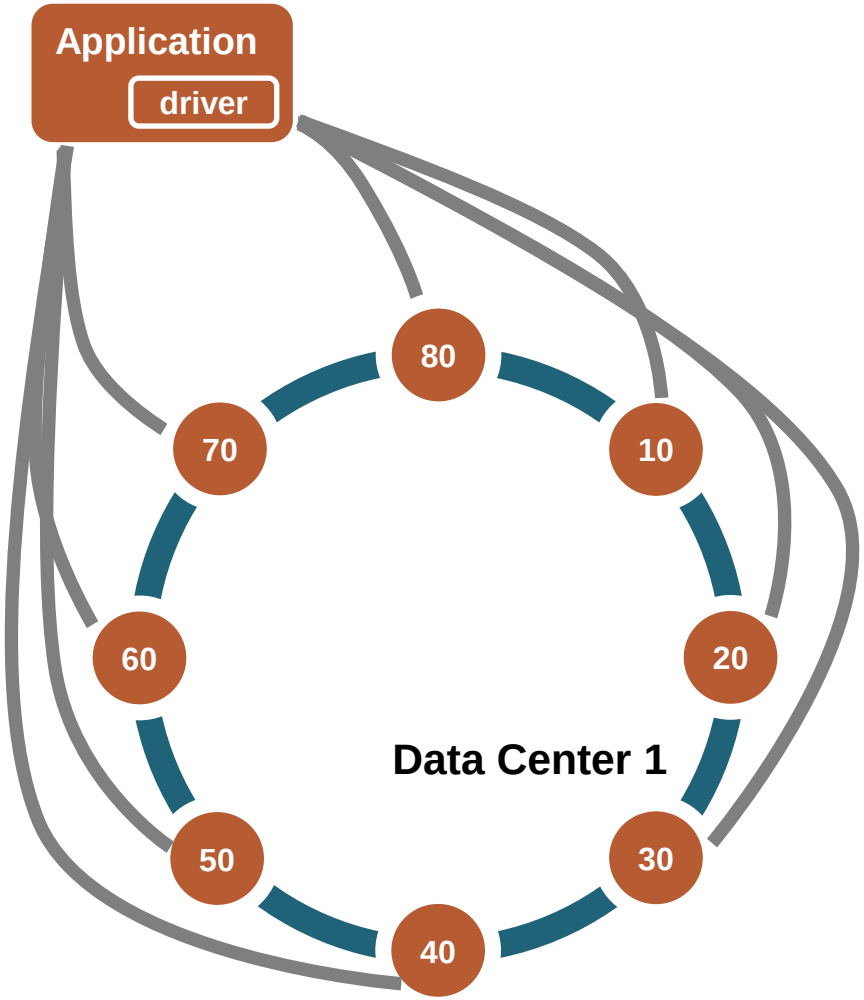
Driver manages connection pool

Driver has load balancing policies that can be applied

Each transaction has a different connection point in the cluster

Asynch operations are faster

Transaction Load Balancing



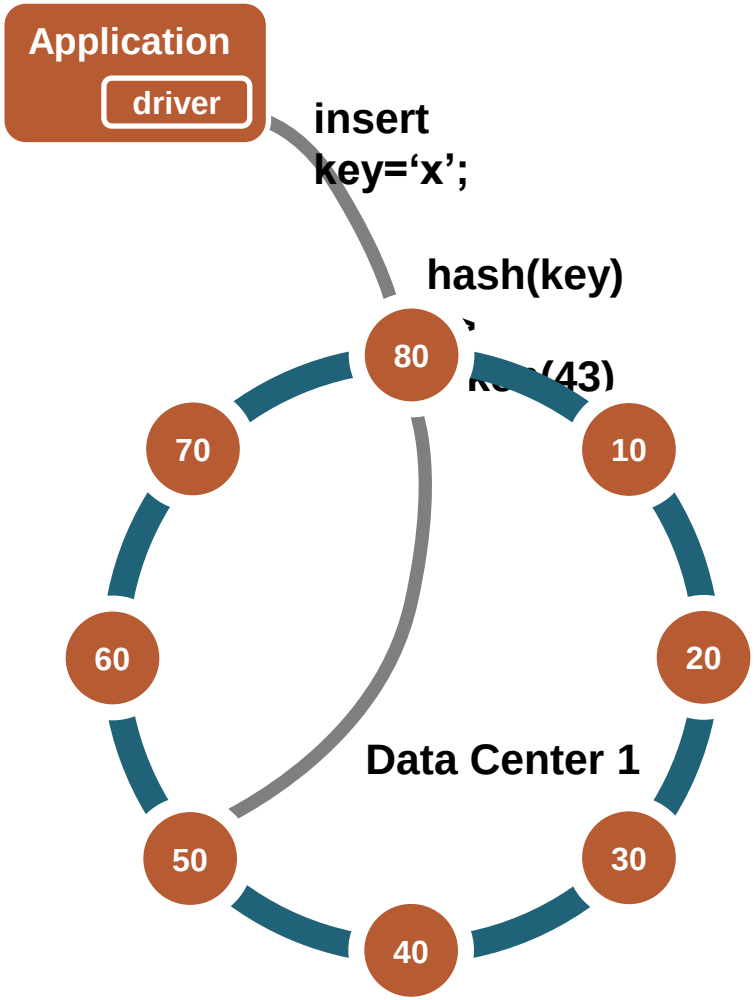
Driver manages connection pool

Driver has load balancing policies that can be applied

Each transaction has a different connection point in the cluster

Asynch operations are faster

Data Partitioning



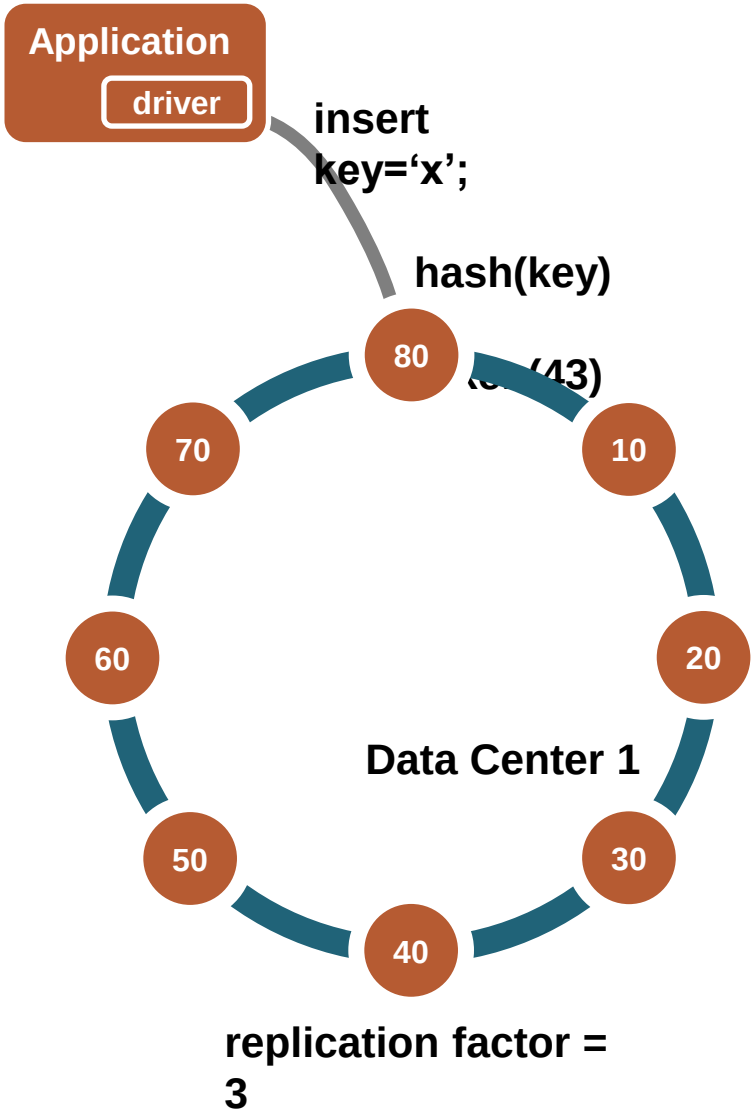
Driver selects the Coordinator node per operation via load balancing

Partitioned token ranges

- Random distribution
- Murmur3
- No hot spots

Each entire row lives on a node

Replication



Replication Factor = # of copies

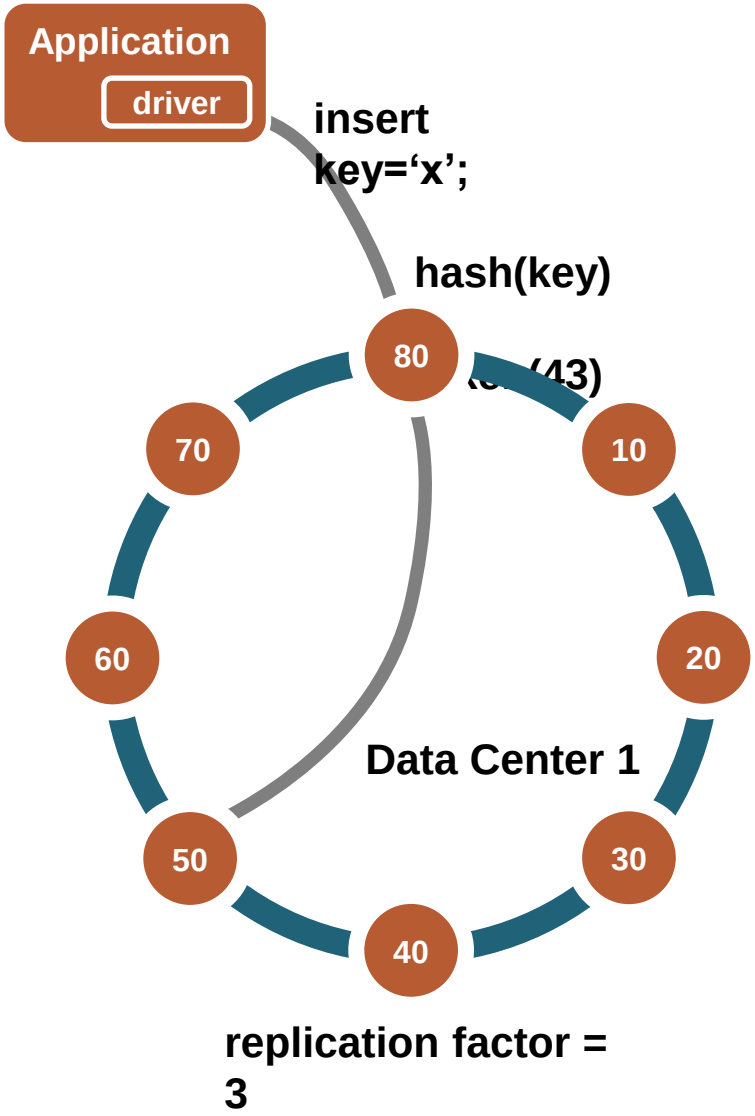
All replication operations in parallel

"Eventual" = micro- or milliseconds

No master or primary node

Each node acknowledges the op

Replication



Replication Factor = # of copies

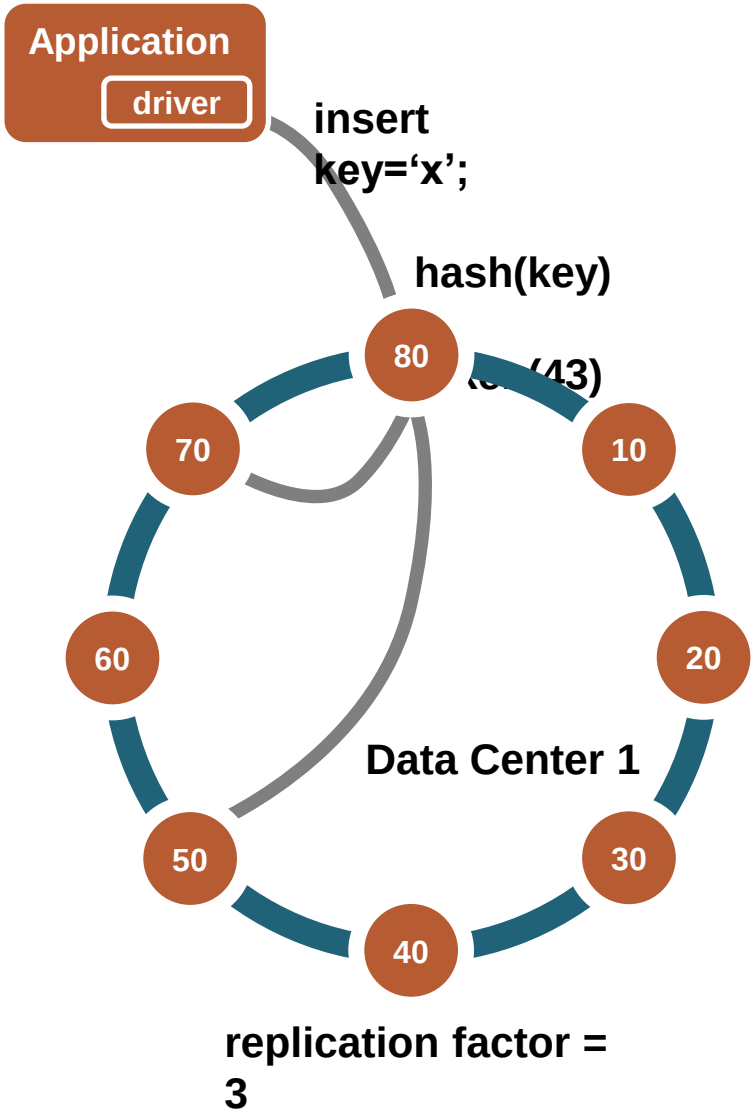
All replication operations in parallel

"Eventual" = micro- or milliseconds

No master or primary node

Each node acknowledges the op

Replication



Replication Factor = # of copies

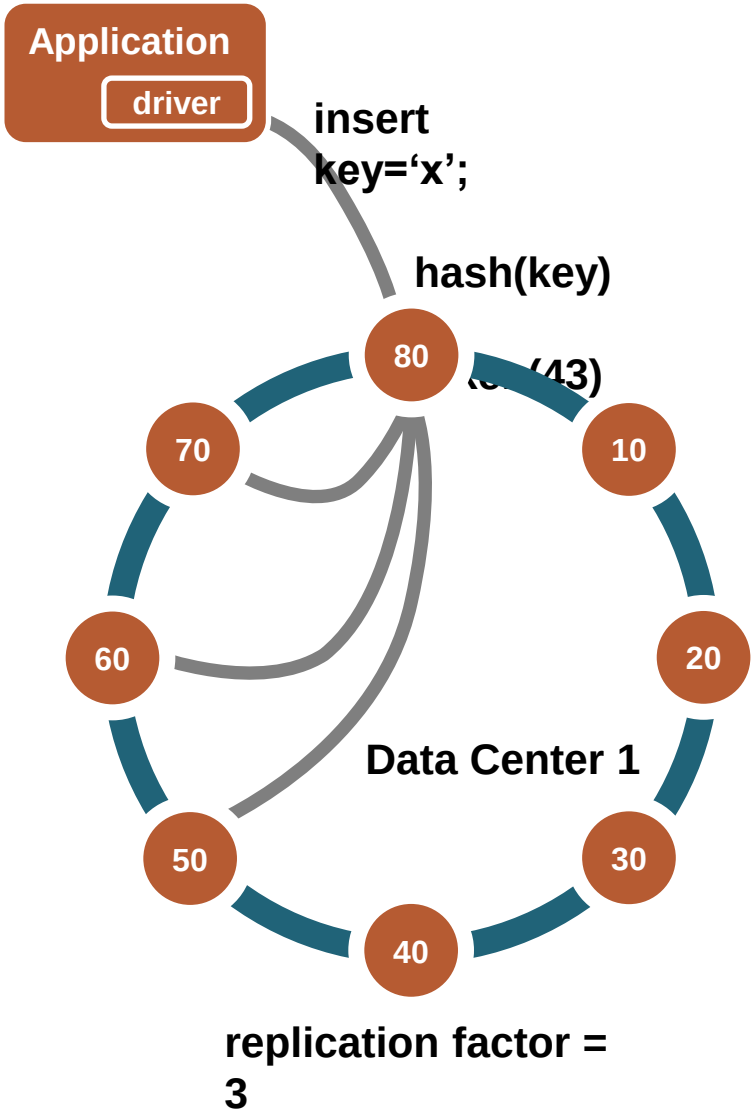
All replication operations in parallel

"Eventual" = micro- or milliseconds

No master or primary node

Each node acknowledges the op

Replication



Replication Factor = # of copies

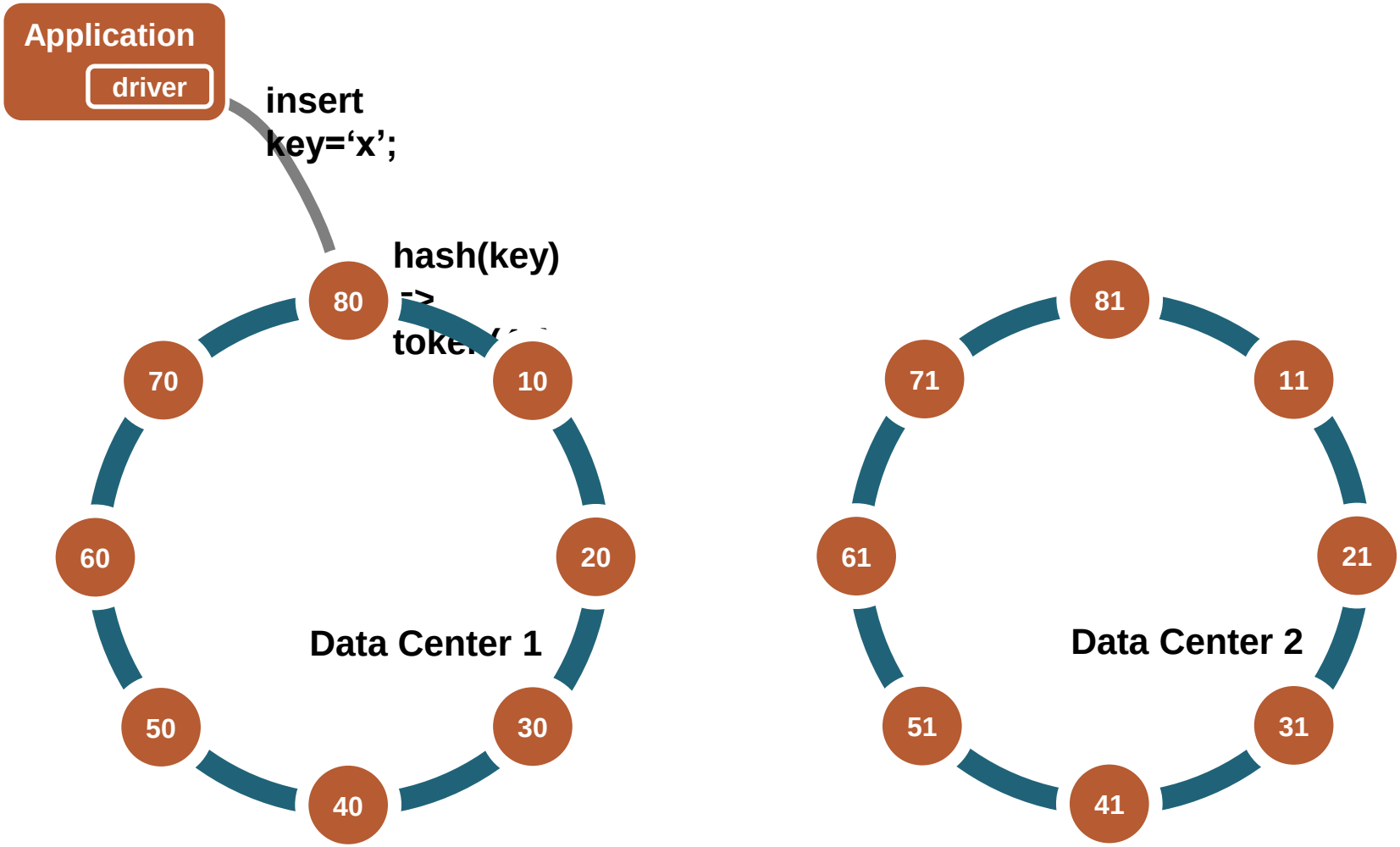
All replication operations in parallel

"Eventual" = micro- or milliseconds

No master or primary node

Each node acknowledges the op

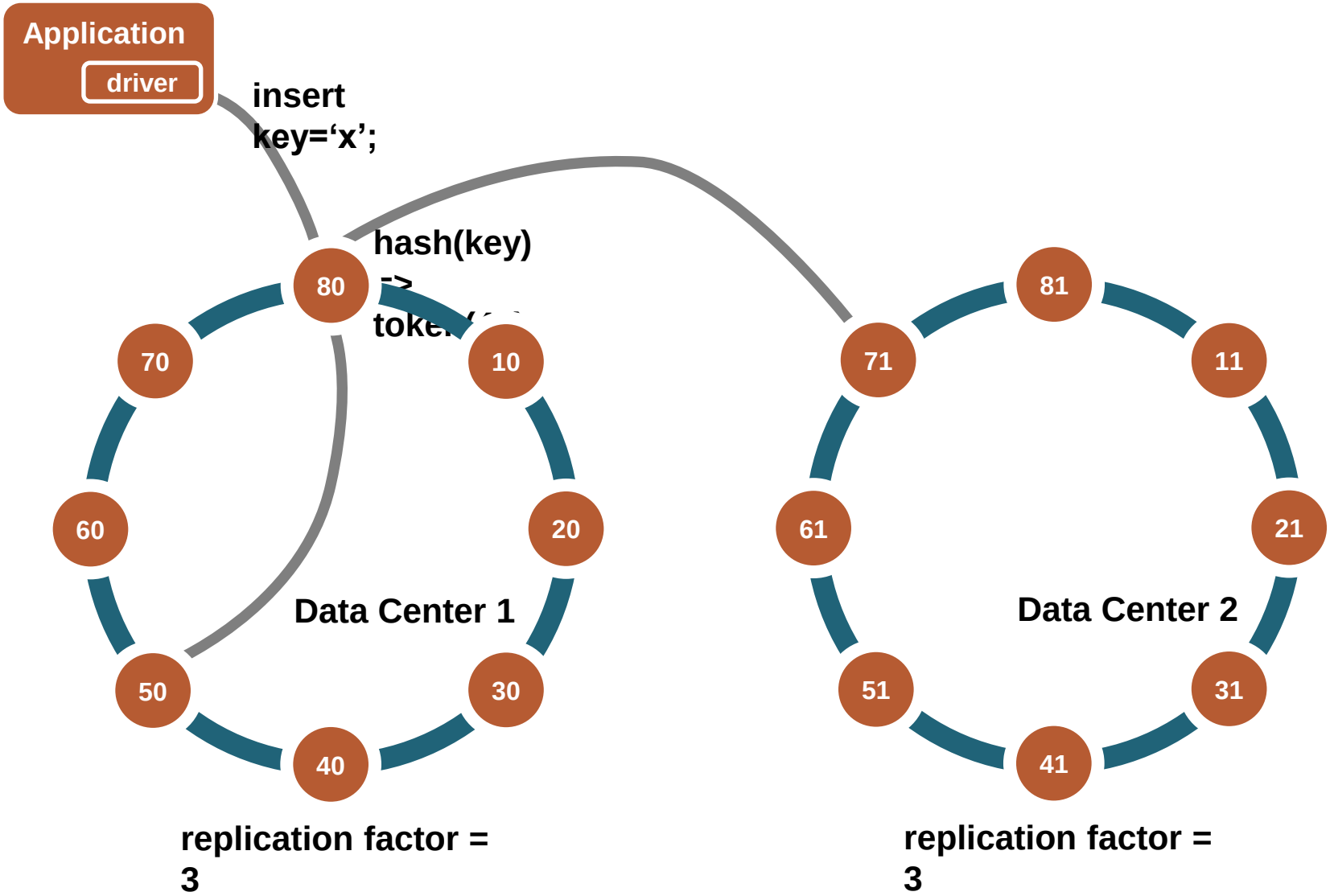
Multi-Data Center Replication



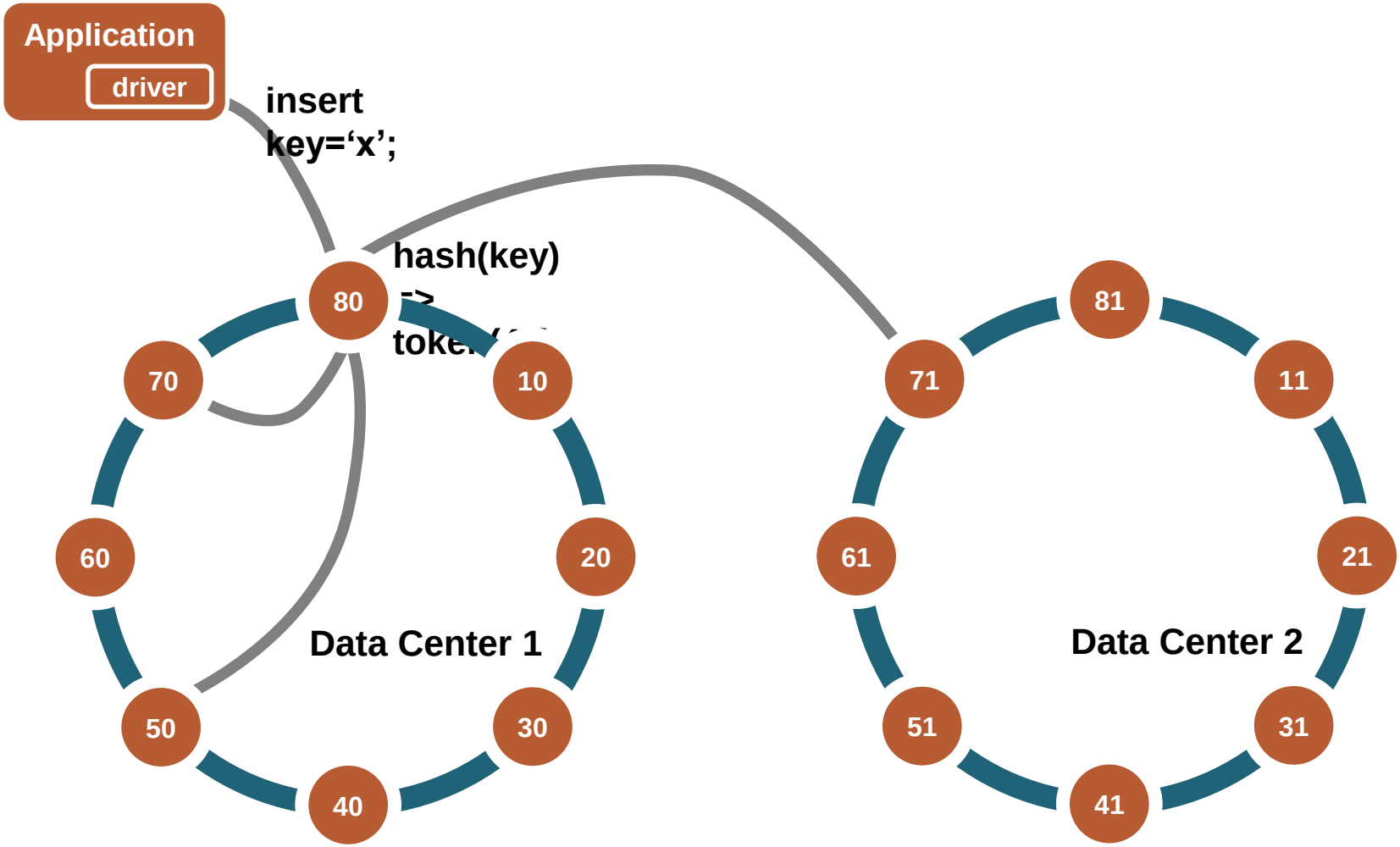
replication factor = 3

replication factor = 3

Multi-Data Center Replication



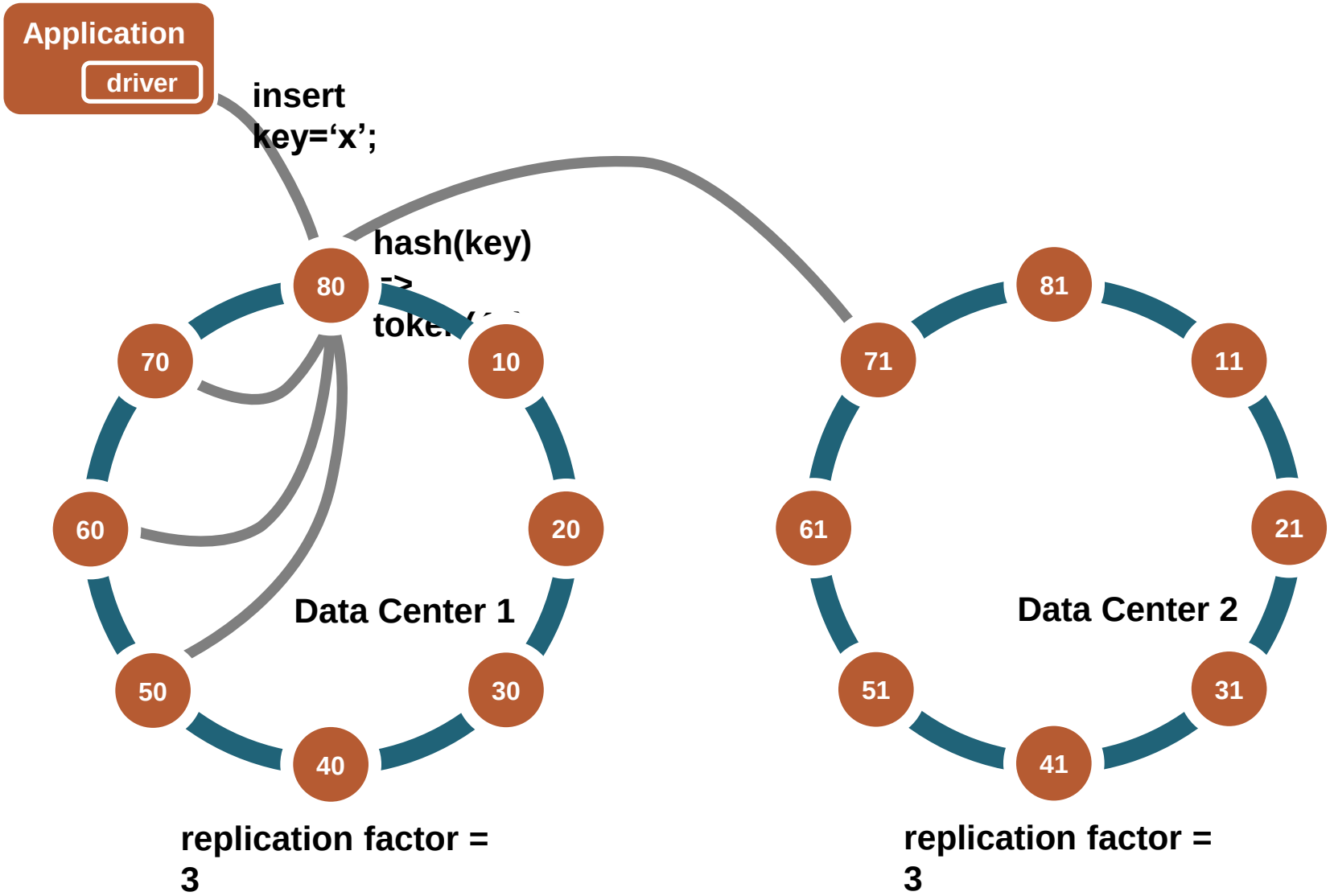
Multi-Data Center Replication



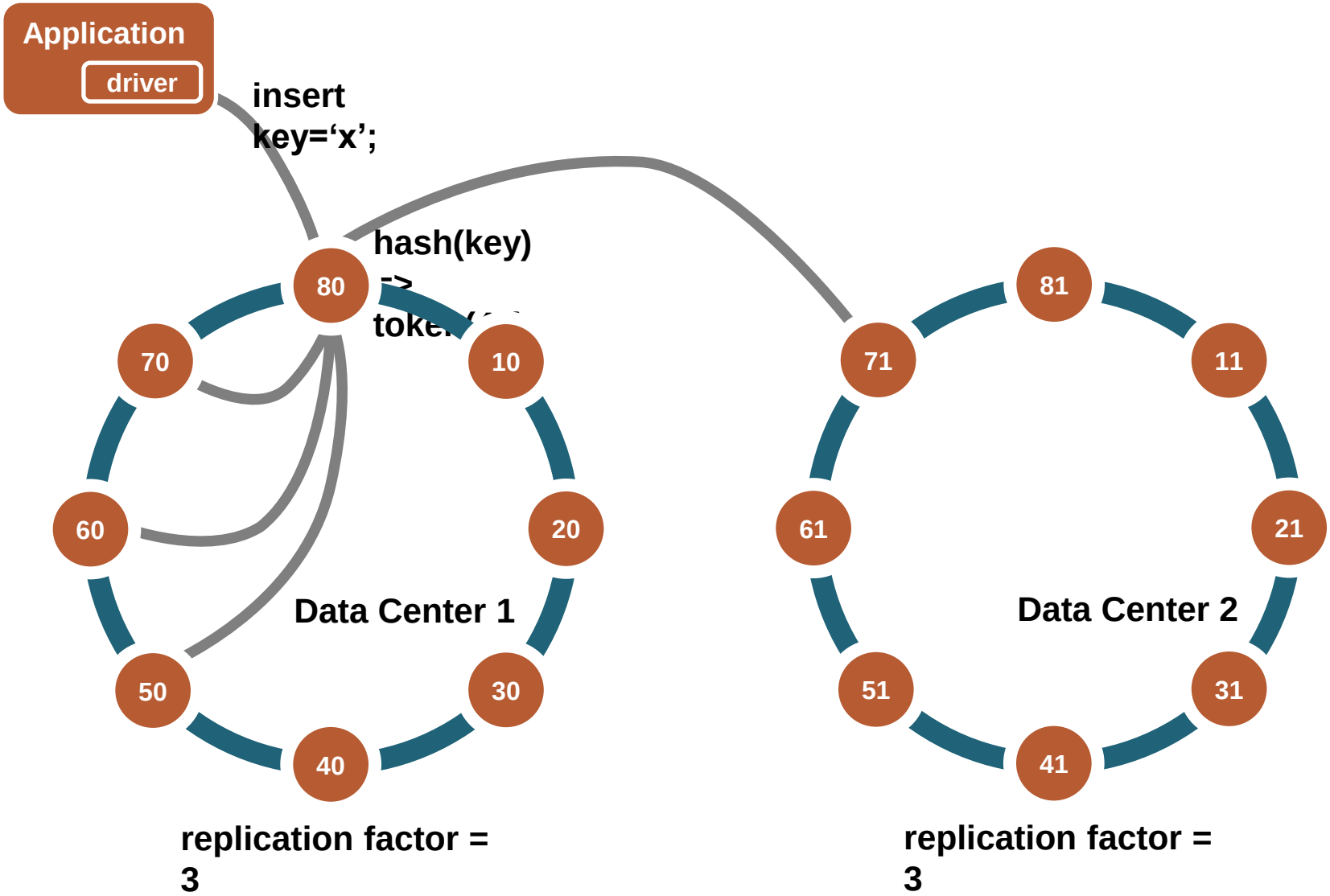
replication factor = 3

replication factor = 3

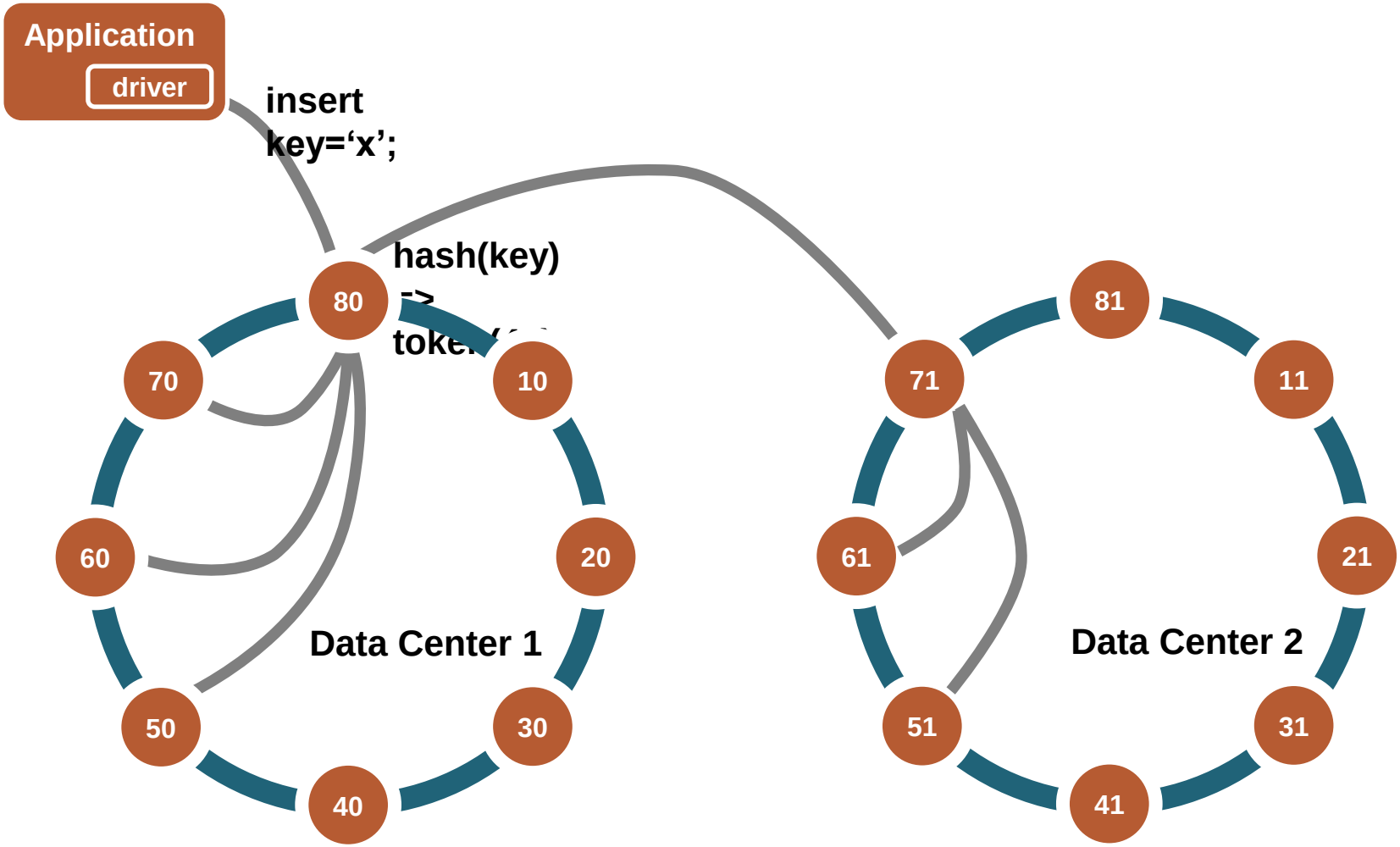
Multi-Data Center Replication



Multi-Data Center Replication



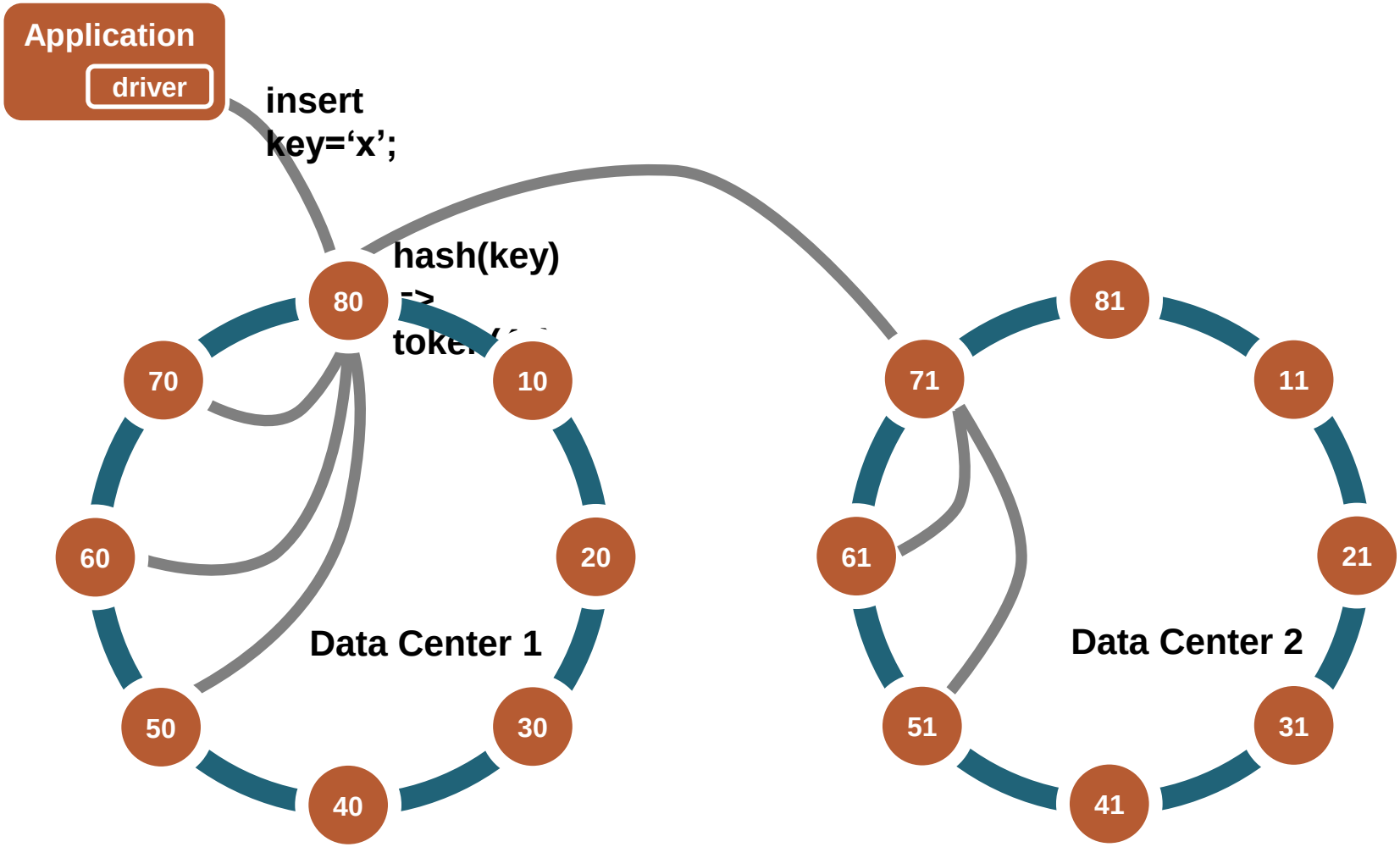
Multi-Data Center Replication



replication factor = 3

replication factor = 3

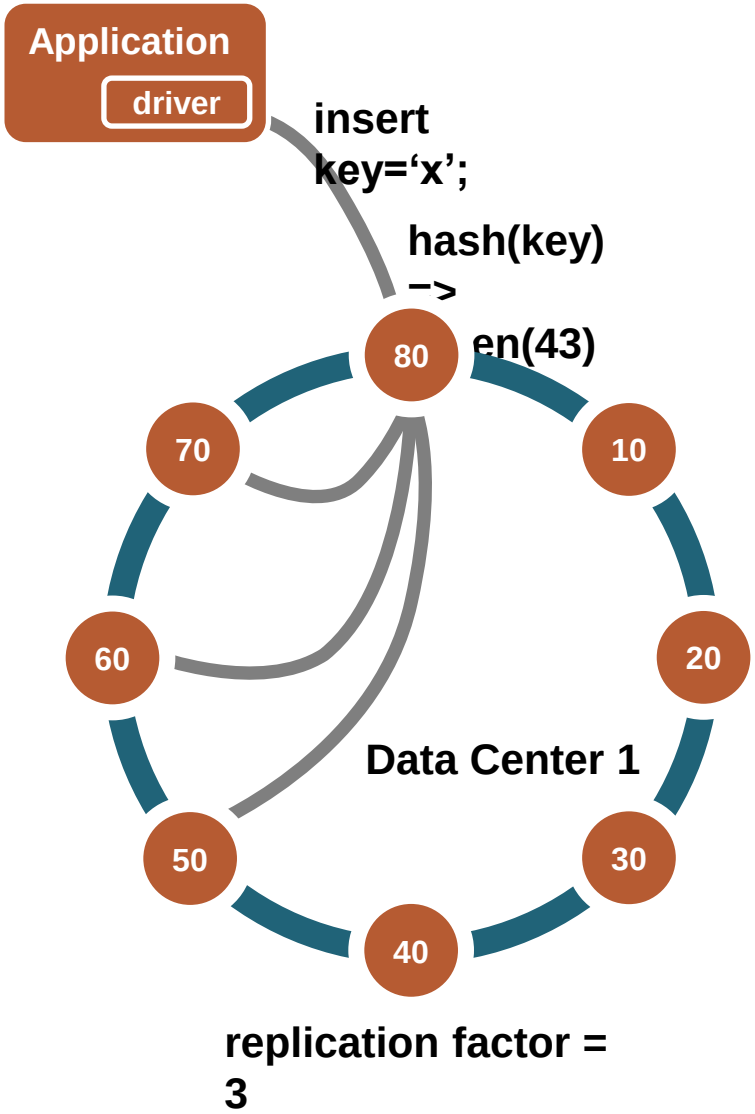
Multi-Data Center Replication



replication factor = 3

replication factor = 3

Consistency



Replication Factor = ?

Consistency Level = # of nodes to acknowledge an operation

read and write "consistency" per operation

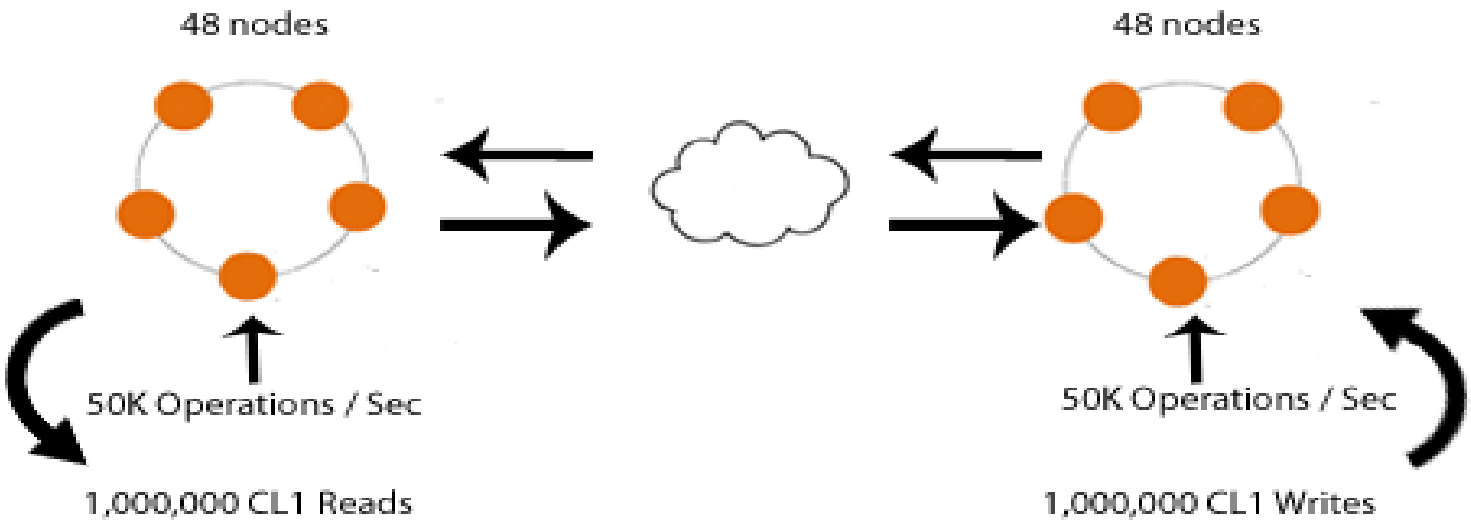
CL(write) + CL(read) > RF

→

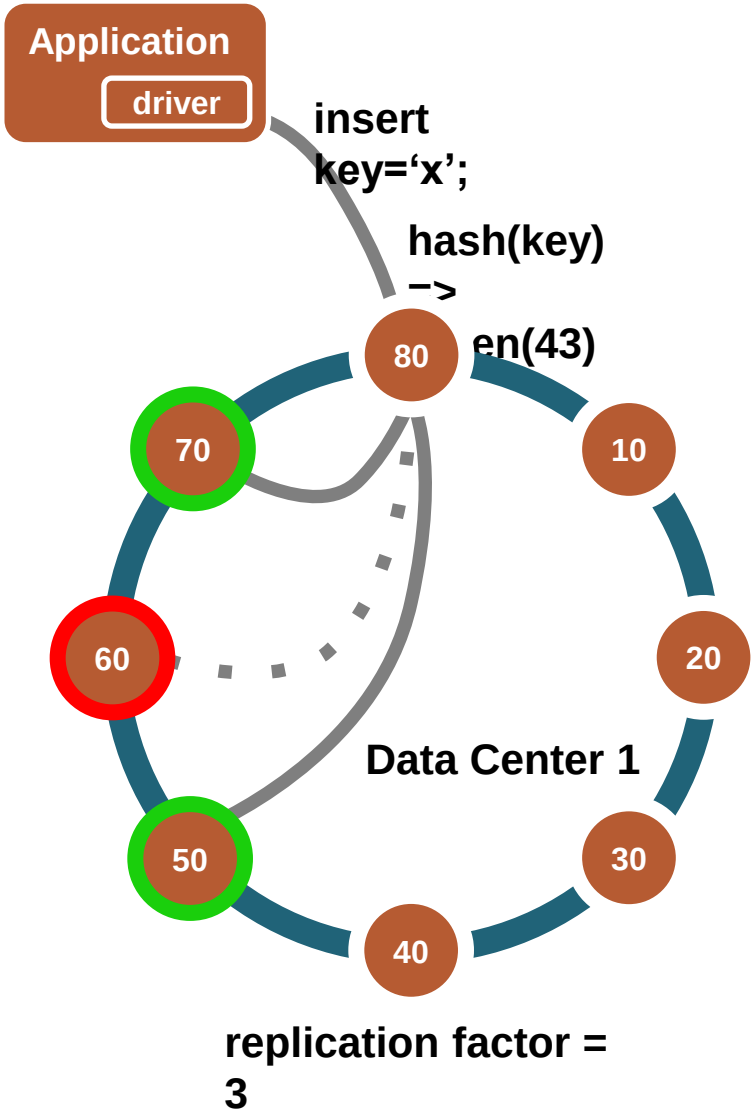
consistency

If needed, tune consistency for performance

Netflix Replication Experiment



Slow Node Anti-Entropy: Hints



IFF Node 60 is:

Slow enough to timeout ops

Down for a short time

Then…

Node 80 holds the op missed by 60

held ops are called Hints

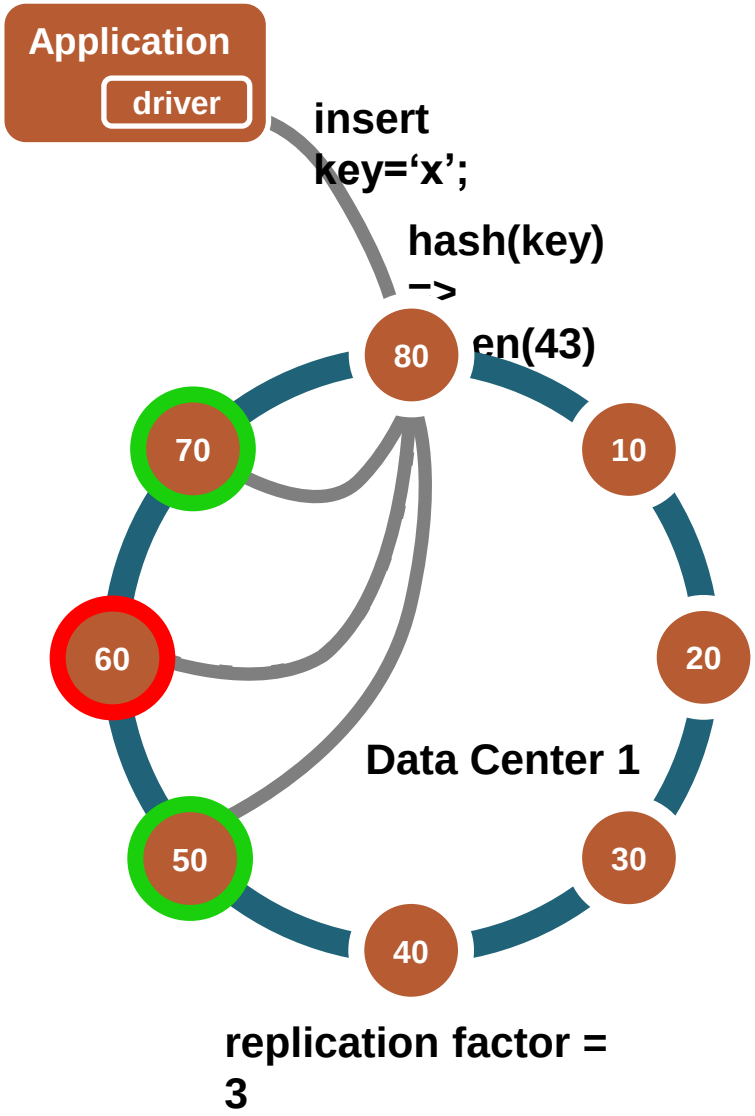
When node 60 becomes responsive

Node 80 sends the hints as ops

…until node 60 is synchronized

Read Repair

Slow Node Anti-Entropy: Hints



IFF Node 60 is:

Slow enough to timeout ops

Down for a short time

Then…

Node 80 holds the op missed by 60

held ops are called Hints

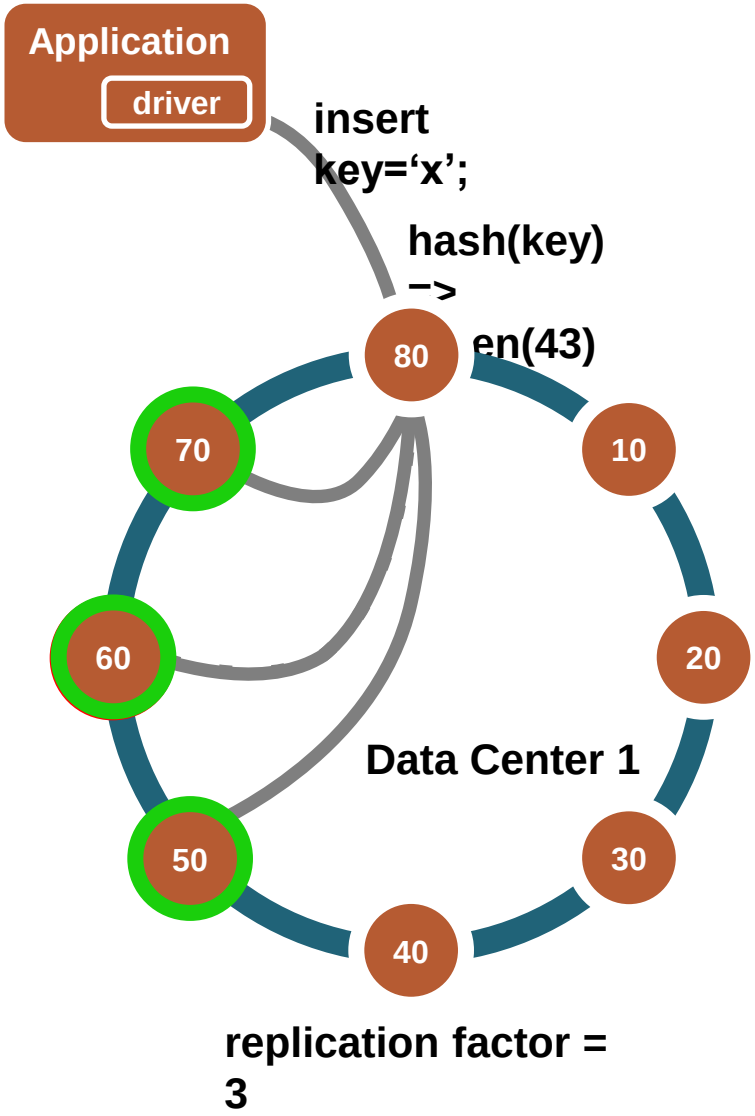
When node 60 becomes responsive

Node 80 sends the hints as ops

…until node 60 is synchronized

Read Repair

Slow Node Anti-Entropy: Hints



IFF Node 60 is:

Slow enough to timeout ops

Down for a short time

Then…

Node 80 holds the op missed by 60

held ops are called Hints

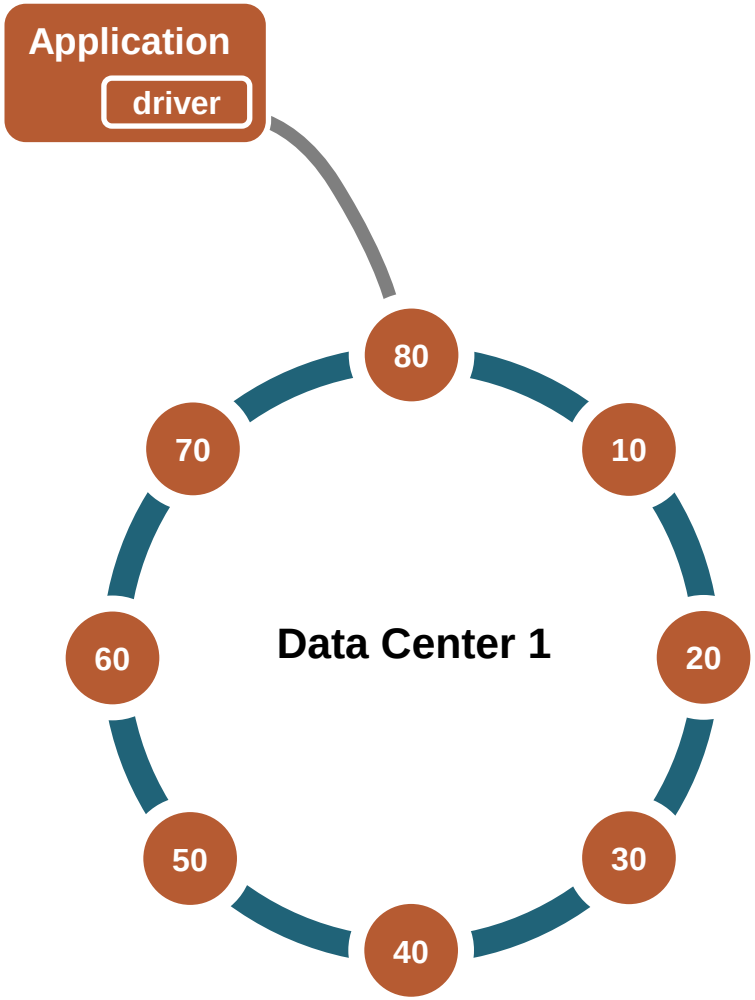
When node 60 becomes responsive

Node 80 sends the hints as ops

…until node 60 is synchronized

Read Repair

What if I need more nodes?



replication factor = 3

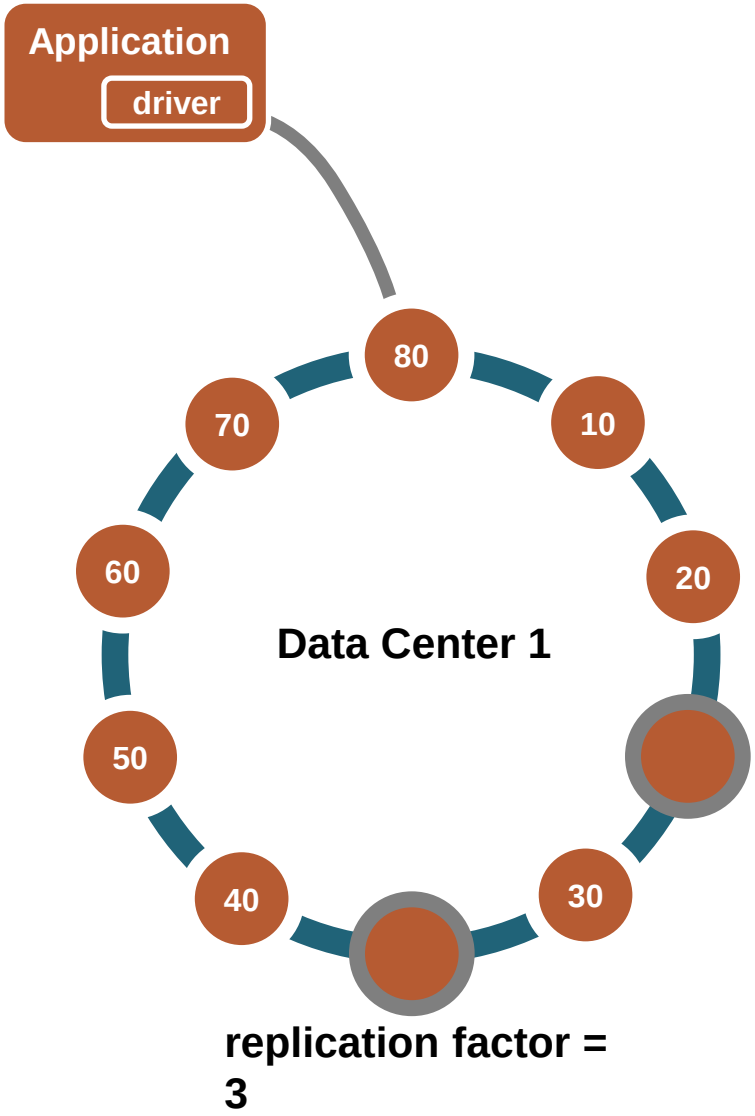
Data set size is growing

Need more TPS

- Client application demand
- Hardware limits being reached
- Looking for lower latency

Moving some tables to in-memory

Cluster Expansion: Add Nodes

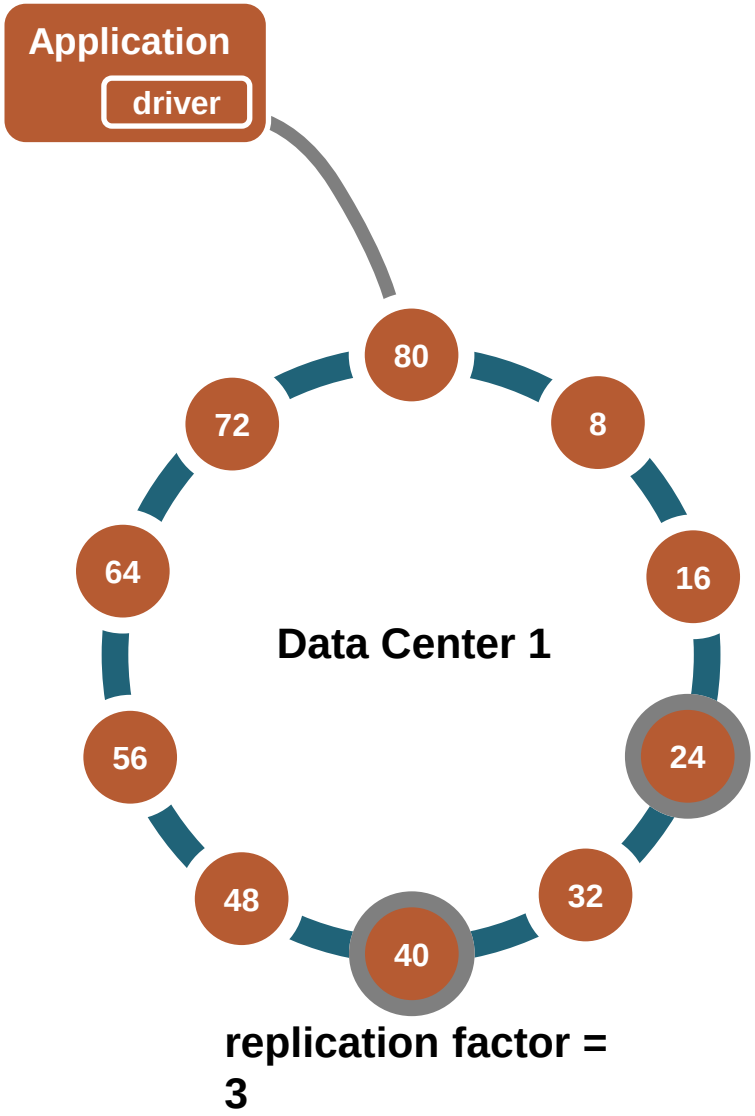


Introduce new nodes to cluster

Nodes do not yet own any token ranges (data)

Cluster operates as normal

Cluster Expansion: Rebalance



Rebalance = redistribution of token ranges around the cluster

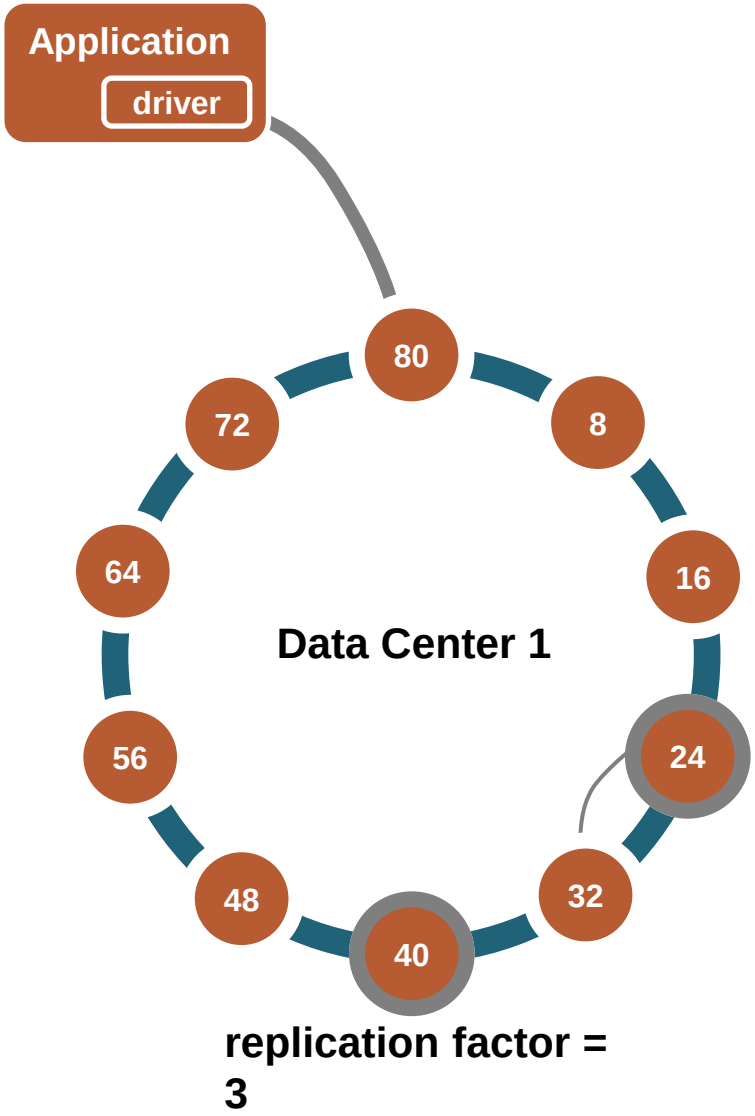
Data is streamed in small chunks to synchronize the cluster

- see “Vnodes”
- minimizes streaming
- distributes rebalance workload

New nodes begin taking ops

Cleanup = reclaims space on nodes dedicated to tokens no longer owned

Cluster Expansion: Rebalance



Rebalance = redistribution of token ranges around the cluster

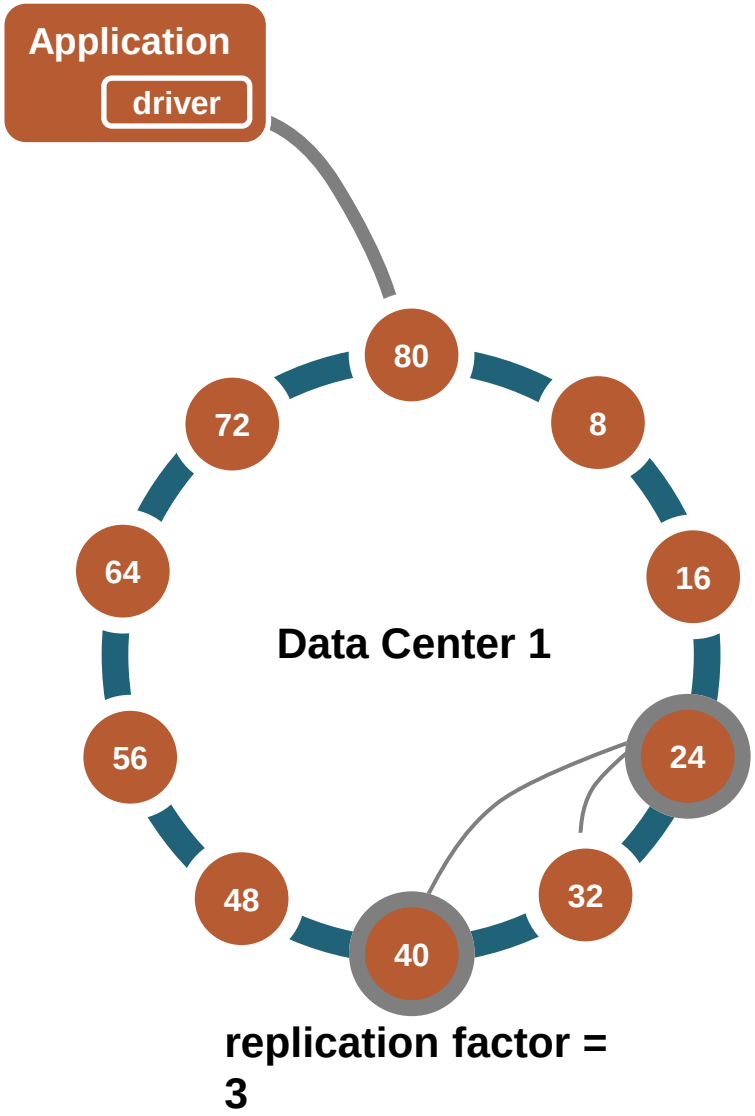
Data is streamed in small chunks to synchronize the cluster

- see “Vnodes”
- minimizes streaming
- distributes rebalance workload

New nodes begin taking ops

Cleanup = reclaims space on nodes dedicated to tokens no longer owned

Cluster Expansion: Rebalance



Rebalance = redistribution of token ranges around the cluster

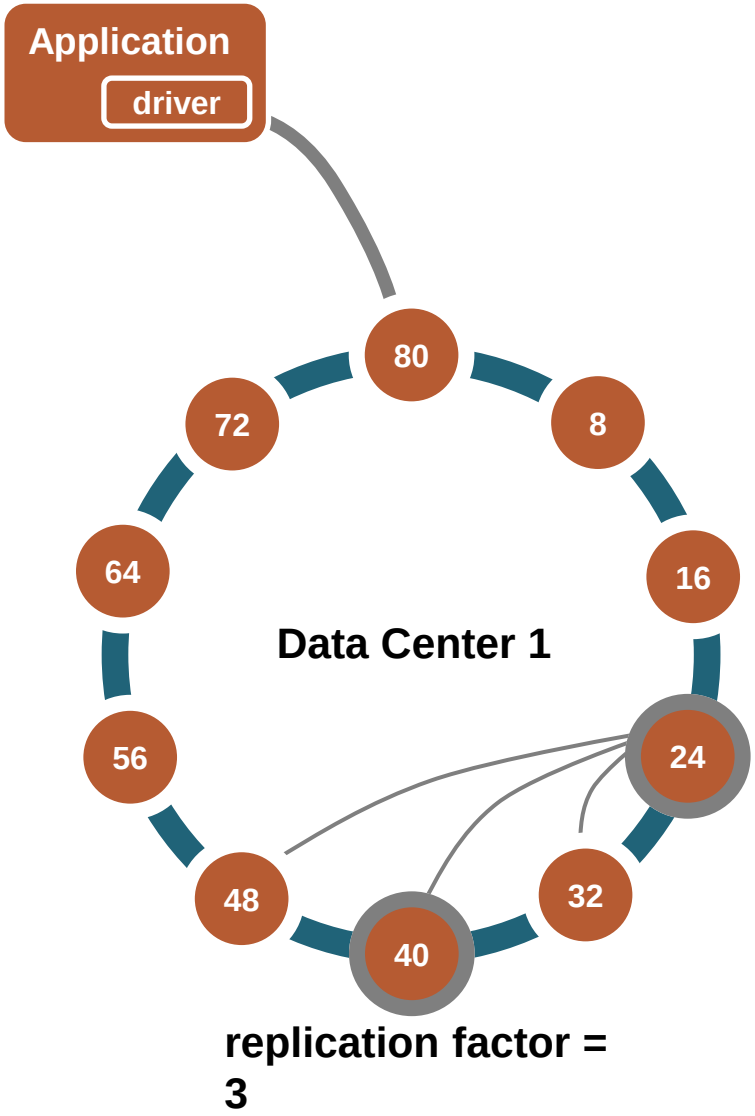
Data is streamed in small chunks to synchronize the cluster

- see “Vnodes”
- minimizes streaming
- distributes rebalance workload

New nodes begin taking ops

Cleanup = reclaims space on nodes dedicated to tokens no longer owned

Cluster Expansion: Rebalance



Rebalance = redistribution of token ranges around the cluster

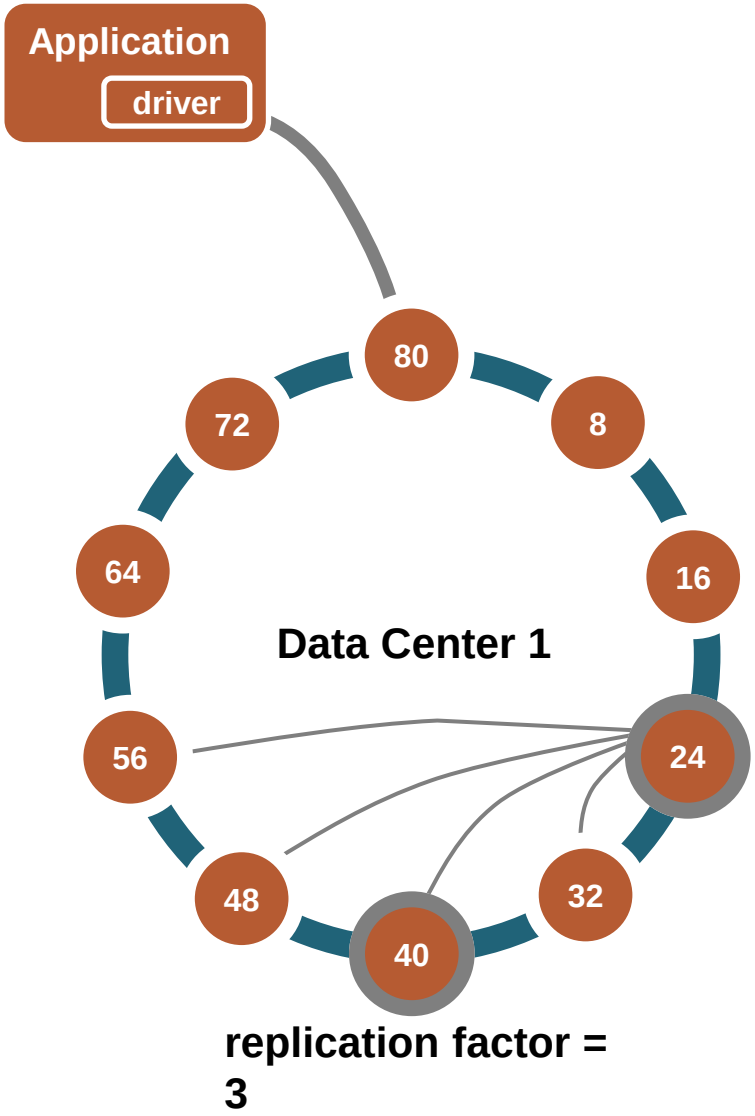
Data is streamed in small chunks to synchronize the cluster

- see “Vnodes”
- minimizes streaming
- distributes rebalance workload

New nodes begin taking ops

Cleanup = reclaims space on nodes dedicated to tokens no longer owned

Cluster Expansion: Rebalance



Rebalance = redistribution of token ranges around the cluster

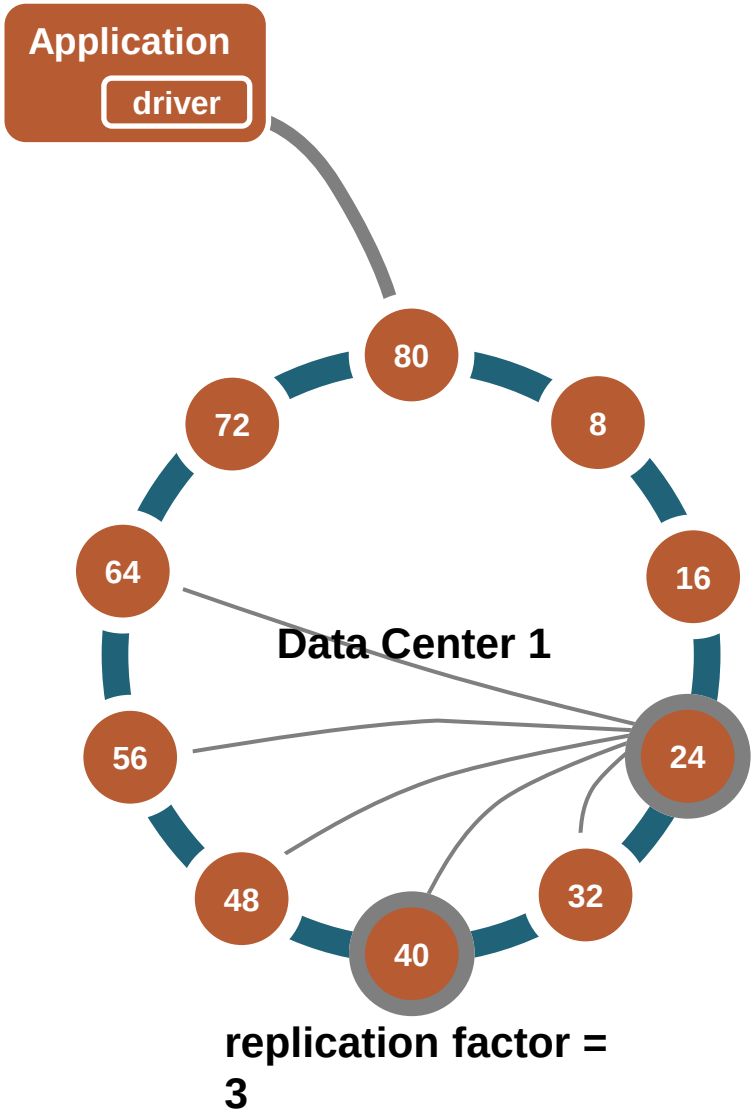
Data is streamed in small chunks to synchronize the cluster

- see “Vnodes”
- minimizes streaming
- distributes rebalance workload

New nodes begin taking ops

Cleanup = reclaims space on nodes dedicated to tokens no longer owned

Cluster Expansion: Rebalance



Rebalance = redistribution of token ranges around the cluster

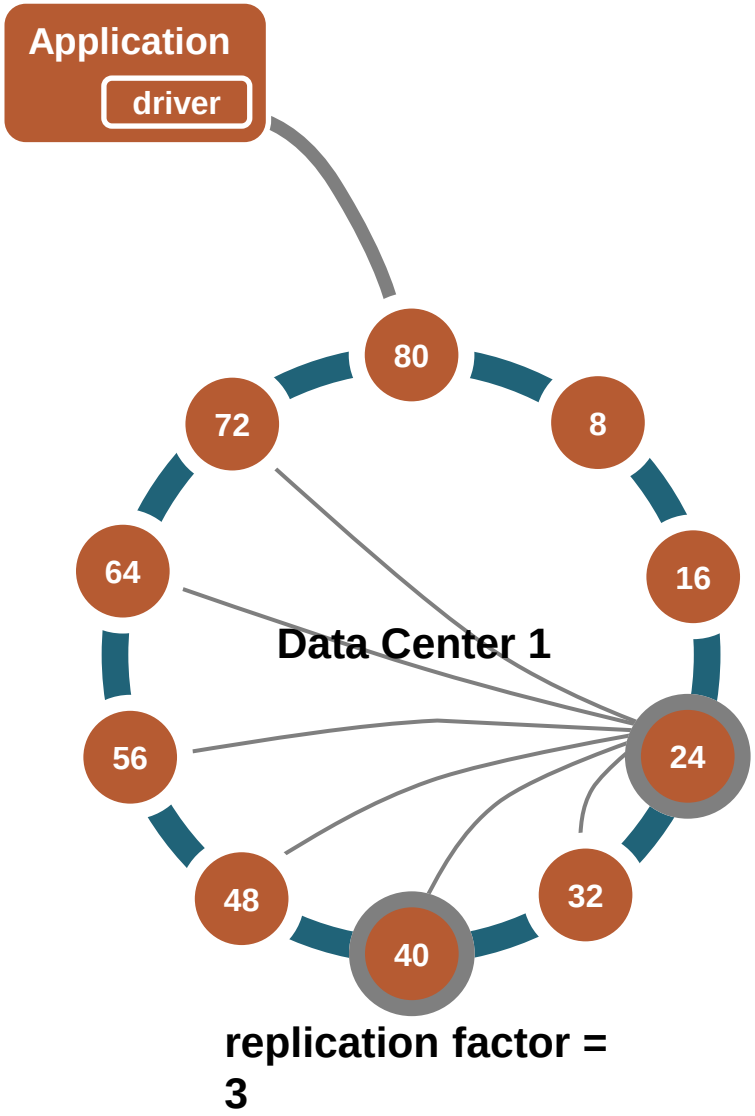
Data is streamed in small chunks to synchronize the cluster

- see “Vnodes”
- minimizes streaming
- distributes rebalance workload

New nodes begin taking ops

Cleanup = reclaims space on nodes dedicated to tokens no longer owned

Cluster Expansion: Rebalance



Rebalance = redistribution of token ranges around the cluster

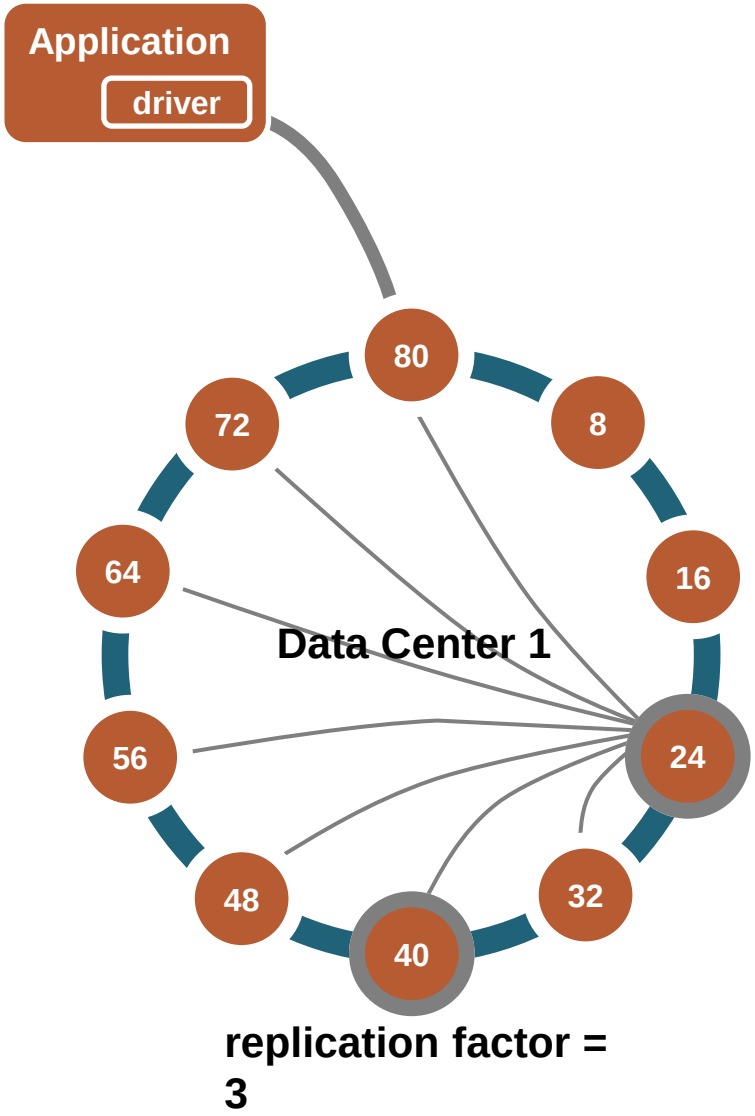
Data is streamed in small chunks to synchronize the cluster

- see “Vnodes”
- minimizes streaming
- distributes rebalance workload

New nodes begin taking ops

Cleanup = reclaims space on nodes dedicated to tokens no longer owned

Cluster Expansion: Rebalance



Rebalance = redistribution of token ranges around the cluster

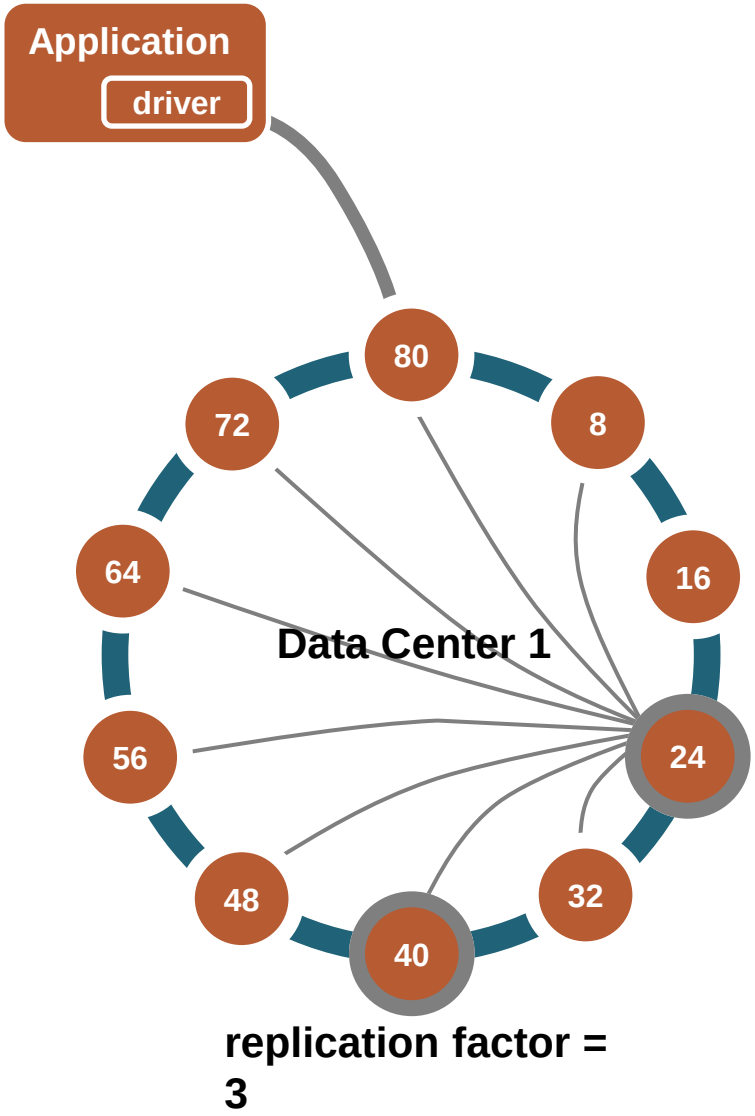
Data is streamed in small chunks to synchronize the cluster

- see “Vnodes”
- minimizes streaming
- distributes rebalance workload

New nodes begin taking ops

Cleanup = reclaims space on nodes dedicated to tokens no longer owned

Cluster Expansion: Rebalance



Rebalance = redistribution of token ranges around the cluster

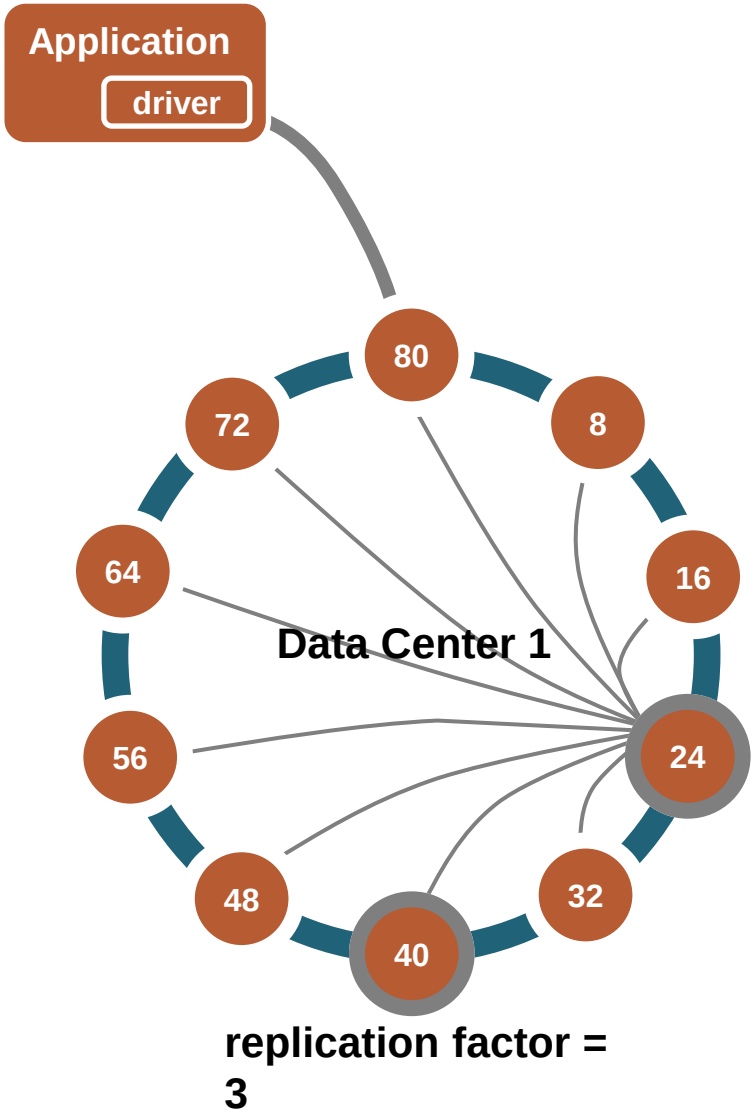
Data is streamed in small chunks to synchronize the cluster

- see “Vnodes”
- minimizes streaming
- distributes rebalance workload

New nodes begin taking ops

Cleanup = reclaims space on nodes dedicated to tokens no longer owned

Cluster Expansion: Rebalance



Rebalance = redistribution of token ranges around the cluster

Data is streamed in small chunks to synchronize the cluster

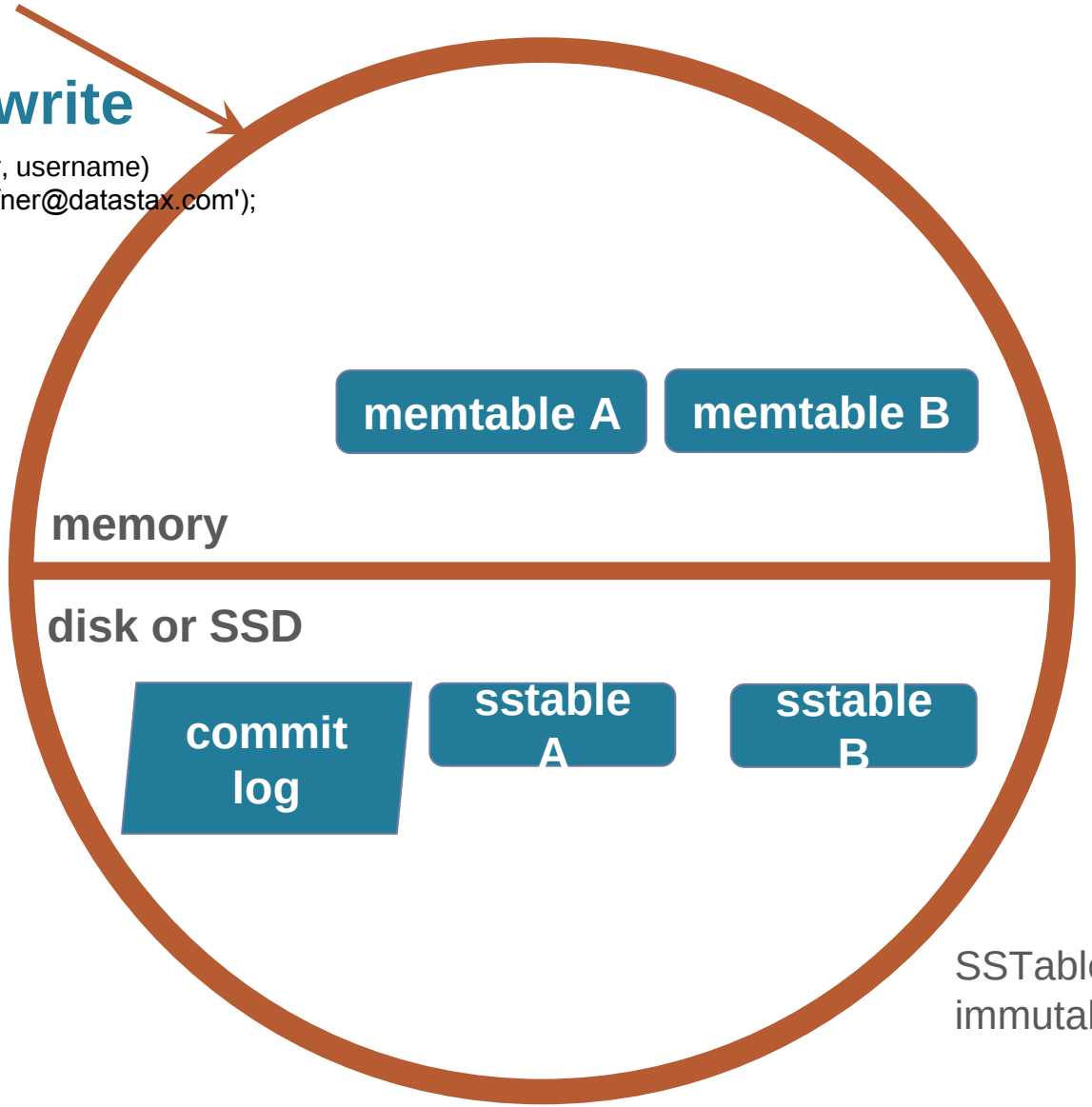
- see “Vnodes”
- minimizes streaming
- distributes rebalance workload

New nodes begin taking ops

Cleanup = reclaims space on nodes dedicated to tokens no longer owned

Read and Write path

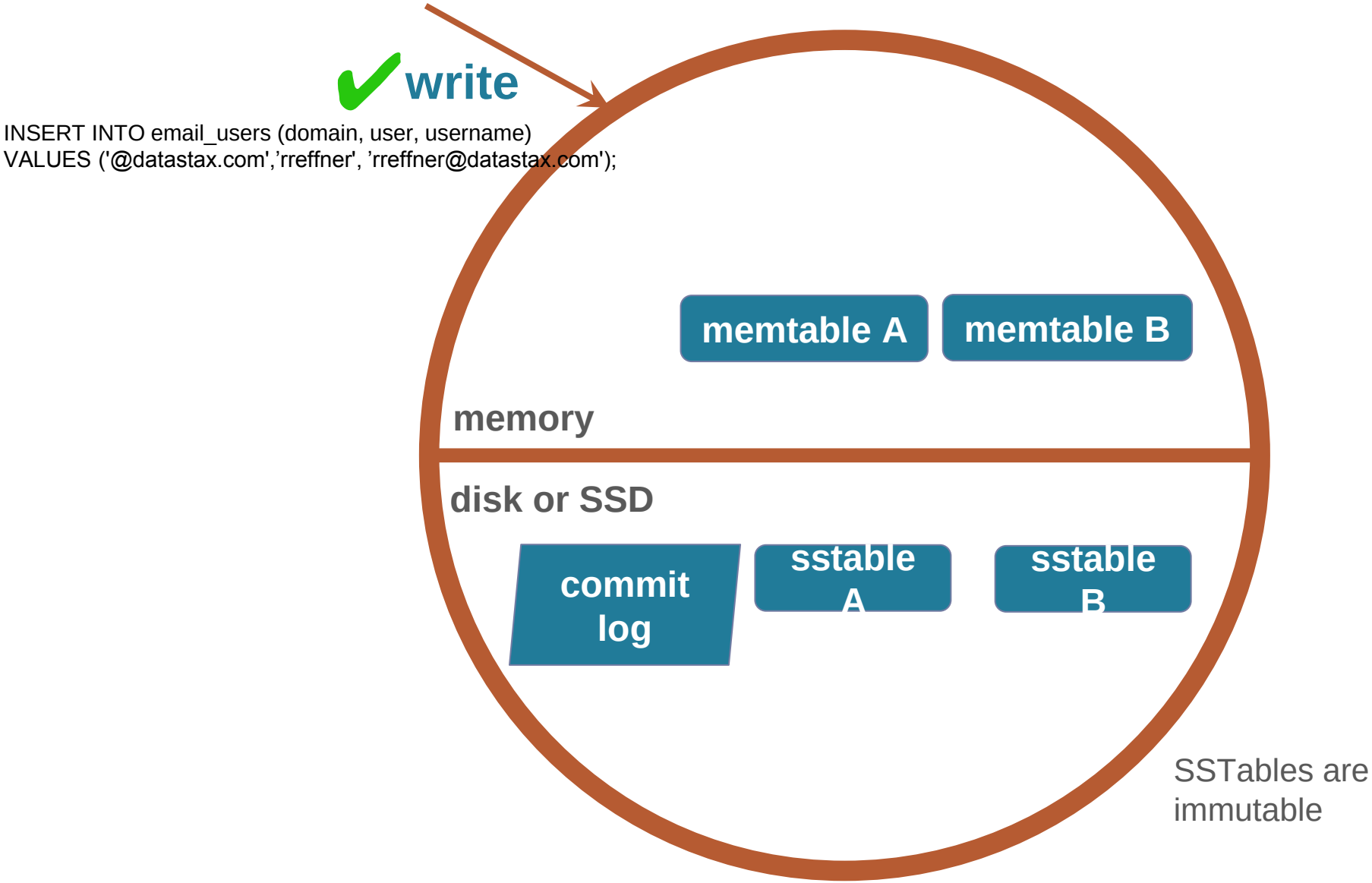
Write Path



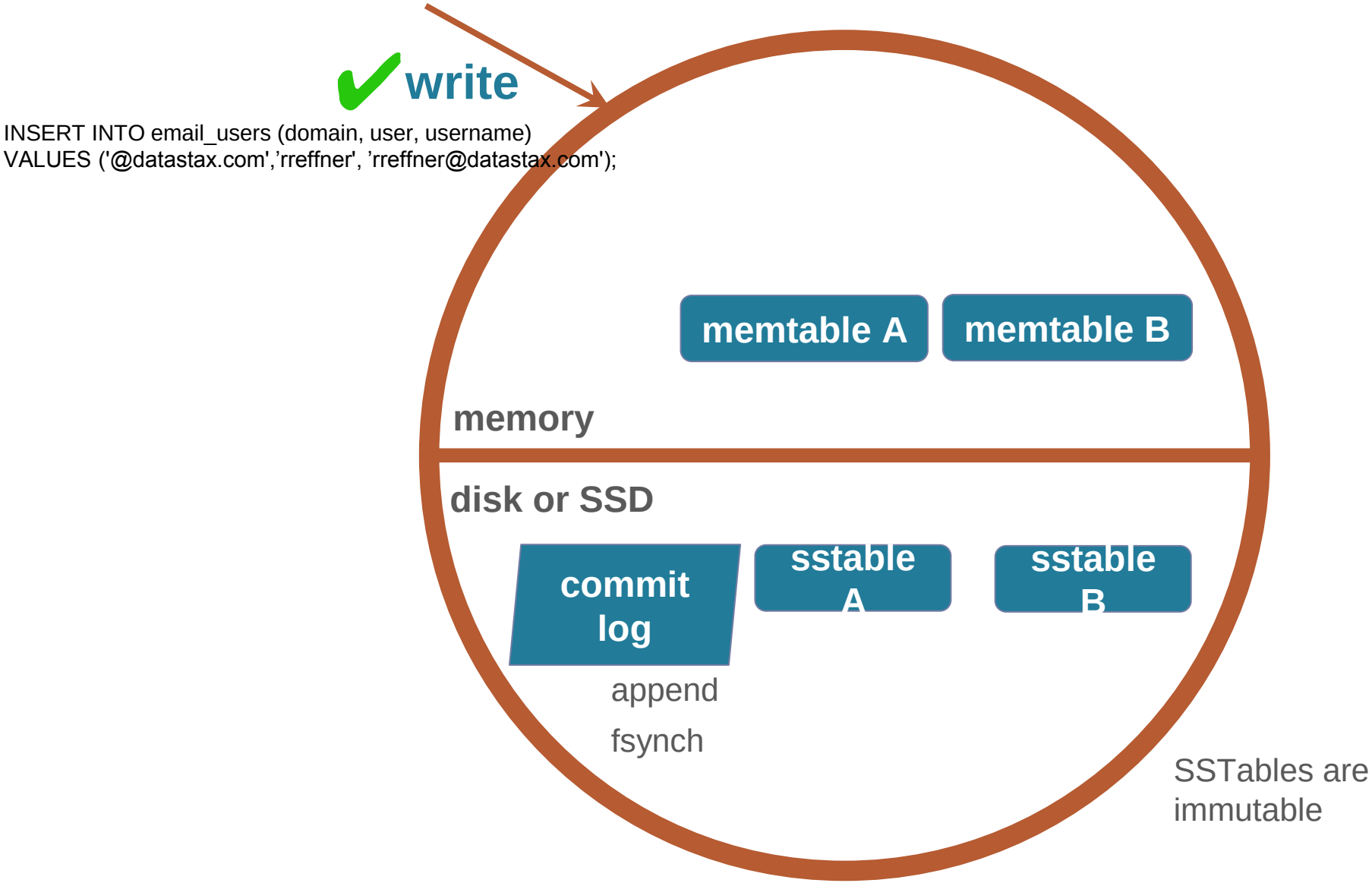

```
INSERT INTO email_users (domain, user, username)
VALUES ('@datastax.com','rreffner', 'rreffner@datastax.com');
```

SSTables are immutable

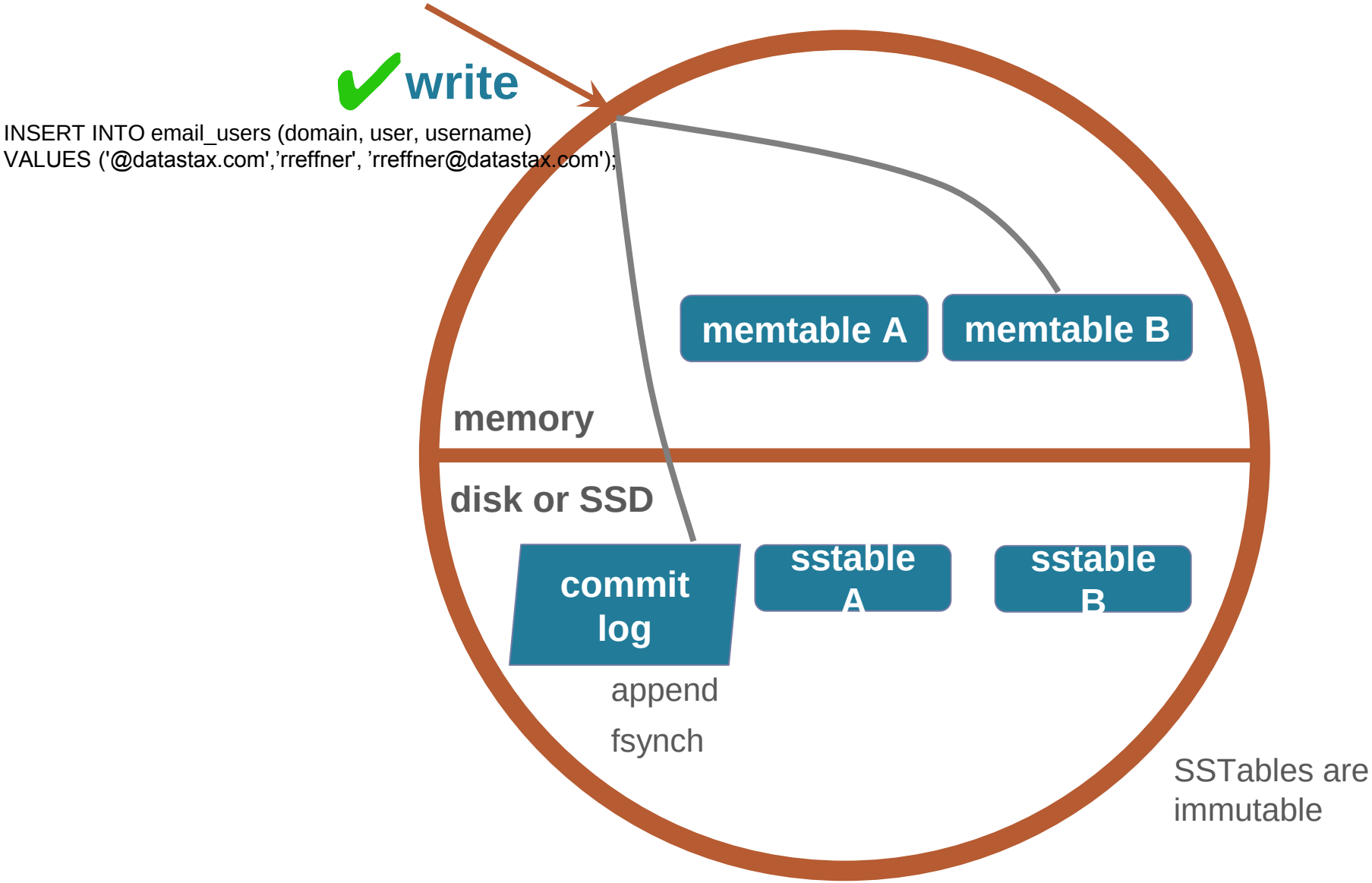
Write Path


write
INSERT INTO email_users (domain, user, username)
VALUES ('@datastax.com','rreffner', 'rreffner@datastax.com');
memtable A
memtable B
memory
disk or SSD
commit log
sstable A
sstable B
SSTables are immutable

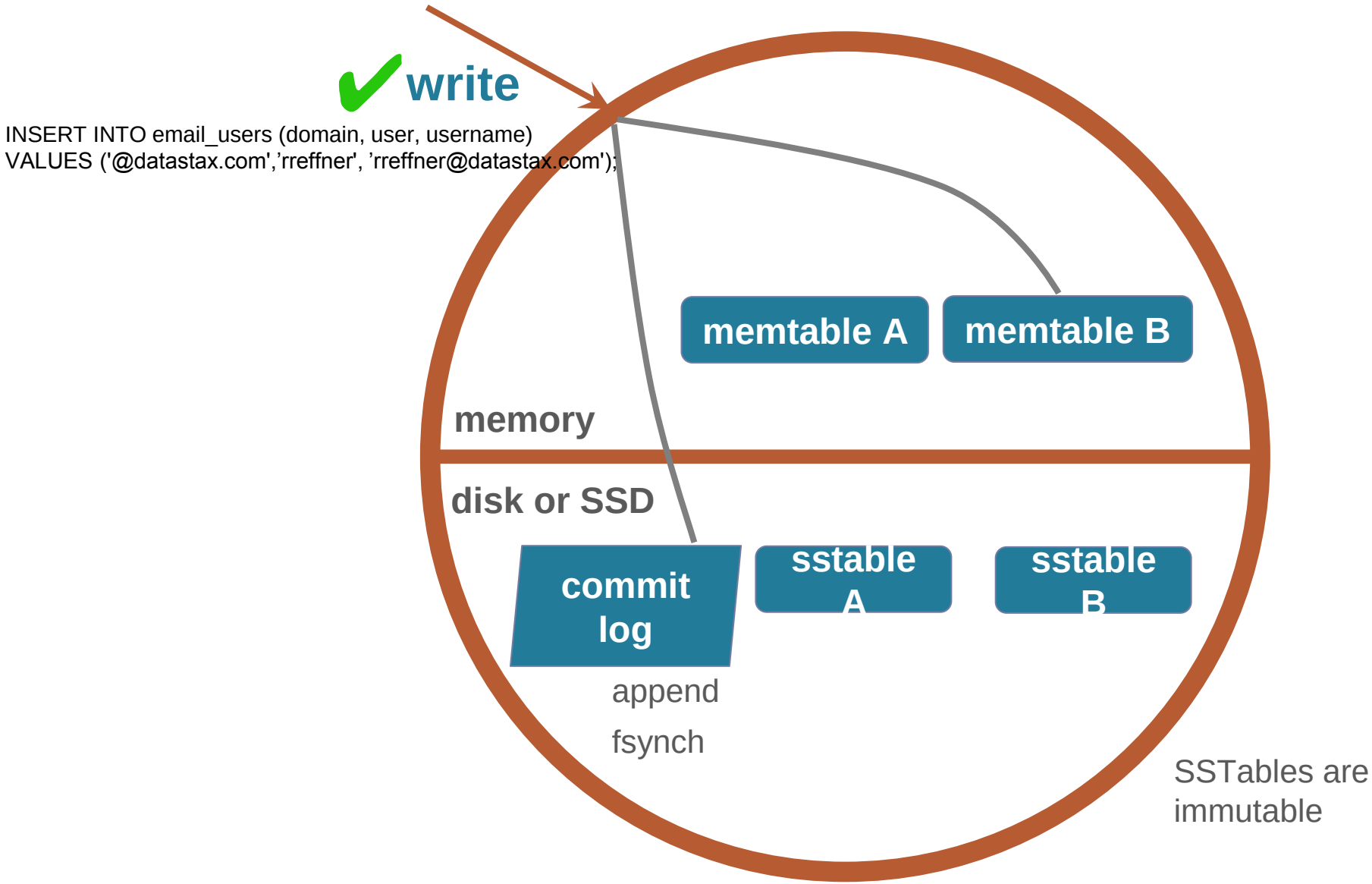
Write Path


write
INSERT INTO email_users (domain, user, username)
VALUES ('@datastax.com','rreffner', 'rreffner@datastax.com');
memtable A
memtable B
memory
disk or SSD
commit log
sstable A
sstable B
append
fsynch
SSTables are immutable

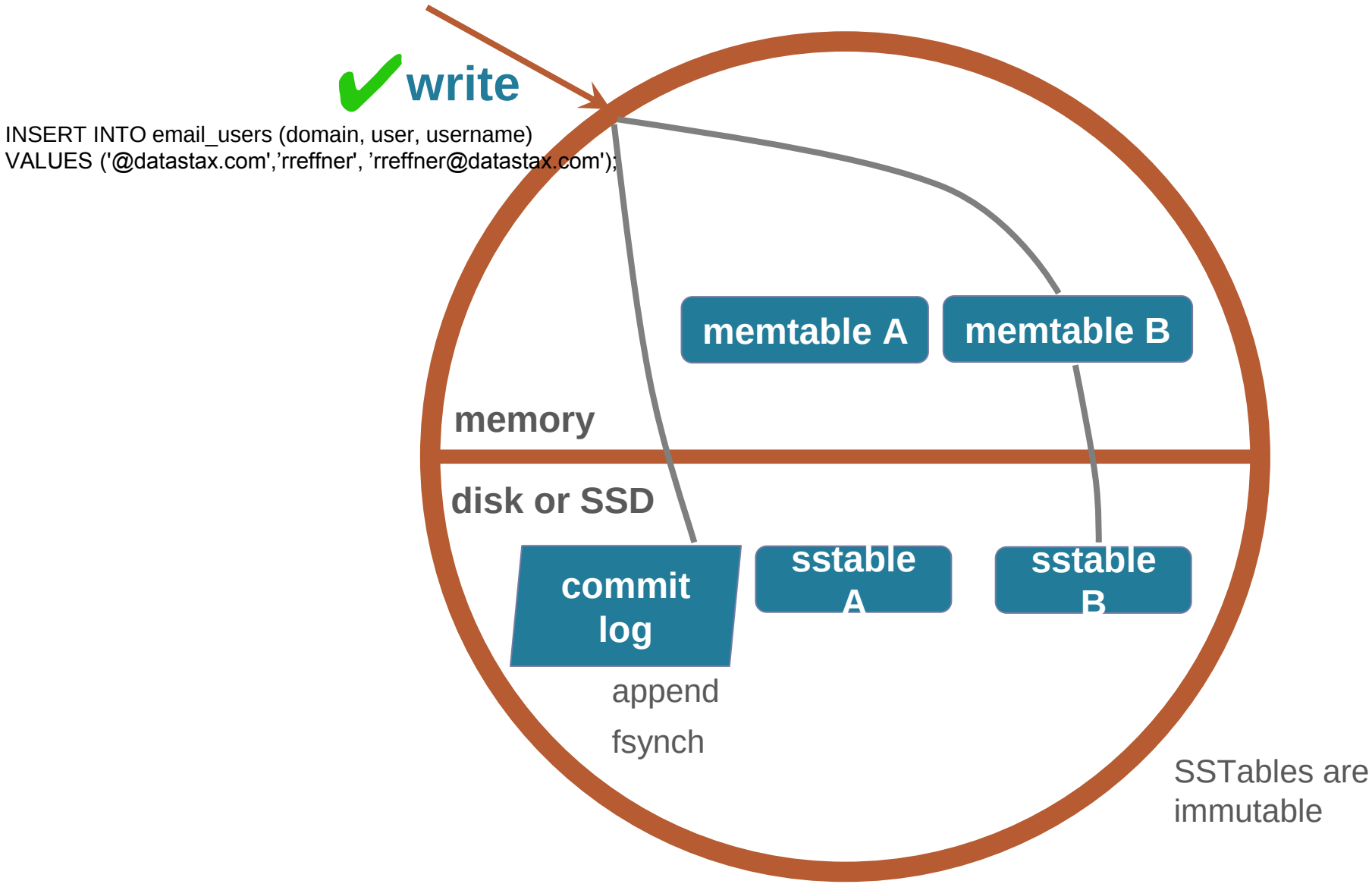
Write Path


write
INSERT INTO email_users (domain, user, username)
VALUES ('@datastax.com','rreffner', 'rreffner@datastax.com');
memtable A
memtable B
memory
disk or SSD
commit log
sstable A
sstable B
append
fsynch
SSTables are immutable

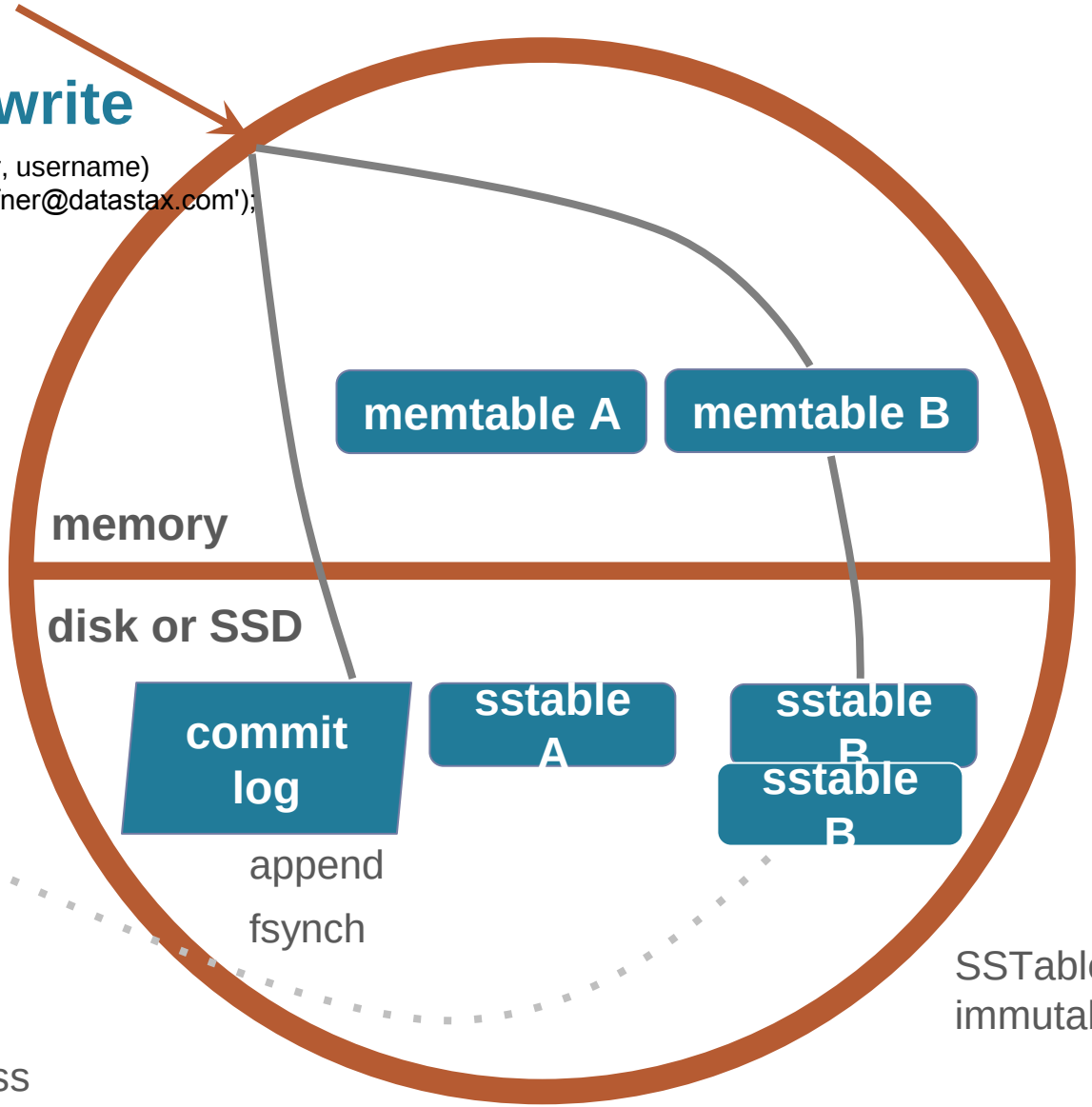
Write Path


write
INSERT INTO email_users (domain, user, username)
VALUES ('@datastax.com','rreffner', 'rreffner@datastax.com');
memtable A
memtable B
memory
disk or SSD
commit log
sstable A
sstable B
append
fsynch
SSTables are immutable

Write Path


write
INSERT INTO email_users (domain, user, username)
VALUES ('@datastax.com','rreffner', 'rreffner@datastax.com');
memtable A
memtable B
memory
disk or SSD
commit log
sstable A
sstable B
append
fsynch
SSTables are immutable

Write Path



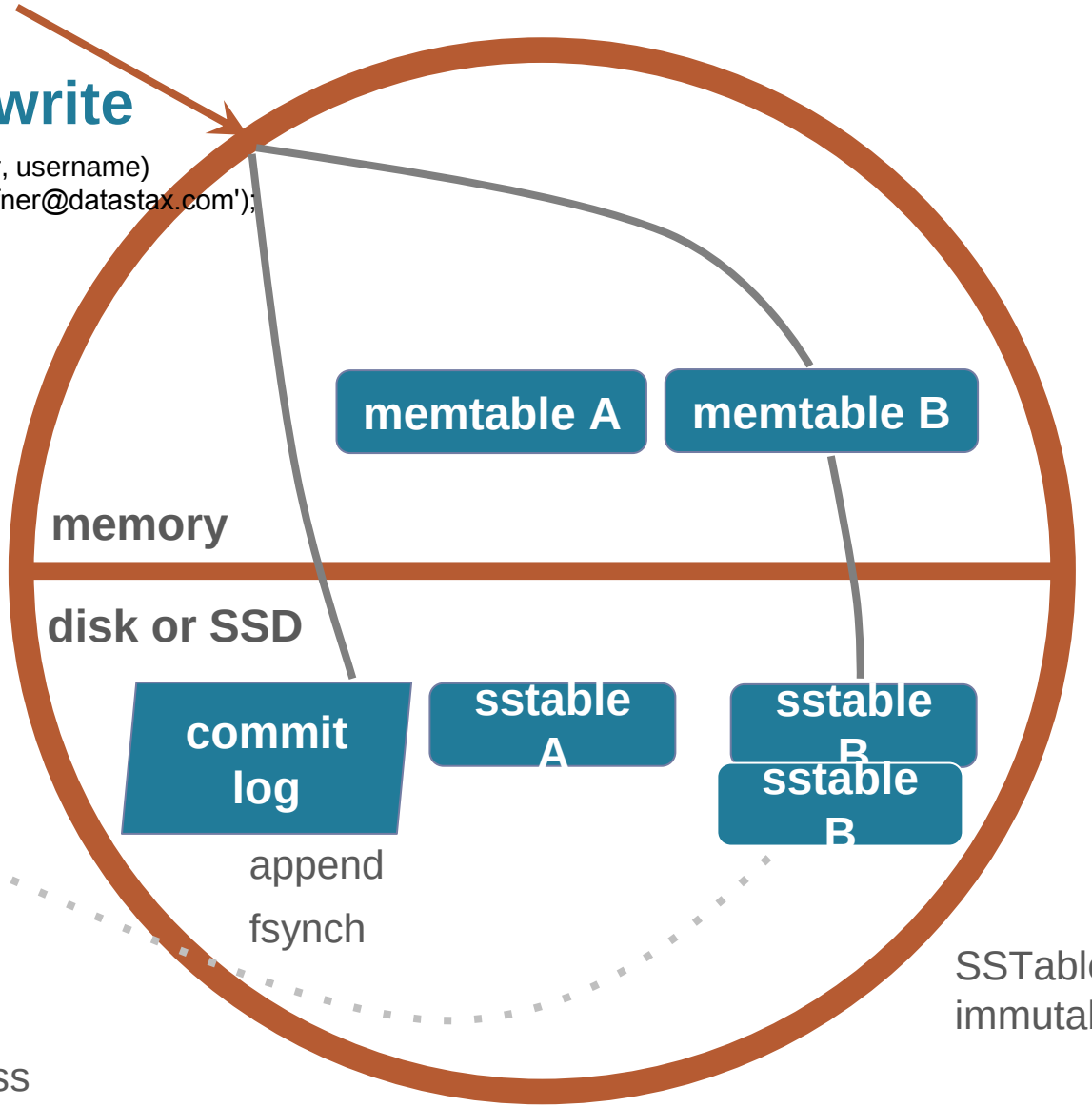
INSERT INTO email_users (domain, user, username)
VALUES ('@datastax.com','rreffner', 'rreffner@datastax.com');

Compaction

- Merge sstables
- Evict tombstones
- Rebuild indexes
- Is a background process

SSTables are immutable

Write Path



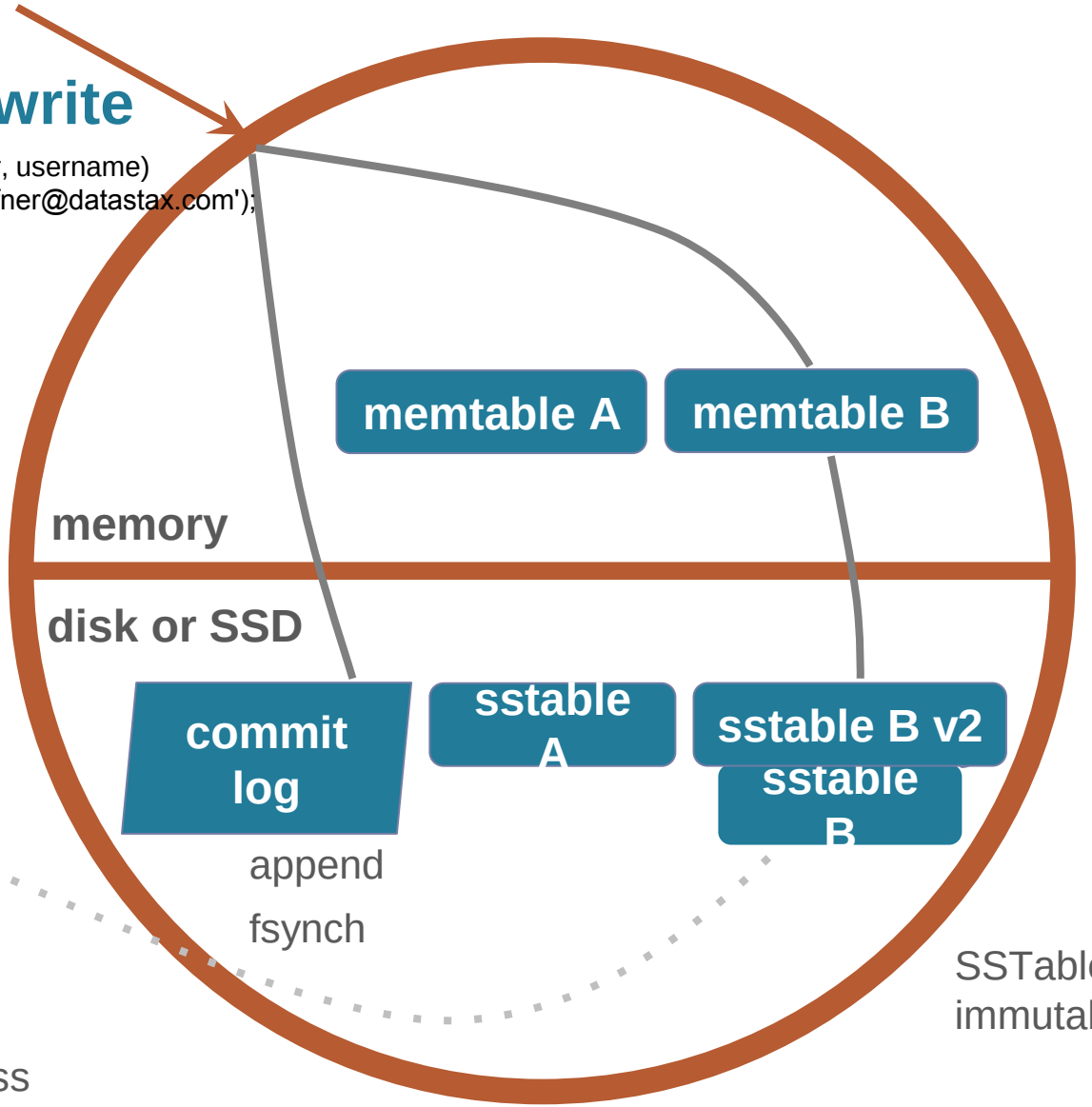
INSERT INTO email_users (domain, user, username)
VALUES ('@datastax.com','rreffner', 'rreffner@datastax.com');

Compaction

- Merge sstables
- Evict tombstones
- Rebuild indexes
- Is a background process

SSTables are immutable

Write Path



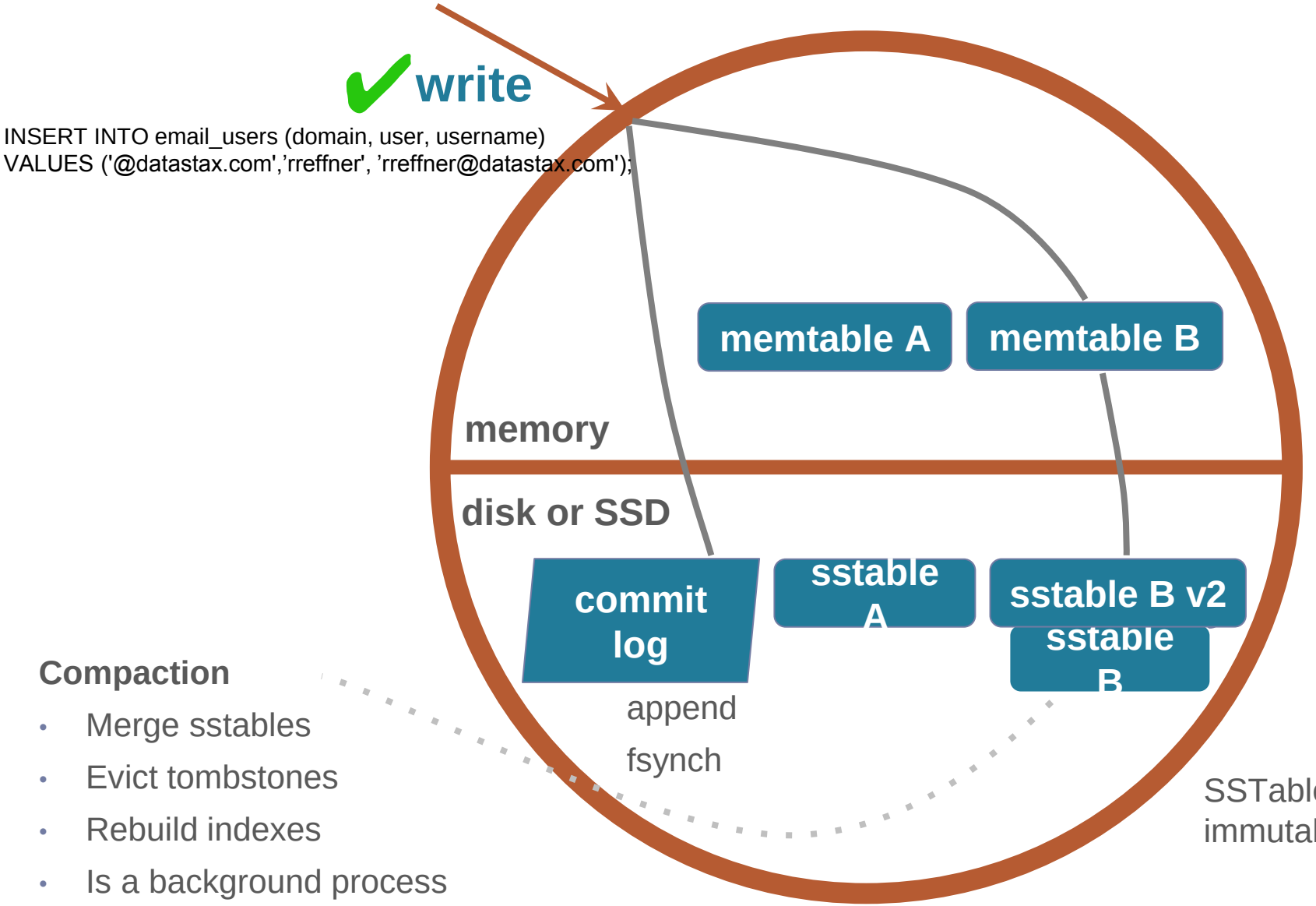
INSERT INTO email_users (domain, user, username)
VALUES ('@datastax.com','rreffner', 'rreffner@datastax.com');

Compaction

- Merge sstables
- Evict tombstones
- Rebuild indexes
- Is a background process

SSTables are immutable

Write Path



Compaction

- Merge sstables
- Evict tombstones
- Rebuild indexes
- Is a background process

Memtable Flush During Write Load

DATASTAX

ID	NAME	DOB
AB1	John Smith	10/11/1972
AB2	Bob Jones	3/1/1964

memtable

memory

disk or SSD

write 1
write 2

commit log

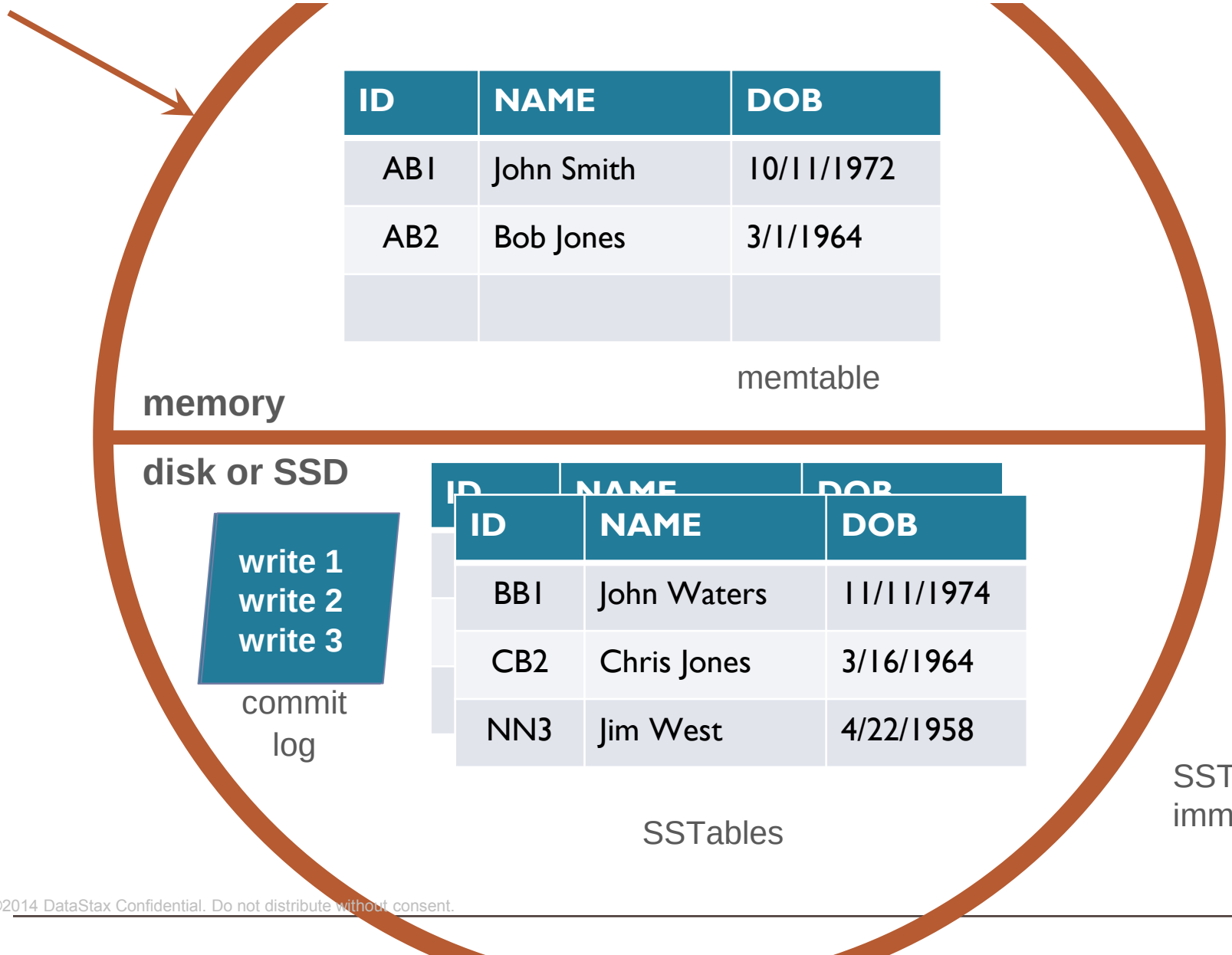
ID	NAME	DOB
BB1	John Waters	11/11/1974
CB2	Chris Jones	3/16/1964
NN3	Jim West	4/22/1958

SSTables

SSTables are immutable

©2014 DataStax Confidential. Do not distribute without consent.

Memtable Flush During Write Load



©2014 DataStax Confidential. Do not distribute without consent.

Memtable Flush During Write Load

memory

ID	NAME	DOB
AB1	John Smith	10/11/1972
AB2	Bob Jones	3/1/1964
ZZ3	Mike West	4/22/1968

memtable

disk or SSD

commit log

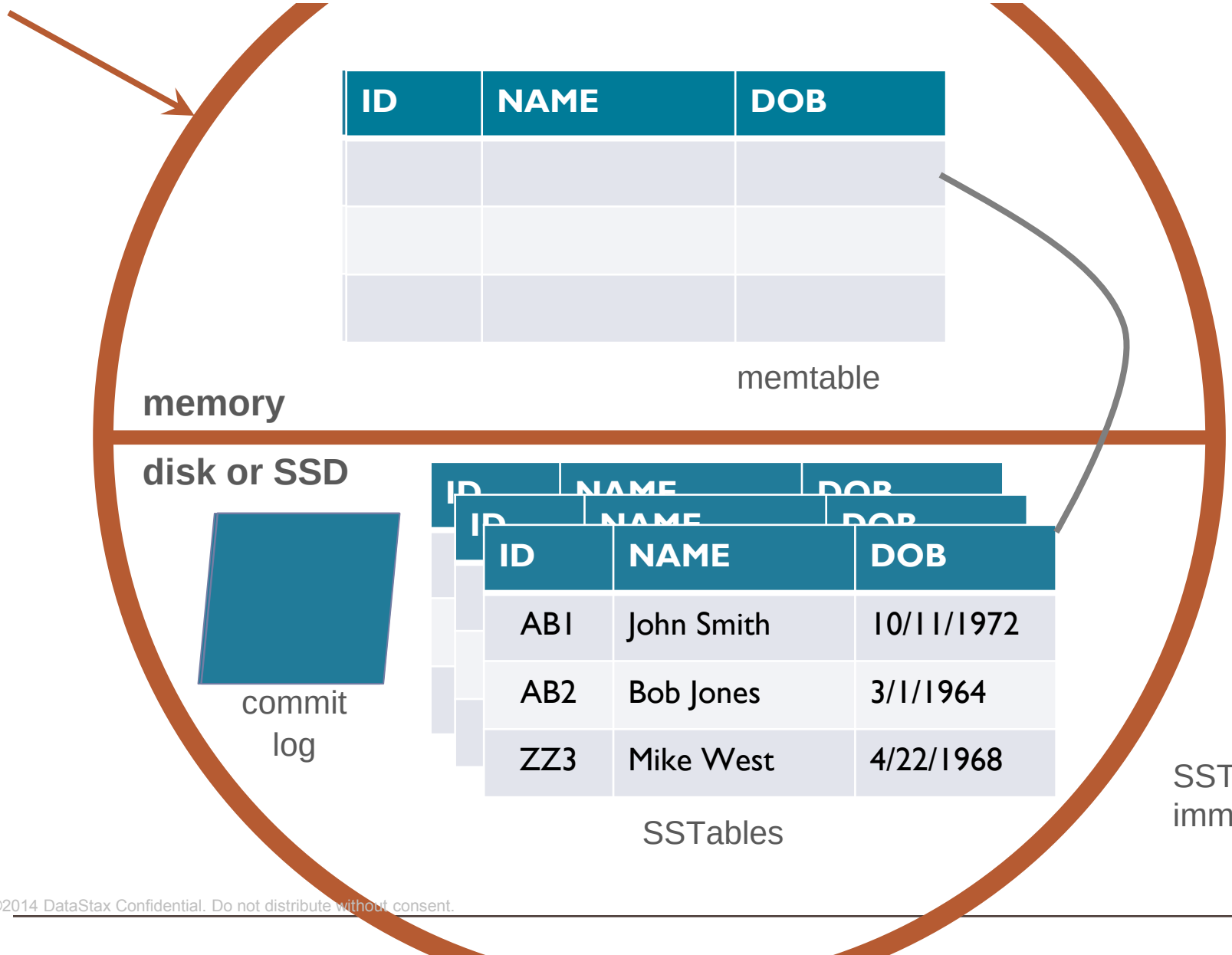
ID	NAME	DOB
BB1	John Waters	11/11/1974
CB2	Chris Jones	3/16/1964
NN3	Jim West	4/22/1958

SSTables

SSTables are immutable

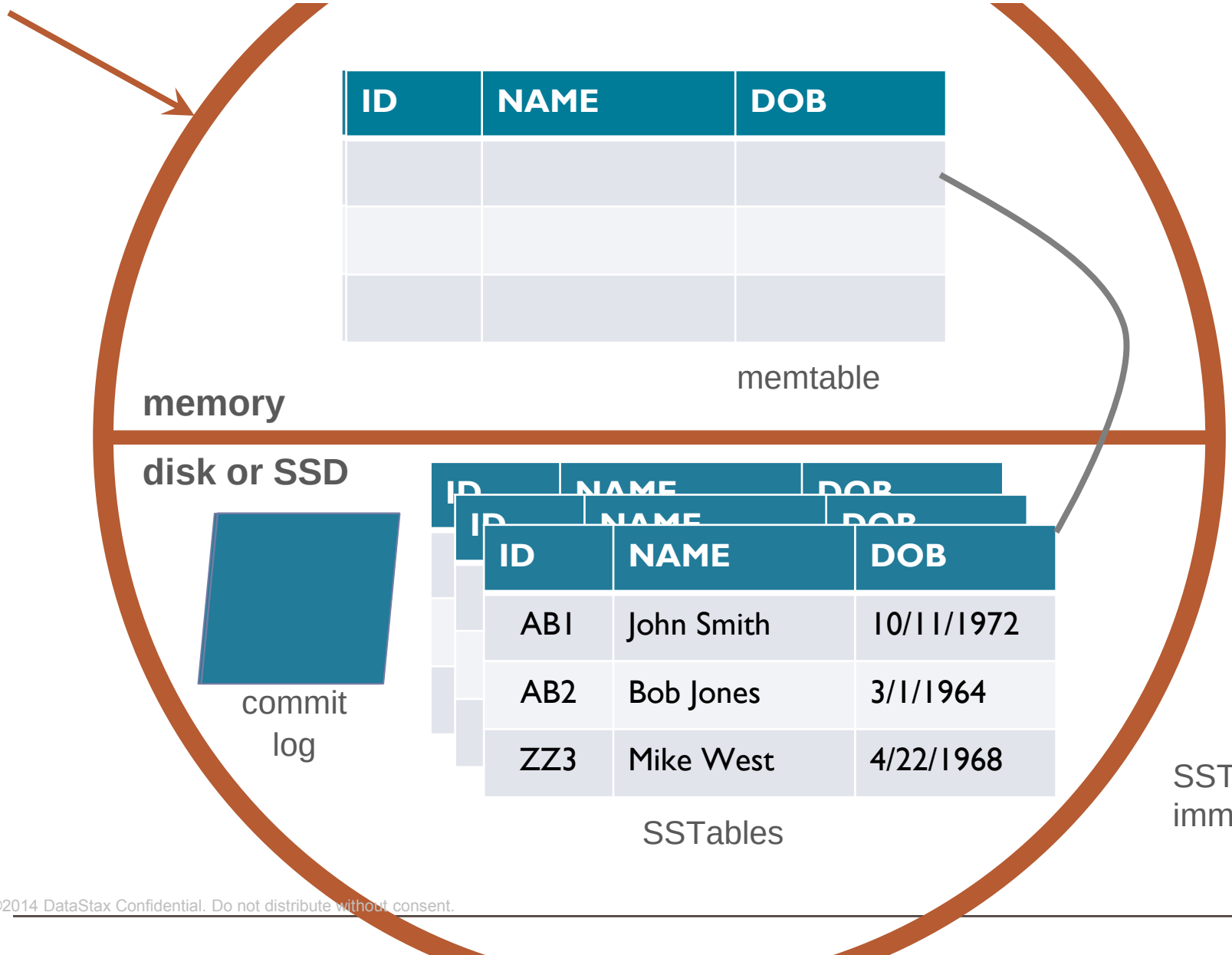
©2014 DataStax Confidential. Do not distribute without consent.

Memtable Flush During Write Load



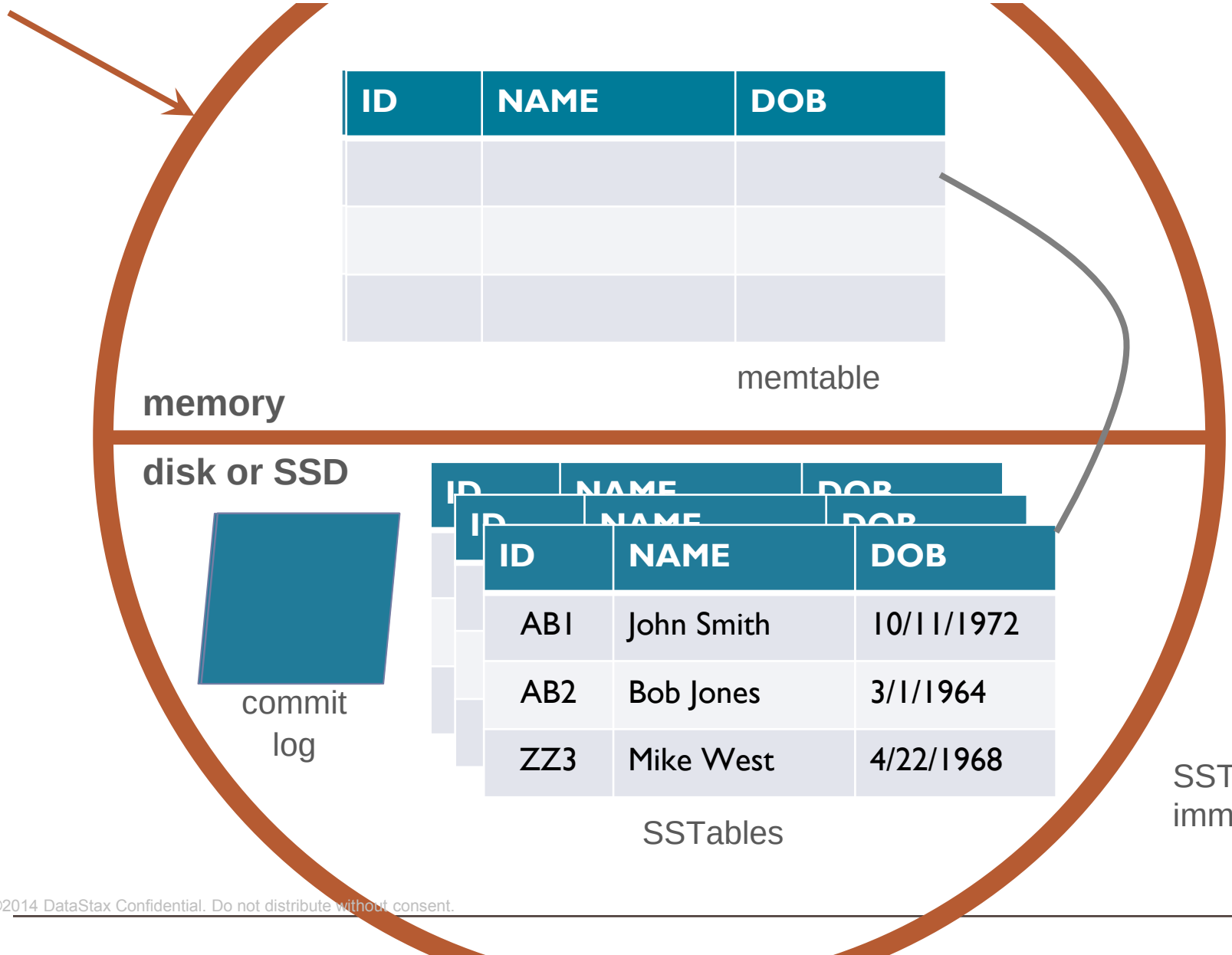
©2014 DataStax Confidential. Do not distribute without consent.

Memtable Flush During Write Load



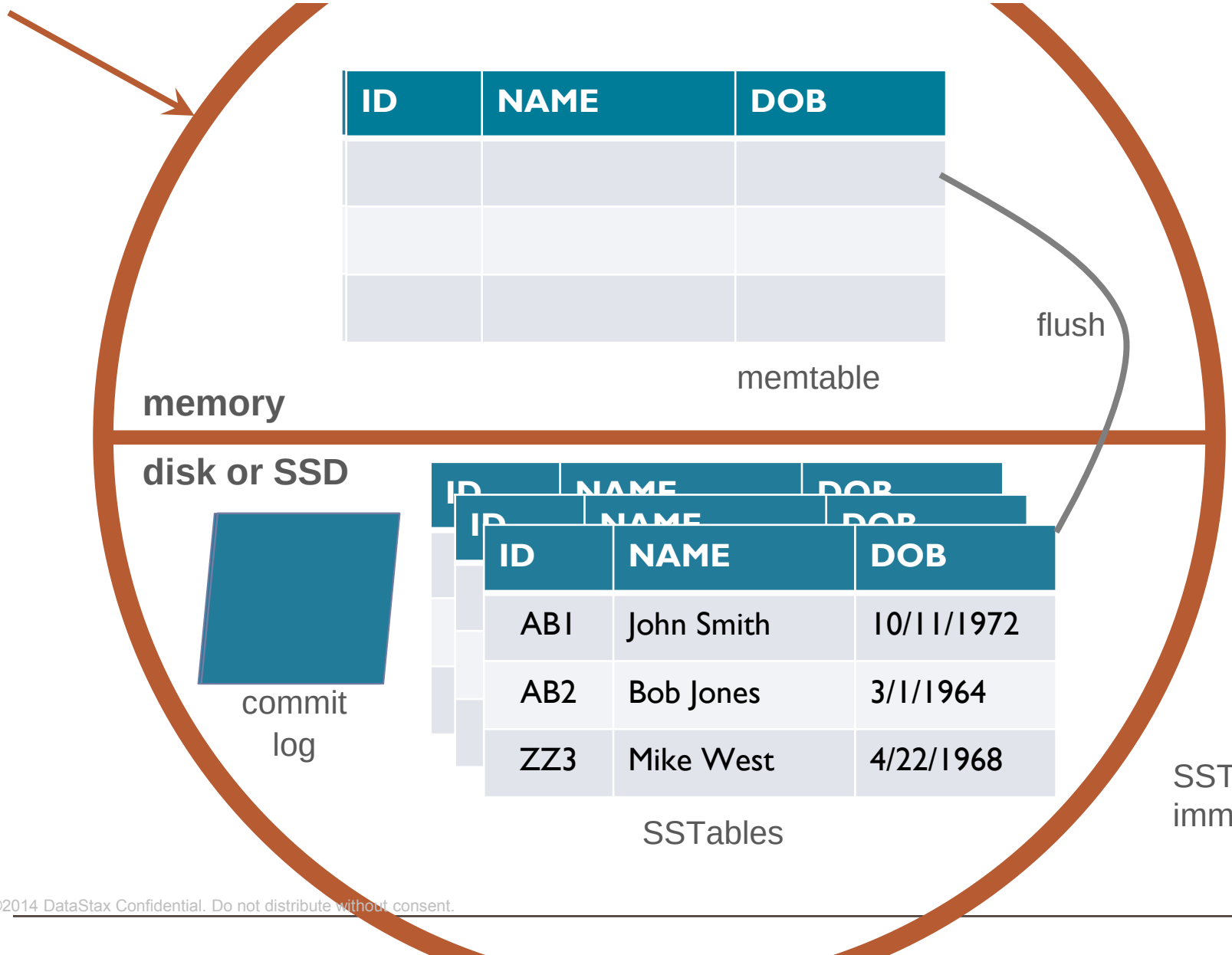
©2014 DataStax Confidential. Do not distribute without consent.

Memtable Flush During Write Load



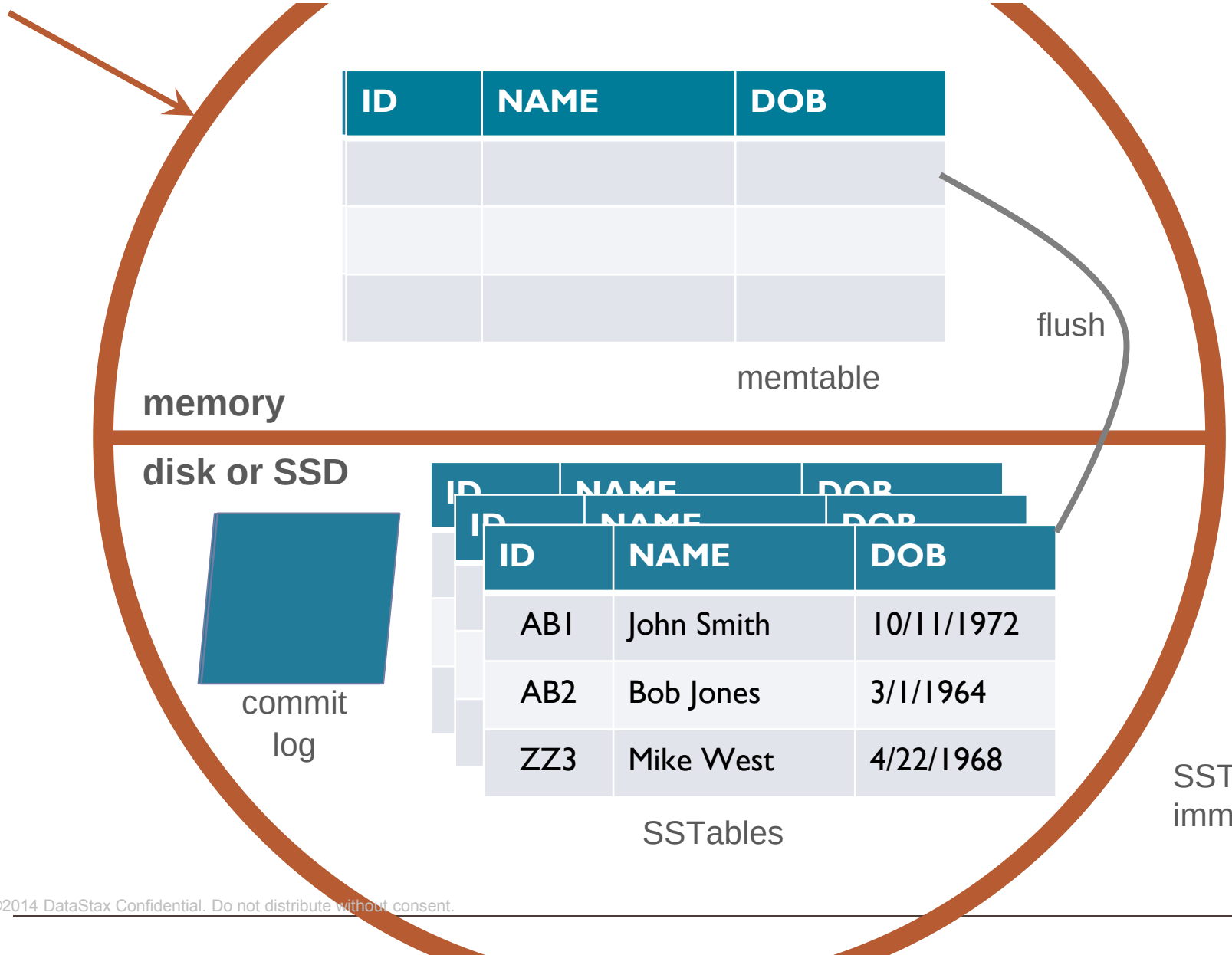
©2014 DataStax Confidential. Do not distribute without consent.

Memtable Flush During Write Load



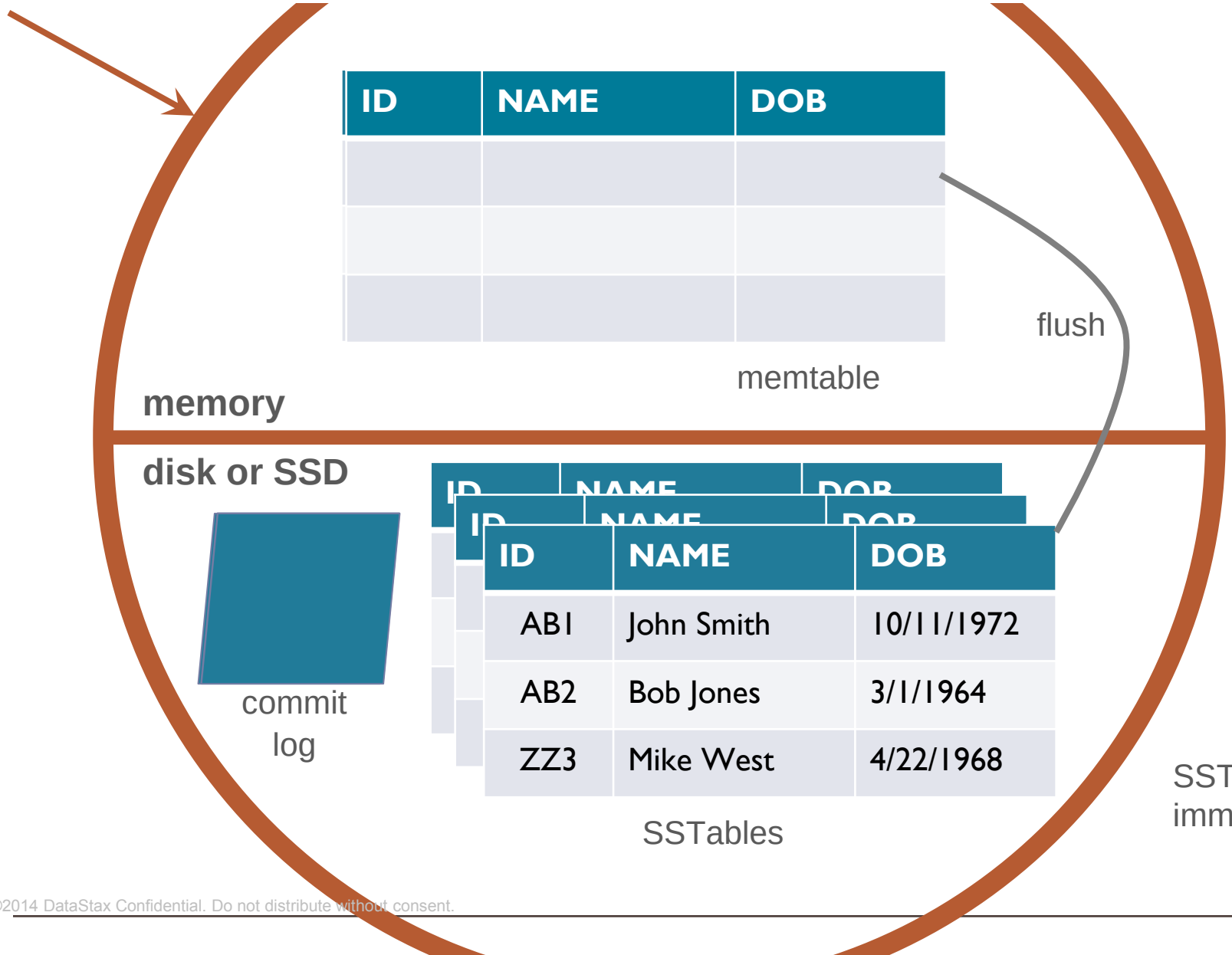
©2014 DataStax Confidential. Do not distribute without consent.

Memtable Flush During Write Load



©2014 DataStax Confidential. Do not distribute without consent.

Memtable Flush During Write Load



©2014 DataStax Confidential. Do not distribute without consent.

Compaction During Write Load

memory

disk or SSD

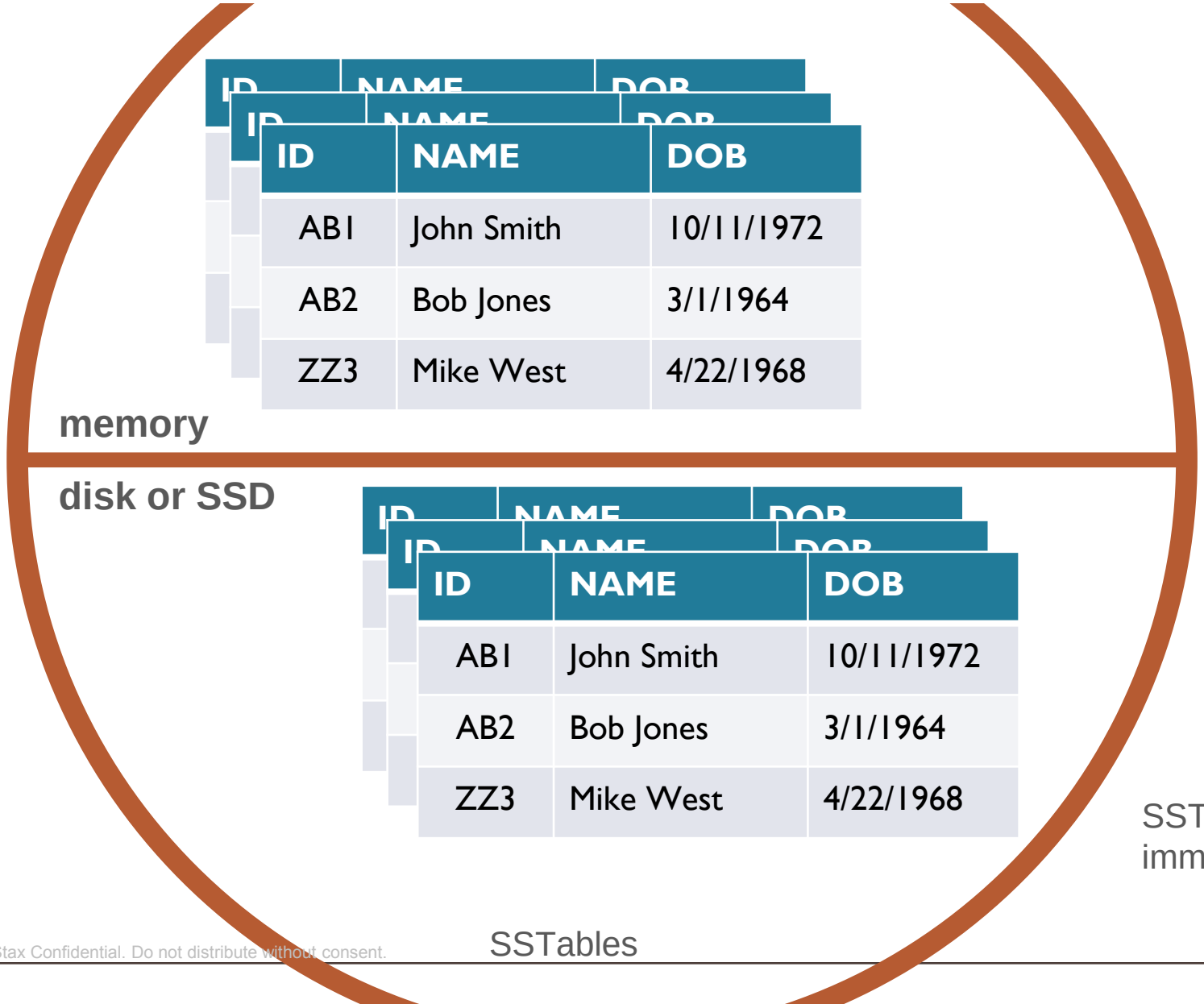
ID	NAME	DOB
AB1	John Smith	10/11/1972
AB2	Bob Jones	3/1/1964
ZZ3	Mike West	4/22/1968

SSTables are immutable

SSTables

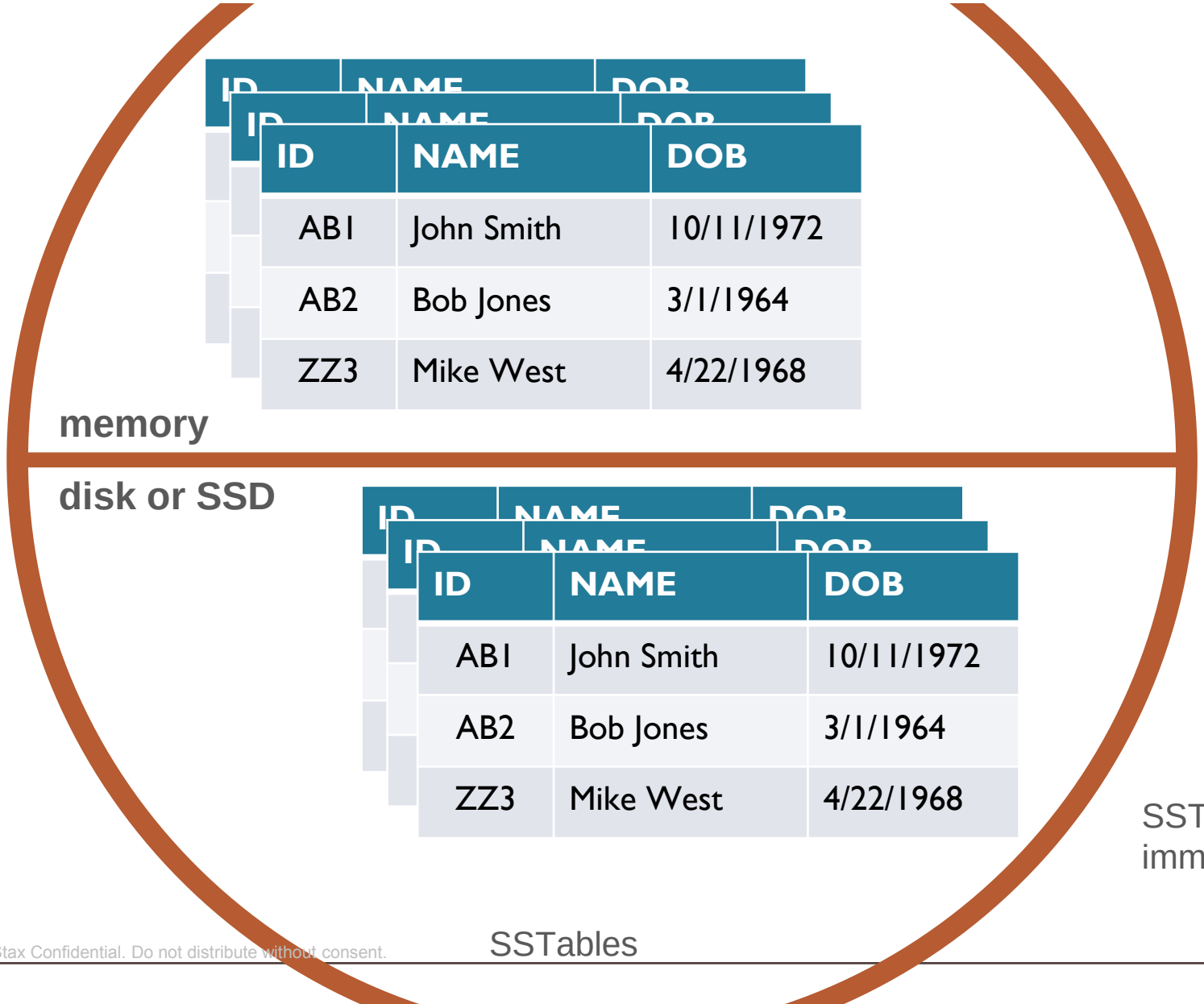
©2014 DataStax Confidential. Do not distribute without consent.

Compaction During Write Load



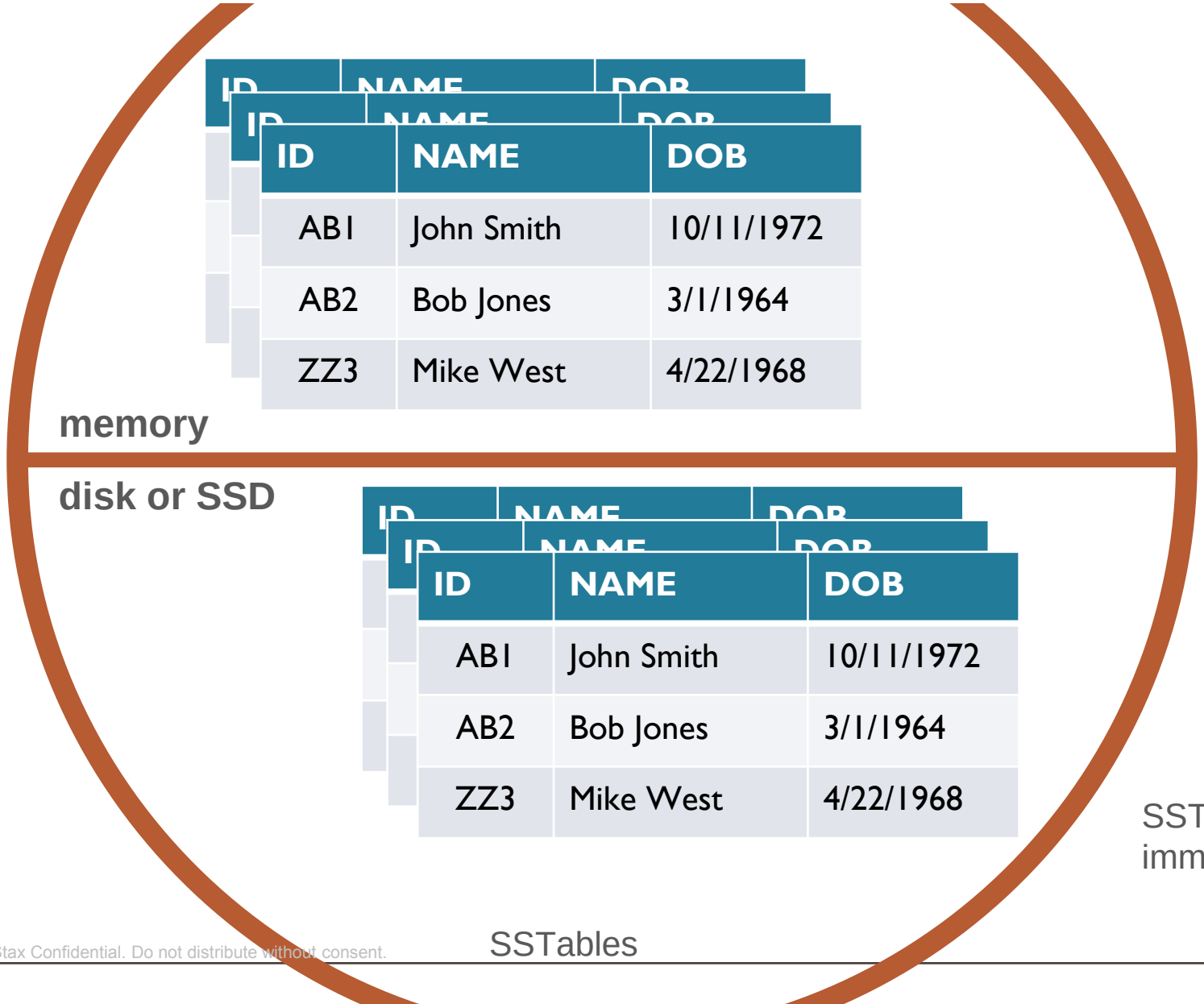
©2014 DataStax Confidential. Do not distribute without consent.

Compaction During Write Load



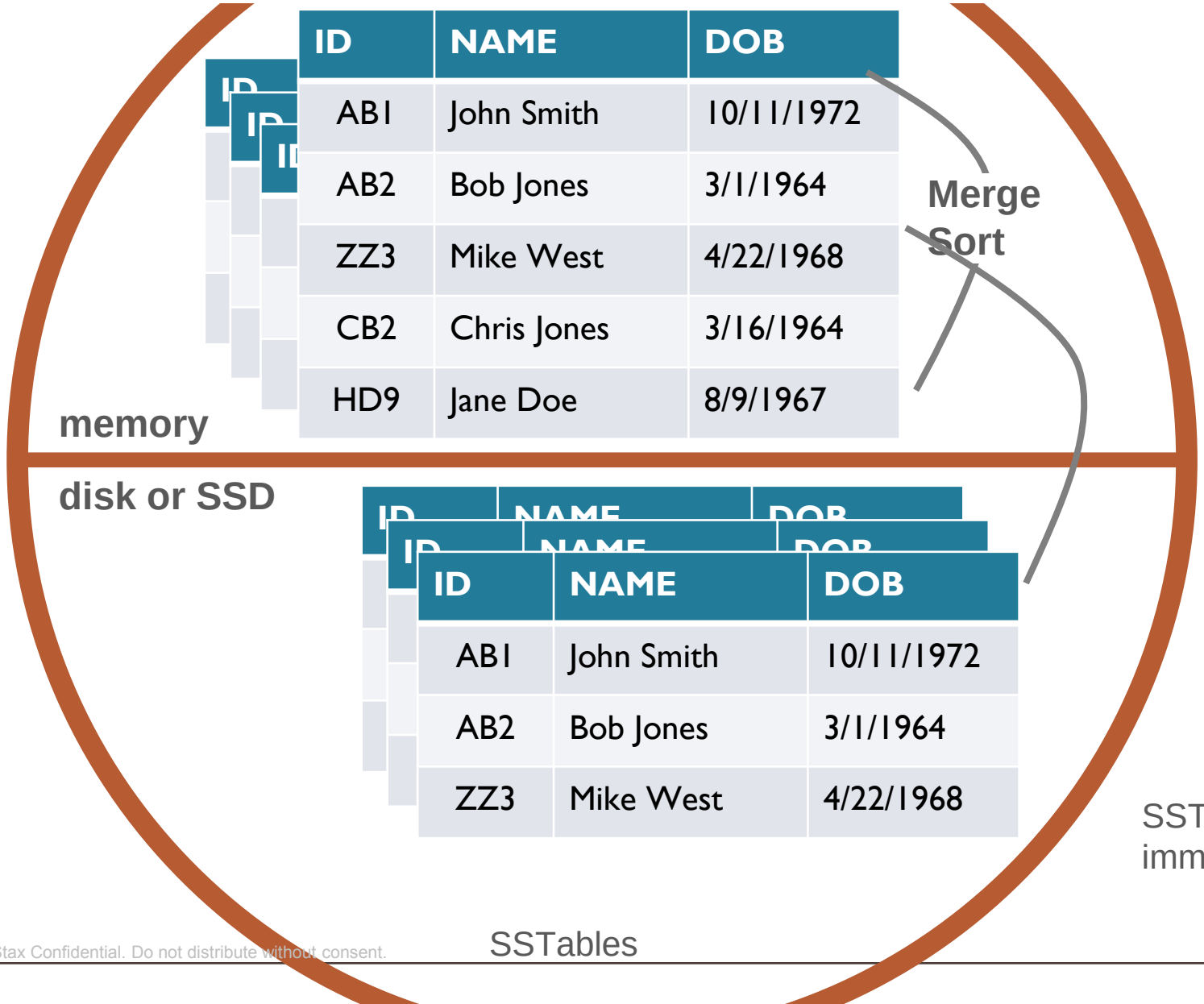
©2014 DataStax Confidential. Do not distribute without consent.

Compaction During Write Load



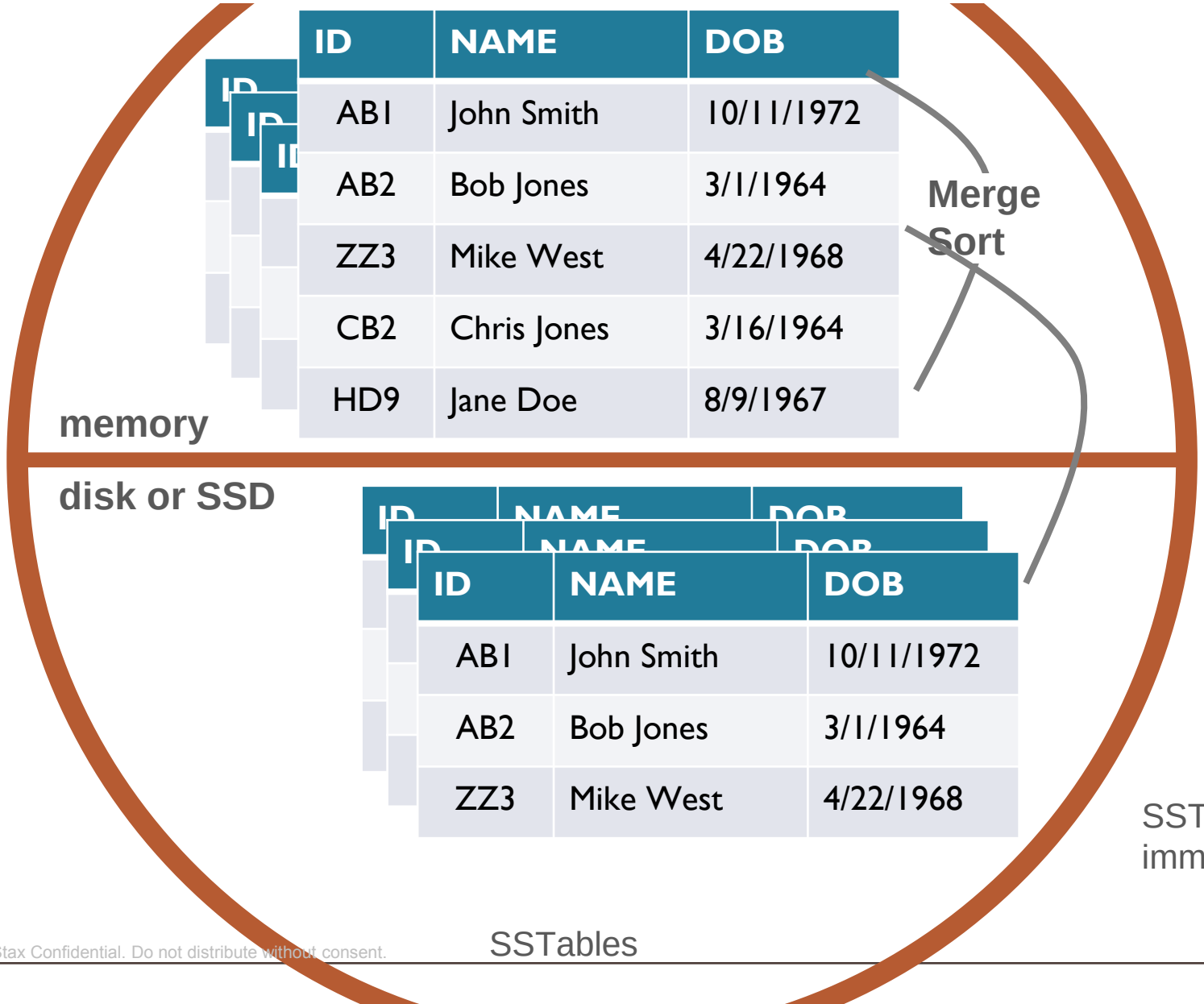
©2014 DataStax Confidential. Do not distribute without consent.

Compaction During Write Load



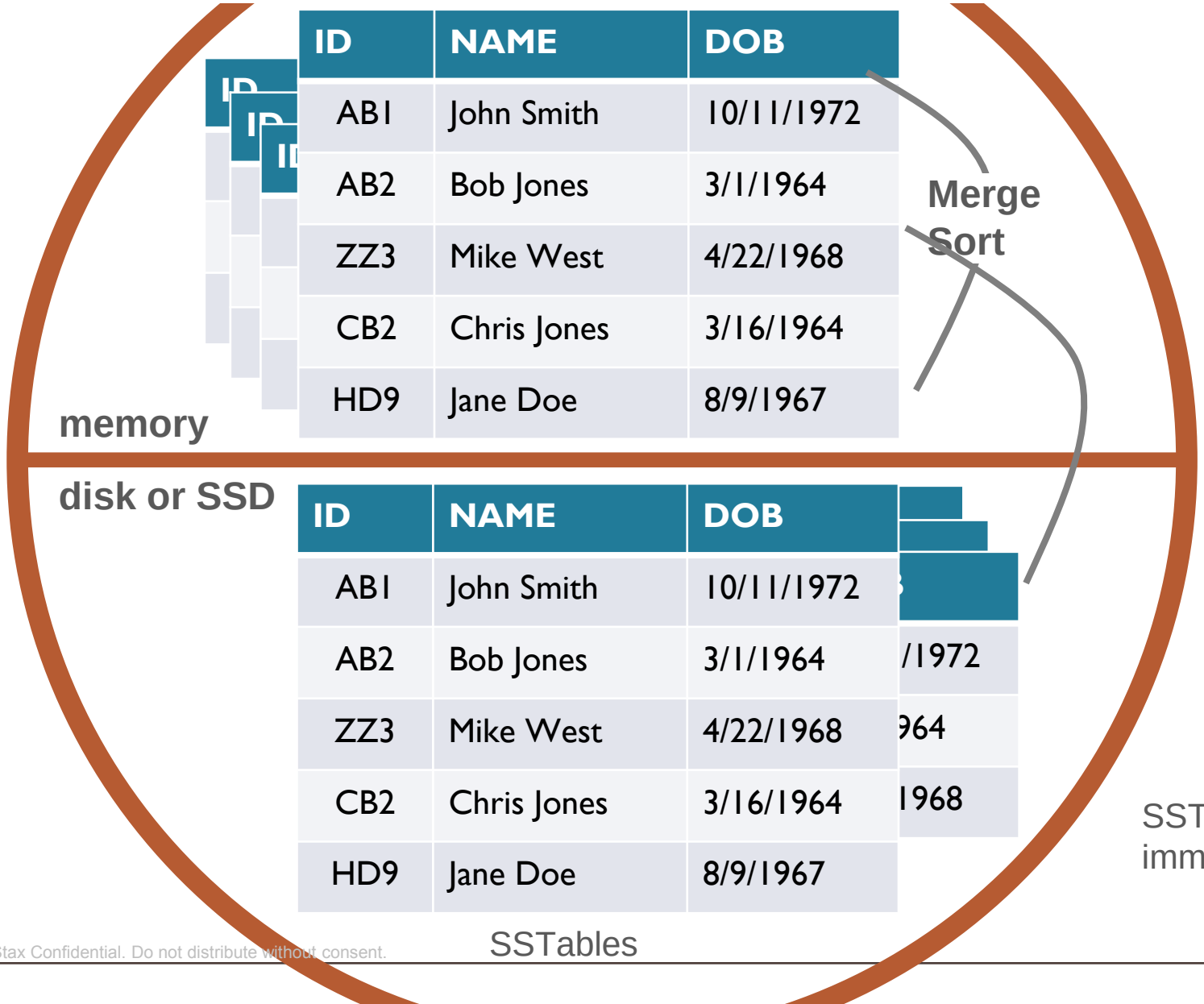
©2014 DataStax Confidential. Do not distribute without consent.

Compaction During Write Load



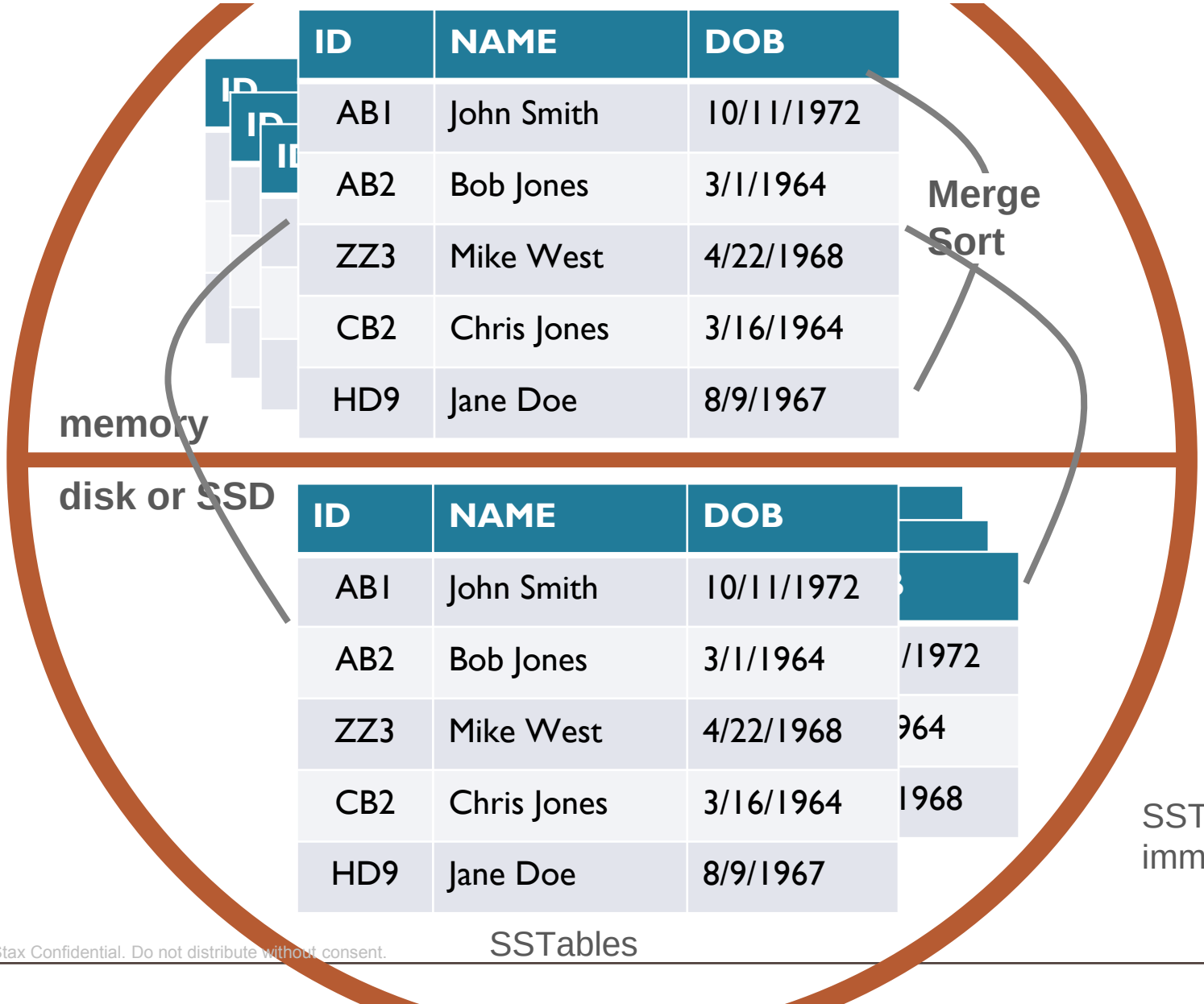
©2014 DataStax Confidential. Do not distribute without consent.

Compaction During Write Load



©2014 DataStax Confidential. Do not distribute without consent.

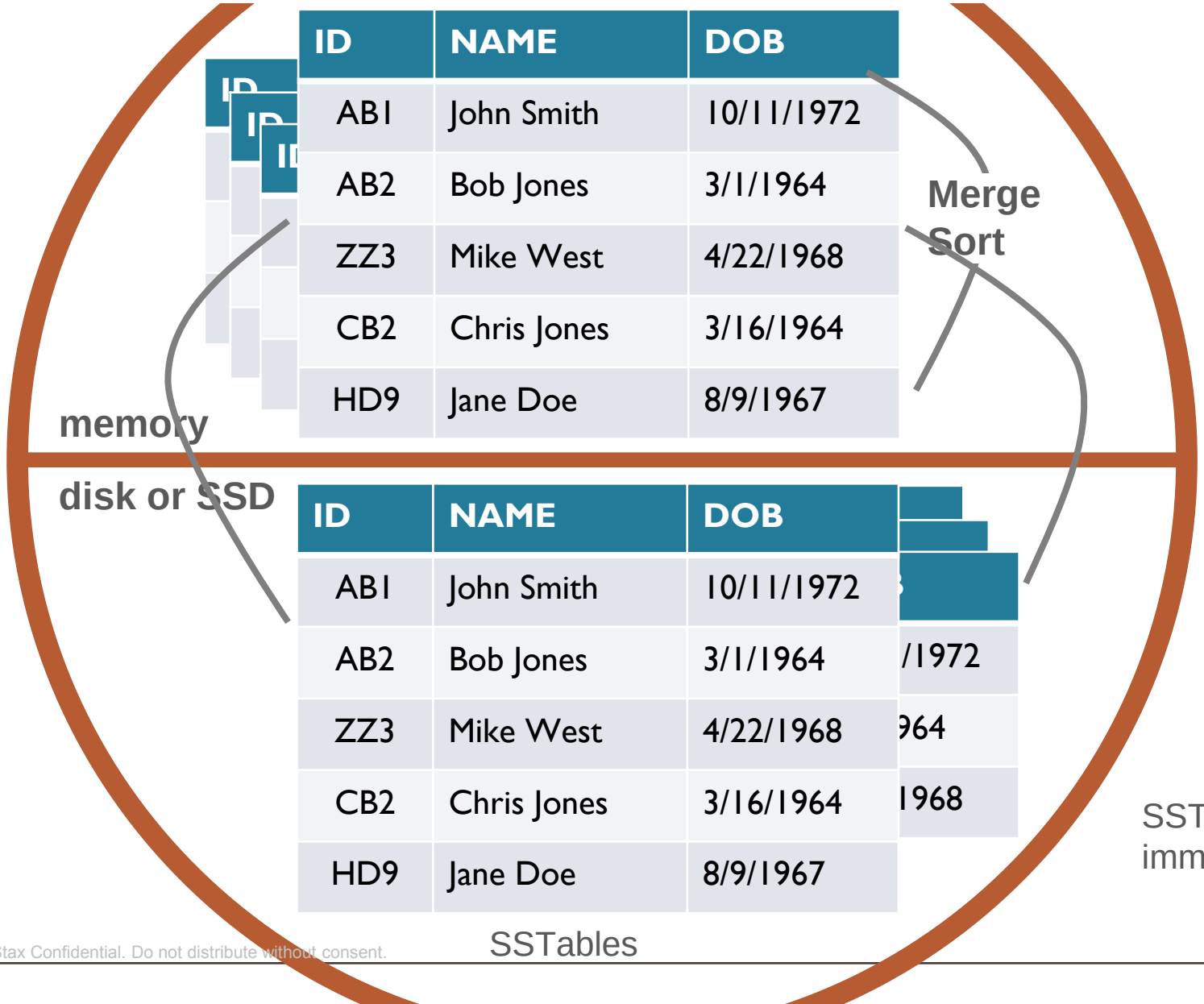
Compaction During Write Load



SSTables are immutable

©2014 DataStax Confidential. Do not distribute without consent.

Compaction During Write Load



SSTables are immutable

©2014 DataStax Confidential. Do not distribute without consent.

Compaction And Tombstones

ID	NAME	DOB
BB1	John Waters	
CB2		4/20/1964
NN3	Jim West	4/22/1958

ID	NAME	DOB
BB1		11/11/1974
CB2	Chris Jones	~~3/16/1964~~
~~NN3~~	~~Jim West~~	~~4/22/1958~~

ID	NAME	DOB
AB1	John Smith	10/11/1972
AB2	Bob Jones	
ZZ3	Mike West	4/22/1968

Note: This is a logical representation, not the SSTable physical file format.

SSTables are immutable

©2014

Compaction And Tombstones

ID	NAME	DOB
BB1	John Waters	
CB2		4/20/1964
NN3	Jim West	4/22/1958

ID	NAME	DOB
BB1		11/11/1974
CB2	Chris Jones	~~3/16/1964~~
~~NN3~~	~~Jim West~~	~~4/22/1958~~

ID	NAME	DOB
AB1	John Smith	10/11/1972
AB2	Bob Jones	
ZZ3	Mike West	4/22/1968

Note: This is a logical representation, not the SSTable physical file format.

ID	NAME	DOB
AB1	John Smith	10/11/1972
AB2	Bob Jones	
ZZ3	Mike West	4/22/1968
CB2	Chris Jones	
BB1	John Waters	11/11/1974

SSTables are immutable

©2014

Compaction And Tombstones

ID	NAME	DOB
BB1	John Waters	
CB2		4/20/1964
NN3	Jim West	4/22/1958

ID	NAME	DOB
BB1		11/11/1974
CB2	Chris Jones	~~3/16/1964~~
~~NN3~~	~~Jim West~~	~~4/22/1958~~

ID	NAME	DOB
AB1	John Smith	10/11/1972
AB2	Bob Jones	
ZZ3	Mike West	4/22/1968

Note: This is a logical representation, not the SSTable physical file format.

ID	NAME	DOB
AB1	John Smith	10/11/1972
AB2	Bob Jones	
ZZ3	Mike West	4/22/1968
CB2	Chris Jones	
BB1	John Waters	11/11/1974

SSTables are immutable

©2014

Compaction And Tombstones

ID	NAME	DOB
BB1	John Waters	
CB2		4/20/1964
NN3	Jim West	4/22/1958

ID	NAME	DOB
BB1		11/11/1974
CB2	Chris Jones	~~3/16/1964~~
~~NN3~~	~~Jim West~~	~~4/22/1958~~

ID	NAME	DOB
AB1	John Smith	10/11/1972
AB2	Bob Jones	
ZZ3	Mike West	4/22/1968

Note: This is a logical representation, not the SSTable physical file format.

ID	NAME	DOB
AB1	John Smith	10/11/1972
AB2	Bob Jones	
ZZ3	Mike West	4/22/1968
CB2	Chris Jones	
BB1	John Waters	11/11/1974

SSTables are immutable

©2014

Compaction And Tombstones

ID	NAME	DOB
BB1	John Waters	
CB2		4/20/1964
NN3	Jim West	4/22/1958

ID	NAME	DOB
BB1		11/11/1974
CB2	Chris Jones	~~3/16/1964~~
~~NN3~~	~~Jim West~~	~~4/22/1958~~

ID	NAME	DOB
AB1	John Smith	10/11/1972
AB2	Bob Jones	
ZZ3	Mike West	4/22/1968

Note: This is a logical representation, not the SSTable physical file format.

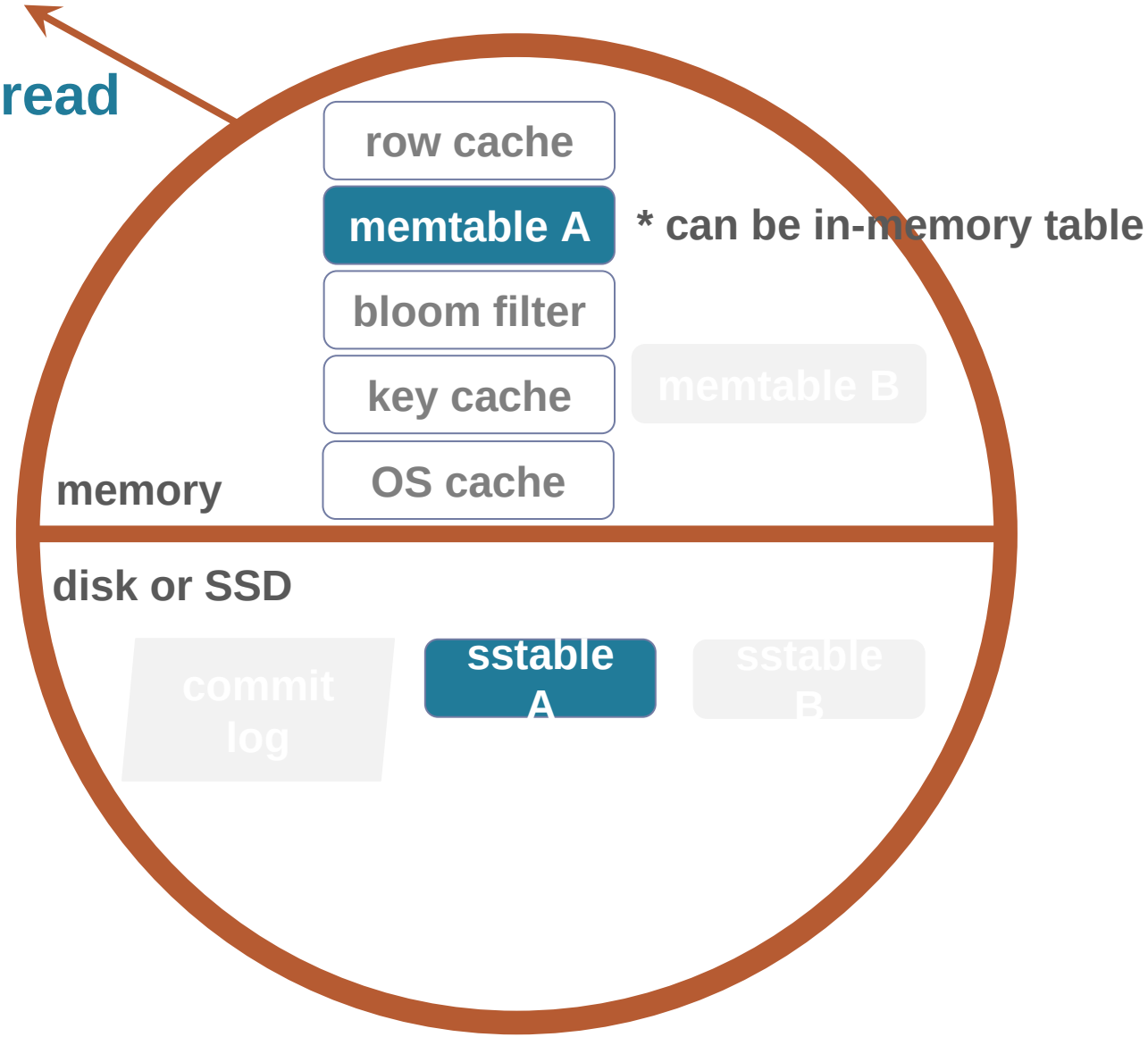
ID	NAME	DOB
AB1	John Smith	10/11/1972
AB2	Bob Jones	
ZZ3	Mike West	4/22/1968
CB2	Chris Jones	
BB1	John Waters	11/11/1974

SSTables are immutable

©2014

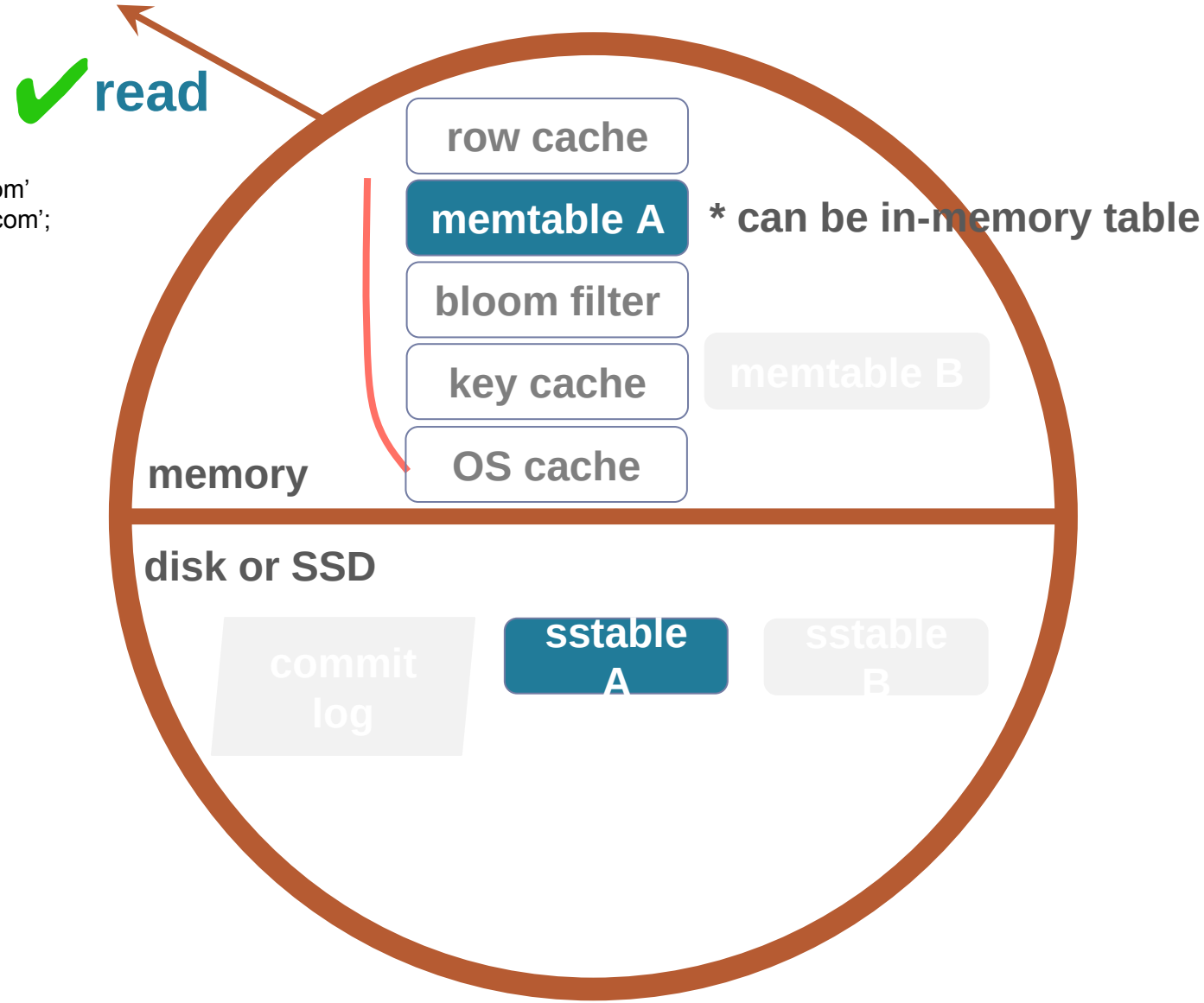
Read Path

```
SELECT * FROM email_users
WHERE domain= '@datastax.com'
AND user = 'rreffner@datastax.com';
```



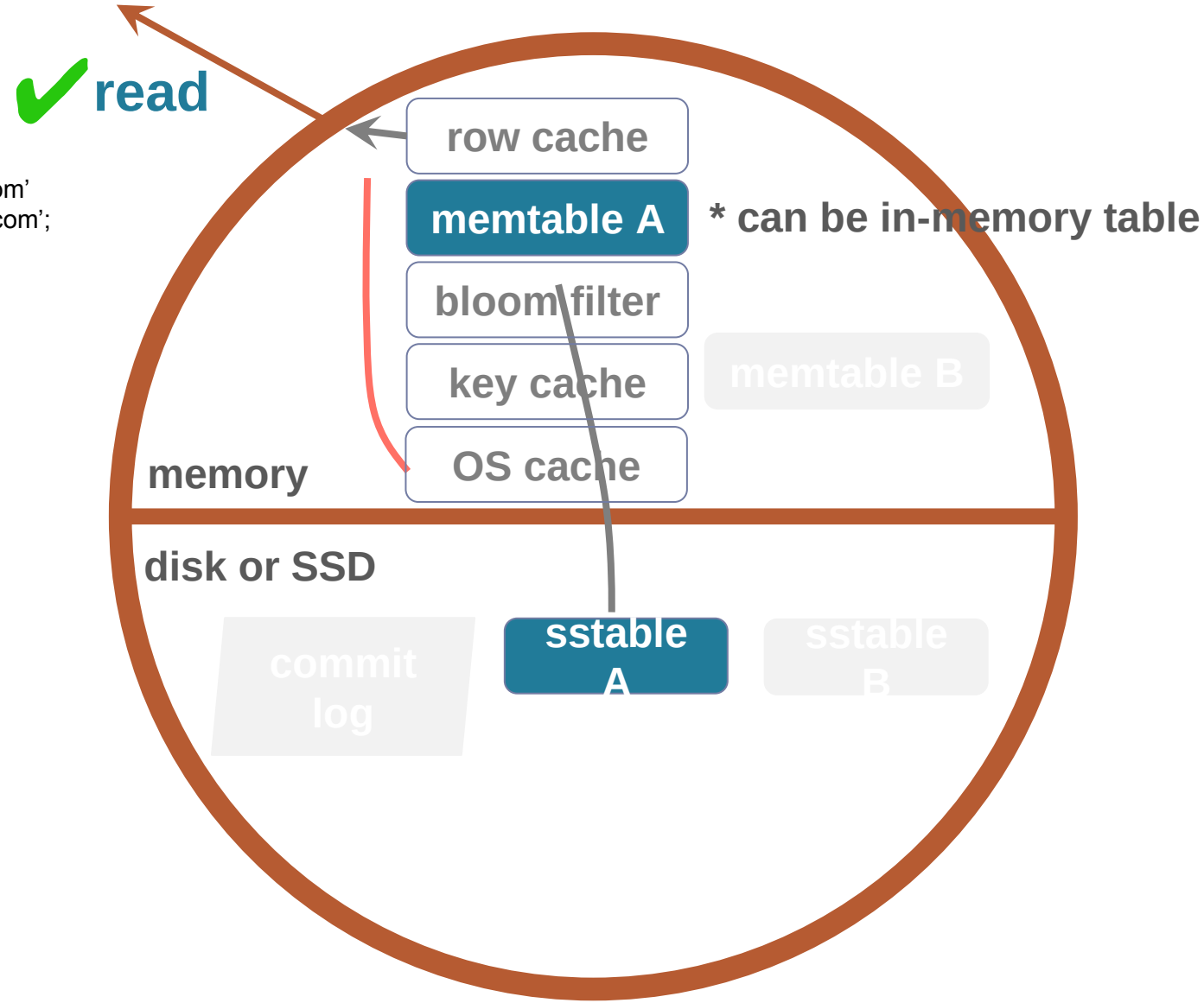
Read Path

SELECT * FROM email_users
WHERE domain= '@datastax.com'
AND user = 'rreffner@datastax.com';

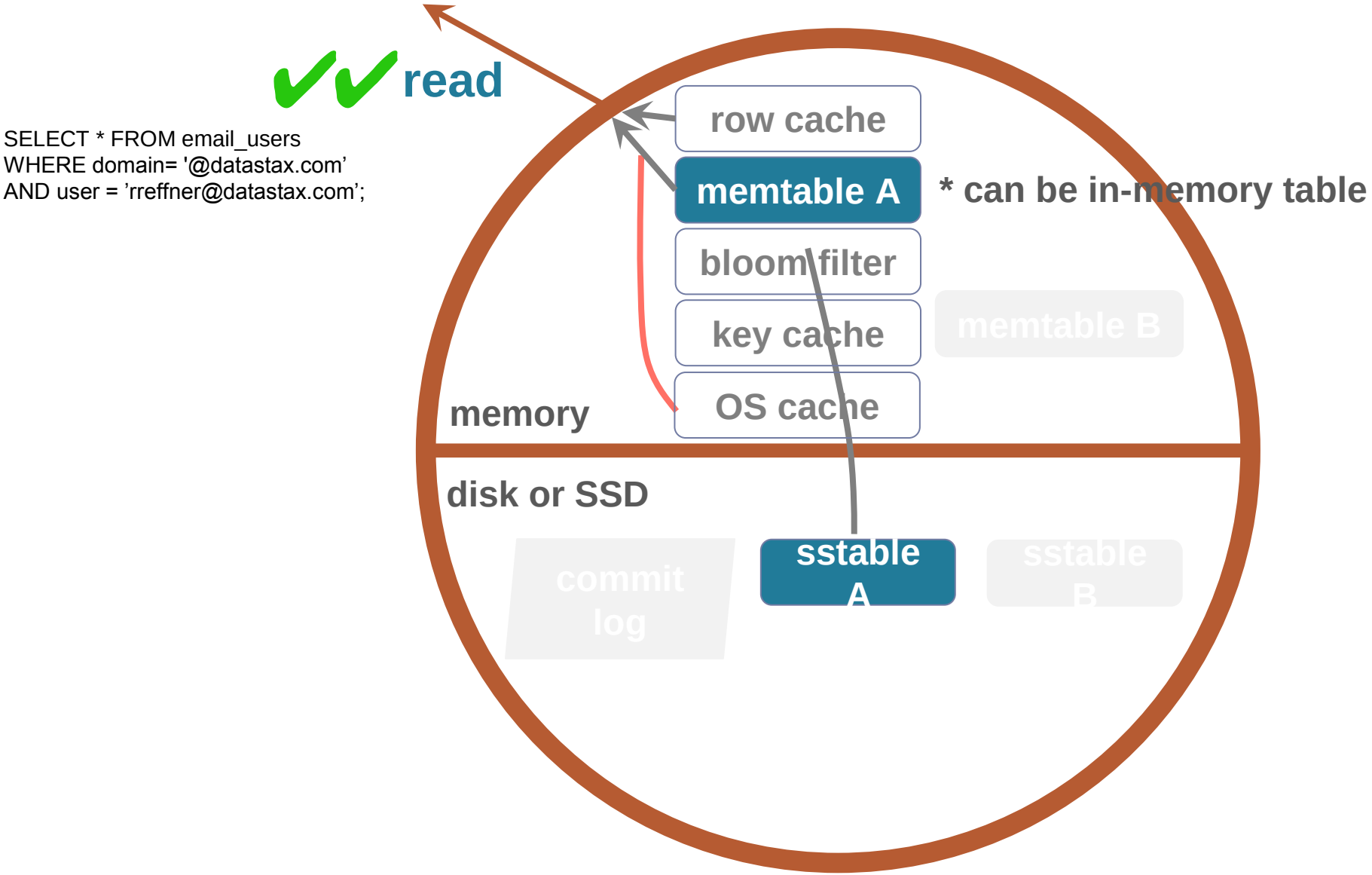


Read Path

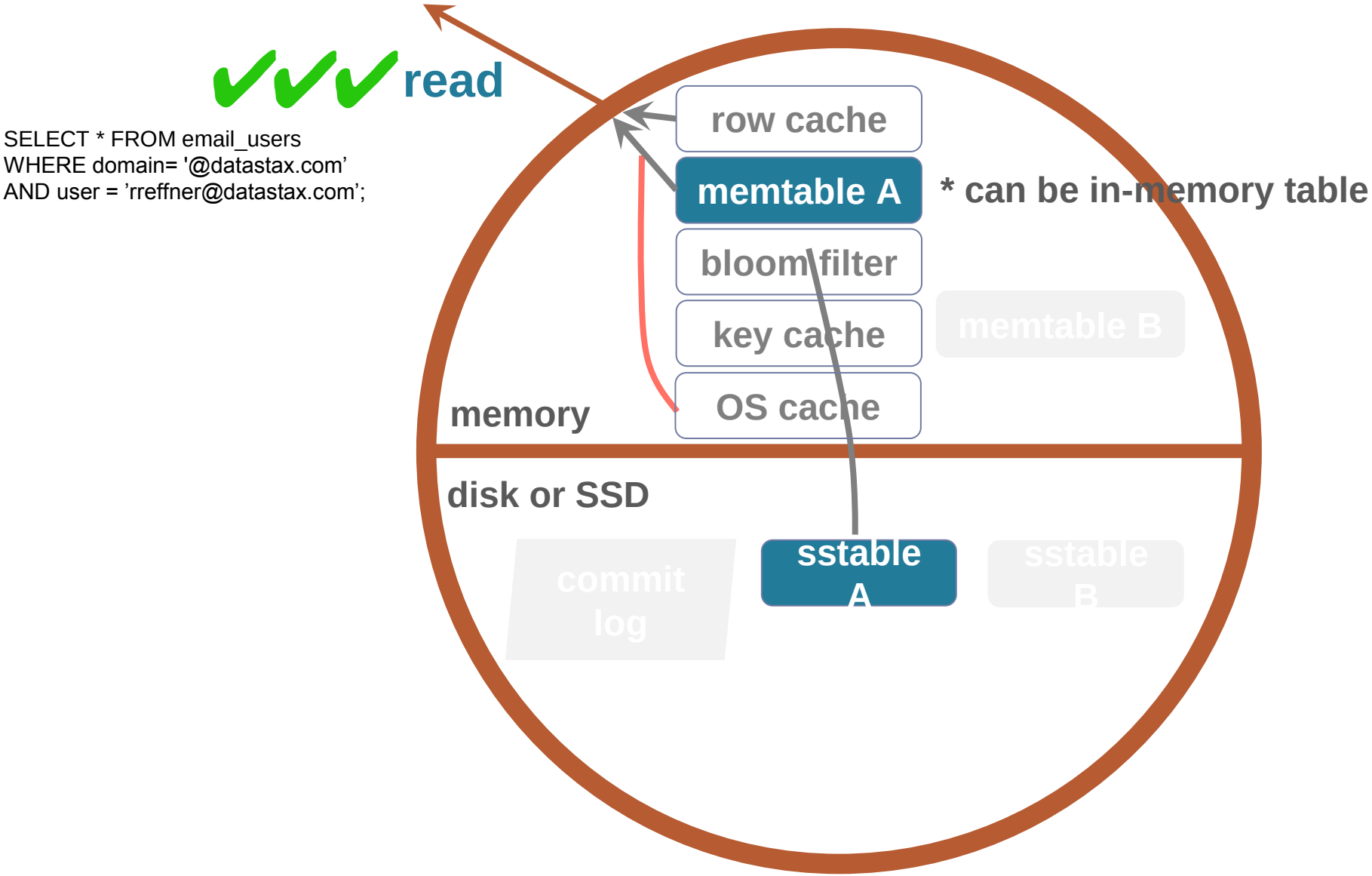
SELECT * FROM email_users
WHERE domain= '@datastax.com'
AND user = 'rreffner@datastax.com';



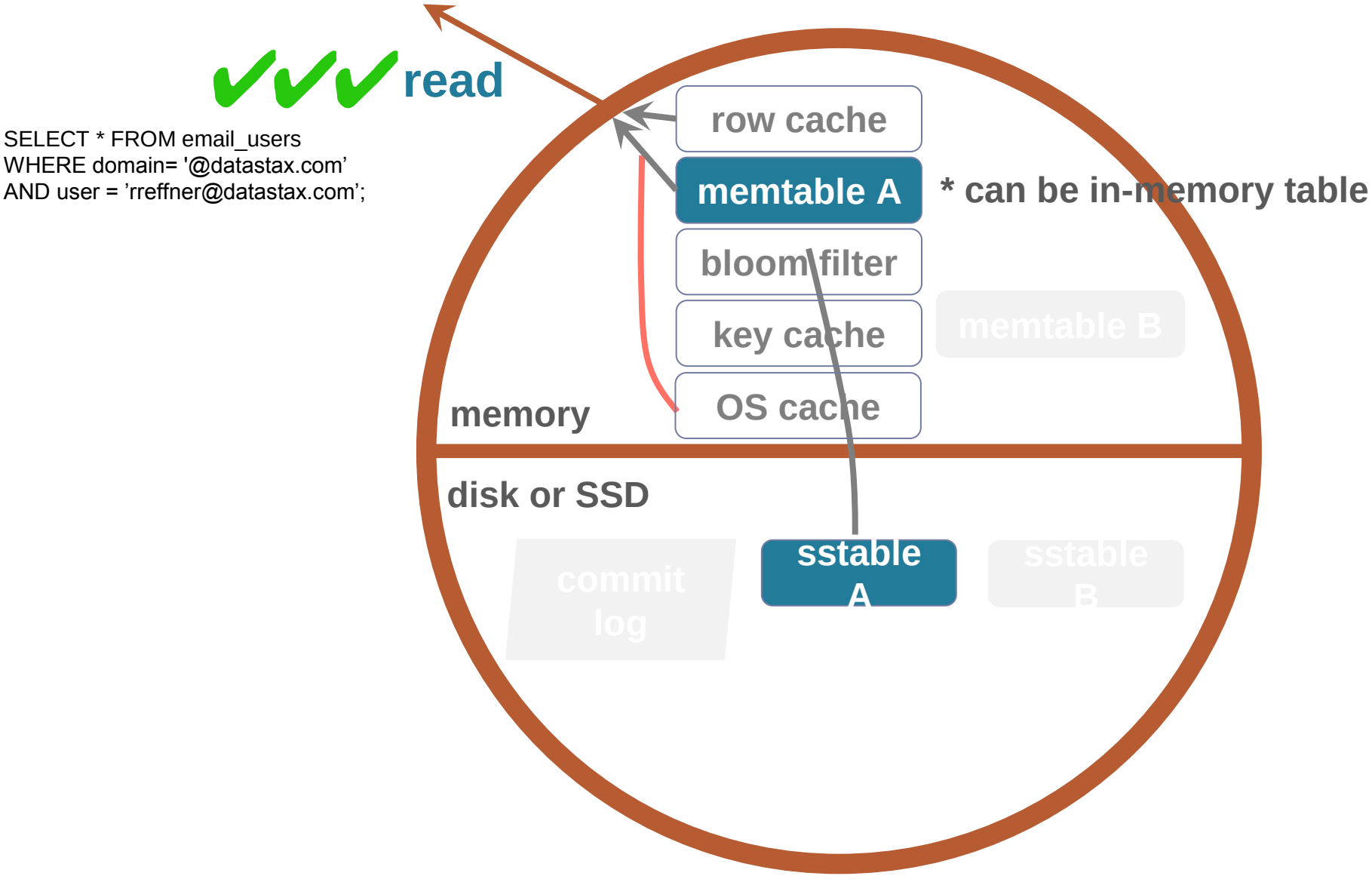
Read Path


read
SELECT * FROM email_users
WHERE domain= '@datastax.com'
AND user = 'rreffner@datastax.com';
row cache
memtable A
* can be in-memory table
bloom filter
key cache
memtable B
OS cache
memory
disk or SSD
commit log
sstable A
sstable B

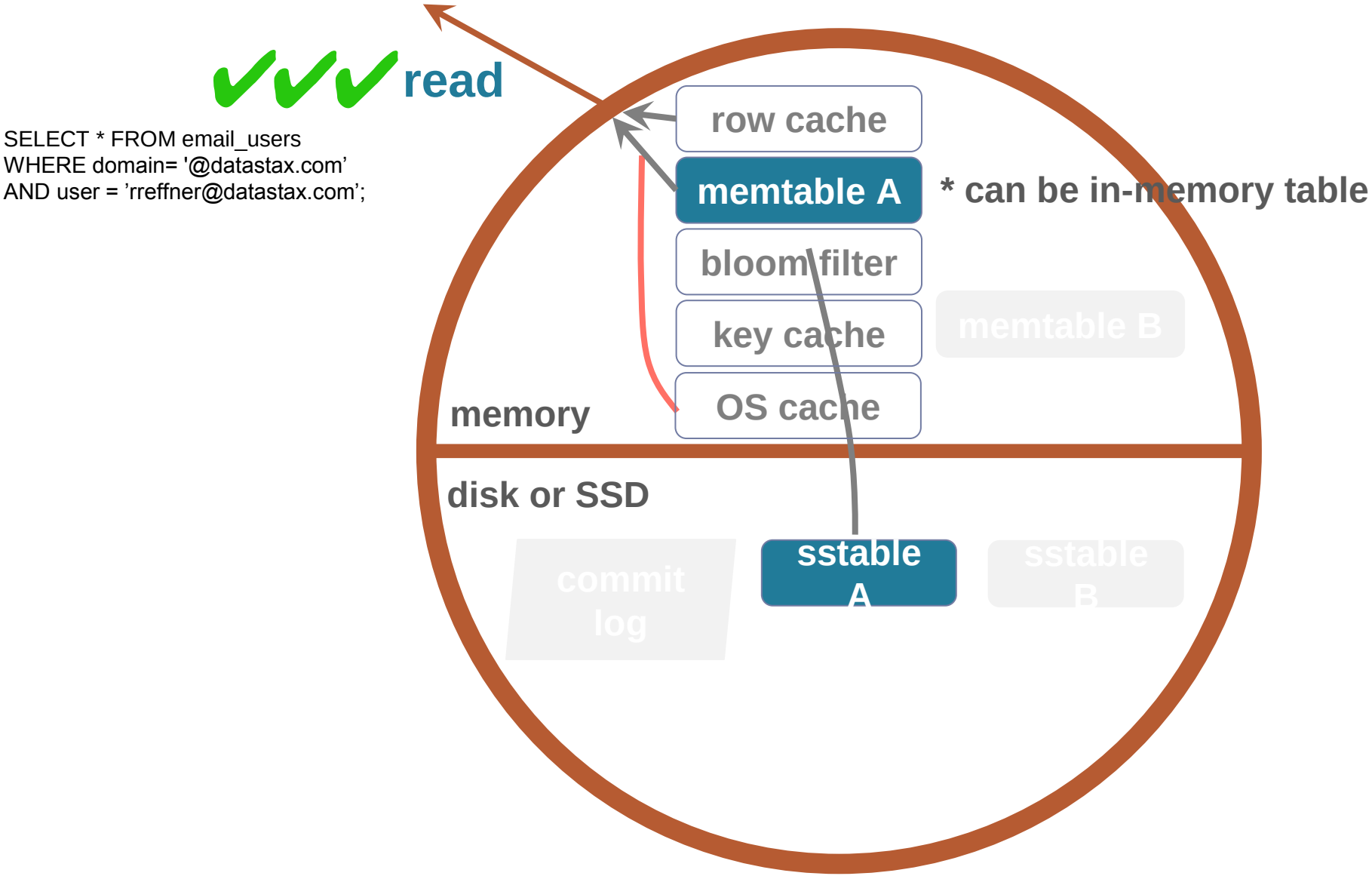
Read Path


read
SELECT * FROM email_users
WHERE domain= '@datastax.com'
AND user = 'rreffner@datastax.com';
row cache
memtable A
* can be in-memory table
bloom filter
key cache
memtable B
OS cache
memory
disk or SSD
commit log
sstable A
sstable B

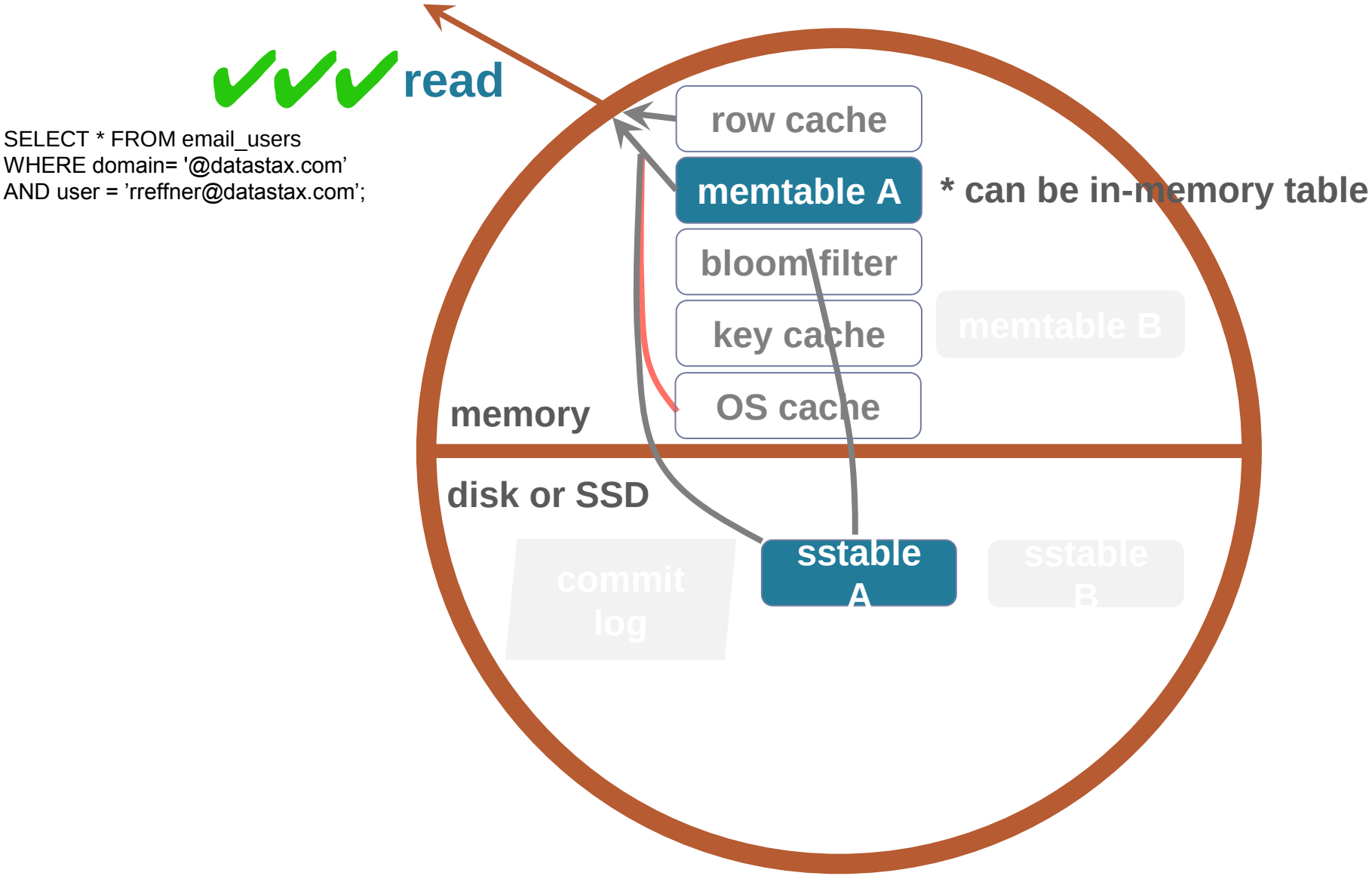
Read Path


read
SELECT * FROM email_users
WHERE domain= '@datastax.com'
AND user = 'rreffner@datastax.com';
row cache
memtable A
* can be in-memory table
bloom filter
key cache
memtable B
OS cache
memory
disk or SSD
commit log
sstable A
sstable B

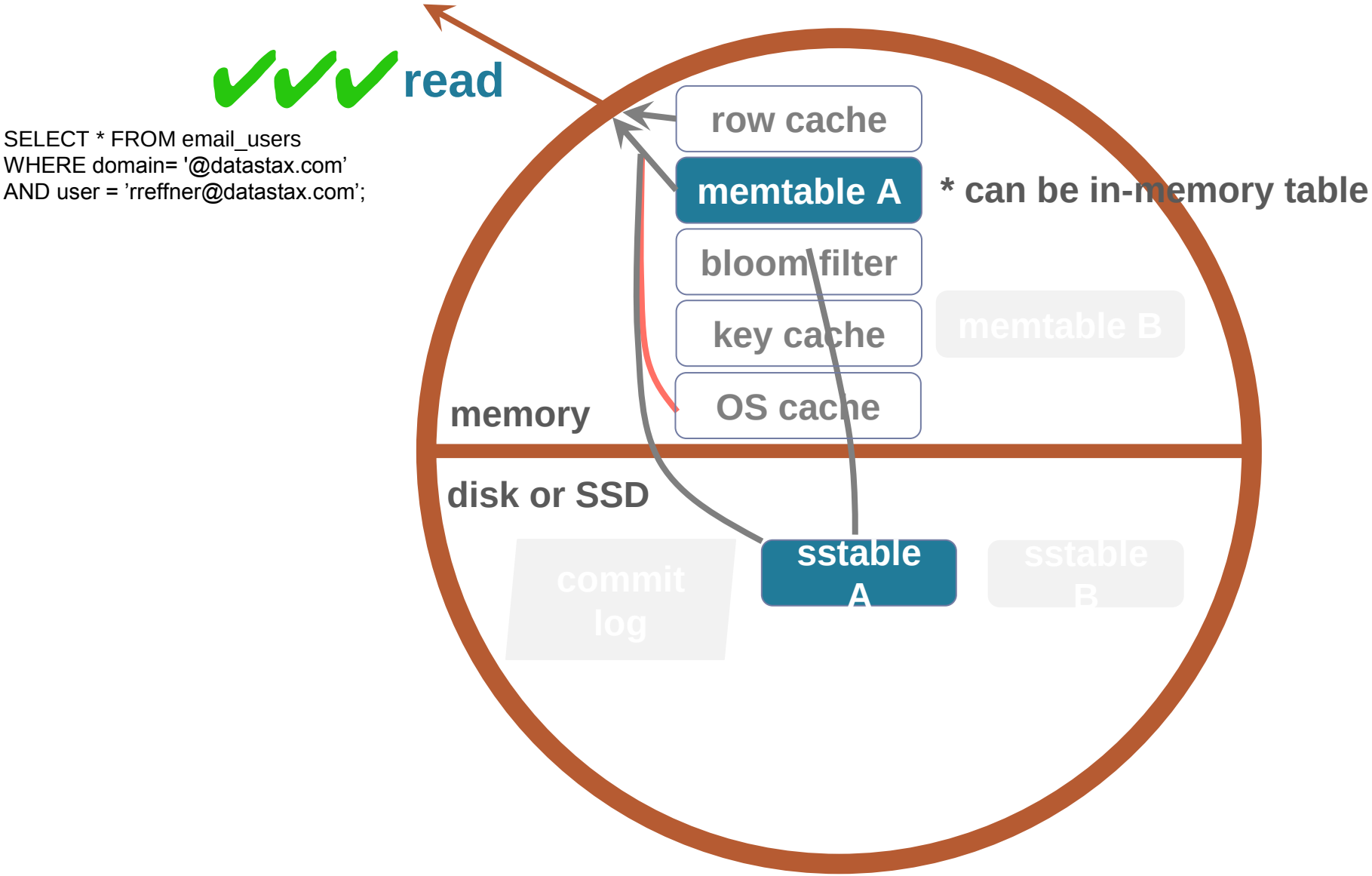
Read Path


read
SELECT * FROM email_users
WHERE domain= '@datastax.com'
AND user = 'rreffner@datastax.com';
row cache
memtable A
* can be in-memory table
bloom filter
key cache
memtable B
OS cache
memory
disk or SSD
commit log
sstable A
sstable B

Read Path


read
SELECT * FROM email_users
WHERE domain= '@datastax.com'
AND user = 'rreffner@datastax.com';
row cache
memtable A
* can be in-memory table
bloom filter
key cache
memtable B
OS cache
memory
disk or SSD
commit log
sstable A
sstable B

Read Path


read
SELECT * FROM email_users
WHERE domain= '@datastax.com'
AND user = 'rreffner@datastax.com';
row cache
memtable A
* can be in-memory table
bloom filter
key cache
memtable B
OS cache
memory
disk or SSD
commit log
sstable A
sstable B

Data Modeling Best Practices

Start with your data access patterns

- What does your app do?
- What are your query patterns?

Optimize for

- Fast writes and reads at the correct consistency
- Consider the results set, order, group, filtering
- Use TTL to manage data aging
- 1 Query = 1 Table
  - Each SELECT should target a single row (1 seek)
  - Range queries should traverse a single row (efficient)

Denormalization Is Expected

Remember: You're *not* optimizing for storage efficiency

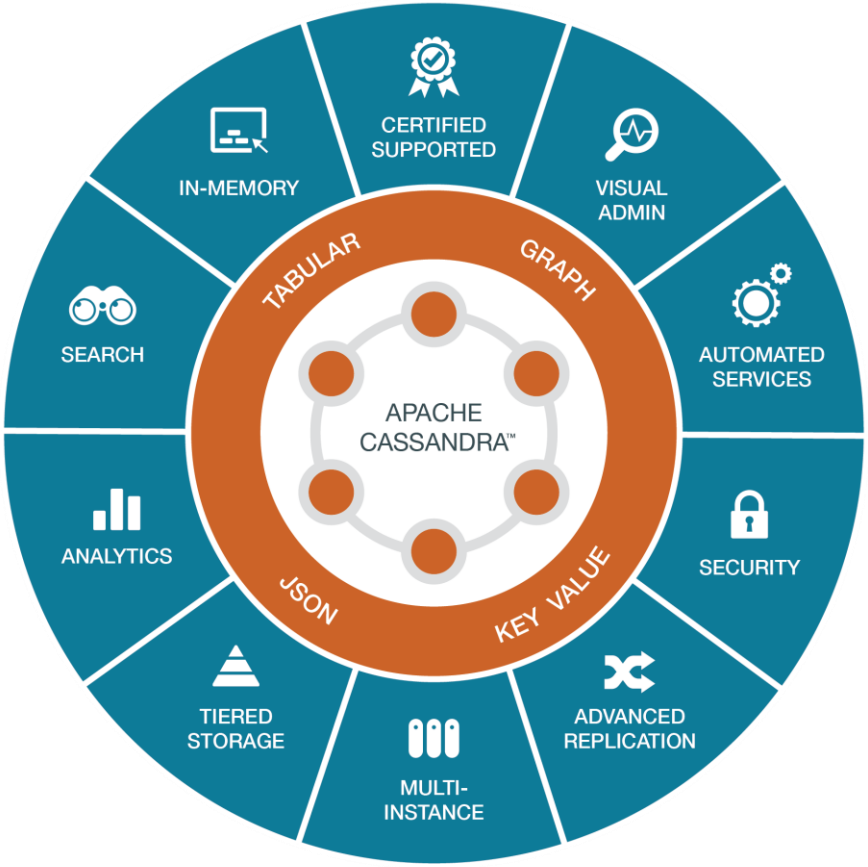
You are optimizing for *performance*

Do this:

- Forget what you've learned about 3rd normal form
- Repeat after me… "slow is down", "storage is cheap"
- Denormalize and do parallel writes!

Don't to this:

- Client side joins
- Reads before writes



DATASTAX ENTERPRISE

© 2016 DataStax, All Rights Reserved.

Thank You!
Questions?

Russ.Katz@Datastax.com

4

Debate

Strength and weakness of Cassandra.

Strengths (1)

- Linear scale performance

The ability to add nodes without failures leads to predictable increases In performance

- Supports multiple languages

Python, C#/.NET, C++, Ruby, Java, Go, and many more…

- Operational and developmental simplicity

There are no complex software tiers to be managed, so administration duties are greatly simplified.

Strengths (2)

- Ability to deploy across data centers

Cassandra can be deployed across multiple, geographically dispersed data centers

- Cloud availability

Installations in cloud environments

- Peer to peer architecture

Cassandra follows a peer-to-peer architecture, instead of master-slave architecture

Strengths (3)

- Flexible data model

Supports modern data types with fast writes and reads

- Fault tolerance

Nodes that fail can easily be restored or replaced

- High Performance

Cassandra has demonstrated brilliant performance under large sets of data

Strengths (4)

- Schema-free/Schema-less

In Cassandra, columns can be created at your will within the rows. Cassandra data model is also famously known as a schema-optional data model

- AP-CAP

Cassandra is typically classified as an AP system, meaning that availability and partition tolerance are generally considered to be more important than consistency in Cassandra

Weaknesses (1)

Use Cases where is better to avoid using Cassandra

- If there are too many joins required to retrieve the data
- To store configuration data
- During compaction, things slow down and throughput degrades
- Basic things like aggregation operators are not supported
- Range queries on partition key are not supported

Weaknesses (2)

Use Cases where is better to avoid using Cassandra

- If there are transactional data which require 100% consistency
- Cassandra can update and delete data but it is not designed to do so

Thanks!

Any questions?